

Springer Series in Supply Chain Management

Maxime C. Cohen
Paul-Emile Gras
Arthur Pentecoste
Renyu Zhang

Demand Prediction in Retail

A Practical Guide to Leverage Data and
Predictive Analytics



Springer

Springer Series in Supply Chain Management

Volume 14

Series Editor

Christopher S. Tang, University of California, Los Angeles, CA, USA

Supply Chain Management (SCM), long an integral part of Operations Management, focuses on all elements of creating a product or service, and delivering that product or service, at the optimal cost and within an optimal timeframe. It spans the movement and storage of raw materials, work-in-process inventory, and finished goods from point of origin to point of consumption. To facilitate physical flows in a time-efficient and cost-effective manner, the scope of SCM includes technology-enabled information flows and financial flows.

The Springer Series in Supply Chain Management, under the guidance of founding Series Editor Christopher S. Tang, covers research of either theoretical or empirical nature, in both authored and edited volumes from leading scholars and practitioners in the field – with a specific focus on topics within the scope of SCM.

Springer and the Series Editor welcome book ideas from authors. Potential authors who wish to submit a book proposal should contact Ms. Jialin Yan, Associate Editor, Springer (Germany), e-mail: jialin.yan@springernature.com

More information about this series at <http://www.springer.com/series/13081>

Maxime C. Cohen • Paul-Emile Gras •
Arthur Pentecoste • Renyu Zhang

Demand Prediction in Retail

A Practical Guide to Leverage Data
and Predictive Analytics

Maxime C. Cohen
Desautels Faculty of Management
McGill University
Montreal, QC, Canada

Paul-Emile Gras
Virtuo Technologies
Paris, France

Arthur Pentecoste
Boston Consulting Group GAMMA
New York, NY, USA

Renyu Zhang
New York University Shanghai
Shanghai, China

ISSN 2365-6395

ISSN 2365-6409 (electronic)

Springer Series in Supply Chain Management

ISBN 978-3-030-85854-4

ISBN 978-3-030-85855-1 (eBook)

<https://doi.org/10.1007/978-3-030-85855-1>

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Switzerland AG 2022

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.

The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.

The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG.
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

“To remain competitive in today’s economy, it is imperative for retailers to undertake a digital transformation. Having demand prediction capabilities is a crucial building block to optimize omnichannel marketing and operations. This book can serve as an invaluable guide on how to leverage data and AI to predict demand and is a must-have on the shelf of practitioners in the retail industry.”

—**Anindya Ghose**, *Heinz Riehl Chair Professor at NYU Stern School of Business and author of TAP: Unlocking the Mobile Economy*

“Predicting retail sales does not need to solely rely on experience and intuition anymore. The recent progress in predictive analytics provides great tools to help retailers predict demand. This book is instrumental for retailers who seek to embrace data-driven decision making.”

—**Georgia Perakis**, *William F. Pounds Professor at MIT Sloan School of Management*

“The key to success for many retailers lies in making sure that the right products are available at the right time in the right store. Failing to meet this goal may adversely affect customer loyalty and long-term profits. The only way to systematically succeed in this goal at scale is to rely on data and algorithms. This book is very pragmatic and explains how to leverage past data to predict future demand for retailers.”

—**Aldo Bensadoun**, *Founder and Executive Chairman of the Aldo Group*

“End-to-end retail decisions from procurement, capacity/inventory, distribution channel management to pricing and promotions crucially rely on robust demand prediction models, making this book vital for retailers. The content of this book is comprehensive yet remains accessible and actionable. An excellent reference and a must-read for data science enthusiasts as well as data science managers who have been changing the retail business as we know it.”

—**Özalp Özer**, *Senior Principal Scientist at Amazon, George and Fonsa Brody Professor at UT Dallas, and author of The Oxford Handbook of Pricing Management*

“For business analytics students and practitioners interested in understanding how to implement statistical demand forecasting models using Python, this book provides an invaluable hands-on approach, with detailed programming examples to guide the reader.”

—**Gerry Feigin**, *Partner and Associate Director at BCG GAMMA and author of Supply Chain Planning and Analytics and The Art of Computer Modeling for Business Analytics*

“Finally a book that methodically demystifies retail demand prediction has arrived. This is a must-read for any aspiring scientists looking to apply statistical and machine learning techniques to real-world demand prediction problems, as well as an excellent refresher for practitioners to stay current.”

—**Nitin Verma**, *Vice President, Digital Solutions and Chief Scientist at Staples*

Preface

In the last decade, the curriculum of both business schools and engineering schools has been significantly revamped. This is partly due to the proliferation of data-rich environments and to the development of novel data science methodologies. Several schools (as well as online programs) have started offering degrees and certifications in business analytics and data science. Given the growing demand of data-related skills, it has become ubiquitous for most companies to have open positions for data scientists and data analysts. Following these trends, many students and scientists are interested in sharpening their skills in data science and analytics. In this context, this book has two complementary motivations: (1) developing relevant and practical teaching material for the next generation of students in analytics and (2) helping practitioners leverage their data and embrace data science capabilities.

Having taught courses in operations and analytics at several universities, I have been constantly looking for real-world case studies that apply scientific methods and concepts to practical settings to use in my lectures. While I could easily find such case studies on several topics, I was surprised by the lack of comprehensive practical material on demand prediction for retail applications. Even more surprising was the fact that many applications and teaching material were relying on having access to a predicted demand function. For example, various tools and methods in supply chain management and inventory planning rely on having access to an accurate demand prediction model. Similarly, several applications in pricing, promotions, and assortment optimization consider demand models as an input. I could also find a myriad of textbooks and academic articles on demand prediction and on time-series forecasting, but these were mainly focusing on mathematical analyses by developing new methods and proving theoretical properties. Based on my experience, implementing a demand prediction method using real data to predict demand for a company involves a number of undocumented steps. This book aims to bridge this gap by covering the entire process of predicting demand for a retailer, starting from data collection all the way to evaluation and visualization. Importantly, we discuss many of the practical intermediate steps involved in the demand prediction process and provide the implementation code in Jupyter Notebooks.

Many companies (from small startups to large corporations) across various verticals are now routinely collecting large amounts of data. An important challenge faced by these companies is how to leverage these data to enhance operational and strategic decisions. In many settings, the first step involves developing data-driven demand prediction capabilities. Having a good demand prediction model can help manage inventory, decide prices and promotions, and guide several other tactical decisions. In fact, it has become common for retailers to have a dedicated team that focuses on demand prediction and planning. Retailers clearly understand the benefits and the competitive advantages of having strong demand prediction capabilities. Retailers also understand that the latter can only be accomplished by leveraging data and predictive analytics. As stated by Aldo Bensadoun, the founder and executive chairman of the Aldo Group: “The key to success for many retailers lies in making sure that the right products are available at the right time in the right store. Failing to meet this goal may adversely affect customer loyalty and long-term profits. The only way to systematically succeed in this goal at scale is to rely on data and algorithms.” This book attempts to provide a practical guide to help retailers leverage their historical transaction data to predict future demand. The material and methods rely on common models from statistics and machine learning. We wrote this book assuming that readers have limited knowledge on data science and statistics, but basic programming skills (preferably in Python) are required. We also provide references and textbooks on all the complementary subjects as well as several pointers to more advanced topics.

The content of this book was inspired by the consulting engagements the authors had in the last few years. Collectively, we have helped more than 20 retailers with predictive capabilities across several sectors and continents. While each retailer faces a unique problem with its own challenges, a large portion of the methodology and implementation details are common across retailers. In this book, our goal is to provide a practical guide by outlining the relevant steps to predict demand in retail applications. For each step, we include the relevant implementation details, so that readers can easily replicate the process and predict demand in their own business setting. We also provide a dataset to illustrate all the concepts and each step in the process. Having access to a dataset allows readers to assimilate the learning concepts while earning some hands-on experience.

In Chap. 1, we present an introduction on demand prediction by discussing the motivation, objective, and scope. We also describe our dataset and discuss several common prediction accuracy metrics. In Chap. 2, we focus on data preprocessing and elaborate on various relevant modeling factors. It is critical to complete these important steps before moving to predictive models and estimation. Several preprocessing tasks involve domain expertise and are sometimes overlooked in traditional textbooks. The concepts of feature engineering and feature selection (i.e., spending time to construct and select the right set of features to be included in the predictive model) are often the most crucial step in demand prediction. Even the best predictive model will fail if its input (predictive features) is not informative and well designed. In Chap. 3, we cover common demand prediction methods, including several variations of linear regressions. We also discuss an interesting

practical trade-off between data aggregation and demand prediction. In Chap. 4, we consider machine learning tree-based methods and explain how they can be applied in the context of demand prediction. We also describe the process of fine-tuning hyperparameters. Chapter 5 presents two common clustering techniques that can be instrumental in aggregating data across several products to ultimately improve the accuracy of demand prediction. Chapter 6 discusses potential ways to evaluate and visualize the prediction results. Chapter 7 considers two advanced methods (the Prophet method and a systematic data-driven approach to perform data aggregation). Finally, we outline our conclusions and discuss several advanced topics in Chap. 8. The dataset and notebooks used to assist the learning process can be accessed in the accompanying website (<http://www.demandpredictionbook.com/>).

Right before publishing this book, we tested its content in a course in the Master of Management in Analytics at the Desautels Faculty of Management at McGill University. It was unanimously well received by the students who felt that the material was highly valuable for job interviews and instrumental to start their first data science job. As one of the students testified: “The material in this book is the perfect embodiment of a successful training in data science. Before starting my Master, I told myself that it will be a worthwhile investment if by the end of the program, I will be able to take a real dataset and perform a predictive task from beginning to end. This book is the utmost enabler of mastering this skill.” We truly hope that this book will be useful to the next generation of data scientists and students in business analytics.

Montreal, QC, Canada

Maxime C. Cohen

Acknowledgments

We would like to sincerely thank all the people who supported the efforts deployed in writing this book. Special thanks go to Yossiri Adulyasak, Lennart Baardman, Nymisha Bandi, Emma Frejinger, Daniel Guetta, Warut Khern-am-nuai, and Niloofar Tarighat, who have carefully reviewed the content of this book and provided several helpful comments and suggestions. We also thank several retailers (the names are not disclosed due to confidentiality) for sharing valuable business knowledge over the past few years and for allowing us to test our approaches using their data. Finally, the support of McGill University and of the Bensadoun School of Retail Management is greatly acknowledged.

Contents

- 1 Introduction 1**
 - 1.1 Motivation 1
 - 1.2 Dataset 3
 - 1.3 Objective and Scope 9
 - 1.3.1 Training and Test Data 9
 - 1.3.2 Prediction Accuracy Metrics 10
 - 1.3.3 Application 11
 - References 12
- 2 Data Pre-Processing and Modeling Factors 13**
 - 2.1 Dealing with Missing Data 13
 - 2.2 Testing for Outliers 17
 - 2.3 Accounting for Time Effects 18
 - 2.4 Price and Lag-Prices 22
 - 2.5 Featured on Main Page 24
 - 2.6 Item Descriptive Features 25
 - 2.7 Additional Features 25
 - 2.8 Scaling 26
 - 2.9 Sorting and Exporting the Dataset 27
 - References 27
- 3 Common Demand Prediction Methods 29**
 - 3.1 Primer: Basic Linear Regression for One SKU 30
 - 3.2 Structuring the Dataset 32
 - 3.3 Centralized Approach 33
 - 3.4 Decentralized Approach 34
 - 3.5 Feature Selection and Regularization 36
 - 3.5.1 Subset Selection 36
 - 3.5.2 Lasso Regularization 42
 - 3.5.3 Ridge Regularization 46
 - 3.5.4 Elastic Net Regularization 47

3.6	Log Transformations	49
3.6.1	Log-Transformation on the Price Variable	49
3.6.2	Log-Transformation on the Target Variable	52
3.6.3	Transformations and Prediction Accuracy	54
3.7	Centralized Approach with SKU-Fixed Effects	56
3.8	Centralized Approach with Price-Fixed Effects	59
3.9	Centralized Approach with SKU-Price-Fixed Effects	62
3.10	Decentralized Approach with Aggregated Seasonality	64
3.11	Summary and Next Steps	66
	References	67
4	Tree-Based Methods	69
4.1	Decision Tree	70
4.1.1	Centralized Decision Tree	71
4.1.2	Decentralized Decision Tree	79
4.2	Random Forest	81
4.2.1	Centralized Random Forest	82
4.2.2	Decentralized Random Forest	84
4.3	Gradient-Boosted Tree	86
4.3.1	Centralized Gradient Boosted Tree	87
4.3.2	Decentralized Gradient-Boosted Tree	89
4.4	Methods Comparison	92
	References	92
5	Clustering Techniques	93
5.1	K-means Clustering	93
5.1.1	Description of K-means Clustering	93
5.1.2	Clustering using Average Price and Weekly Sales	97
5.1.3	Adding Standard Deviations of the Clustering Features	100
5.2	DBSCAN Clustering	103
5.2.1	Description of DBSCAN Clustering	103
5.2.2	Clustering using Average Price and Weekly Sales	108
5.2.3	Adding the Standard Deviation of the Clustering Features	113
	References	114
6	Evaluation and Visualization	115
6.1	Summary of Results	115
6.2	Prediction vs. Actual	118
6.3	Varying the Split Ratio	123
7	More Advanced Methods	129
7.1	The Prophet Method	129
7.1.1	What is the Prophet Method?	129
7.1.2	Forecasting with Prophet	135
7.2	Data Aggregation and Demand Prediction	143
7.2.1	Presentation of the DAC Method	143

- 7.2.2 Fine-Tuning the Hyperparameters 146
 - 7.2.3 Interpretating the DAC Results 147
 - References 149
- 8 Conclusion and Advanced Topics 151**
 - References 155

About the Authors

Maxime C. Cohen is the Scale AI professor of retail and operations management, co-director of the Retail Innovation Lab, and a Bensadoun Faculty Scholar at McGill University. Maxime is also a scientific advisor in AI and data science at IVADO Labs and a scientific director at the non-profit MyOpenCourt.org. Before joining McGill, he was a faculty at NYU Stern and a research scientist at Google AI. His core expertise lies at the intersection of data science and operations. He has collaborated with Google, Waze, Oracle Retail, IBM Research, Via, Spotify, Aldo Group, Circle K, and Staples and serves on the advisory boards of several startups. Maxime has extensive experience leveraging large volumes of data to predict demand and to develop data-driven decision tools. He holds a PhD in operations research from MIT and a BS and MS from the Technion.

Paul-Emile Gras is a data scientist at Virtuo Technologies in Paris. His expertise is at the interface of demand forecasting and revenue management. Prior to joining Virtuo, he was a research assistant in operations research at McGill University. He holds a Master in Business Analytics from McGill University and a Bachelor and Master of Engineering from Ecole Centrale Paris.

Arthur Pentecoste is a data scientist at the Boston Consulting Group GAMMA. Arthur's main scope of expertise is in predictive modeling and analytics applied to demand forecasting and predictive maintenance. He was a research assistant in operations research at McGill University. He obtained a Master in Business Analytics from McGill University and a Bachelor and Master of Engineering from Ecole Centrale Paris.

Renyu Zhang is an assistant professor of operations management at New York University Shanghai and a visiting Associate Professor at the Chinese University of Hong Kong. He is also an economist and Tech Lead at Kuaishou, one of the world's largest online video-sharing and live-streaming platforms. Renyu is an expert in data science and operations research. He obtained his PhD in operations management at Washington University in St. Louis, and his BS in mathematics at Peking University.

Chapter 1

Introduction



This book intends to cover the entire process of predicting demand for retailers. Specifically, we will go over all the steps, starting from data pre-processing and exploration, all the way to evaluating the accuracy of the prediction algorithms. We will present several commonly used methods for demand prediction and discuss various useful implementation details. Each step will be illustrated with the relevant code portion in Python. To assist this learning experience, we will use a dataset available for download. More details on this dataset will be discussed in Sect. 1.3.

1.1 Motivation

Demand prediction or demand forecasting is at the forefront of most retailers' priorities. Being able to accurately predict future demand for each product in each time period (e.g., day, week) can be instrumental for guiding retailers with their operational decisions (e.g., inventory and supply chain management) and, ultimately, boosting profitability. Recent advances in information technology and computing provide tremendous opportunities for demand prediction. It has become ubiquitous to collect transactional data and develop methods to exploit these data for a wide range of retailers, from electronics to fast fashion to food-delivery services. As of 2018, the potential value created by applying artificial intelligence (AI) and data analytics on the retail industry, including consumer packaged goods, was estimated to be 1.26T\$ annually.¹

An important retail application of demand prediction is to improve inventory management decisions. An accurate forecast offers the ability to anticipate and be prepared for unexpected demand surges. Specifically, accurate demand prediction

¹<https://www.mckinsey.com/featured-insights/artificial-intelligence/visualizing-the-uses-and-potential-impact-of-ai-and-other-analytics>.

can help avoid stockouts, which can have adverse effects in terms of customer satisfaction and retention. At the same time, accurate demand prediction can mitigate excessive stock levels, which are often cost prohibitive for retailers. Furthermore, having a good demand prediction system in place can help retailers sharpen their understanding of consumers in terms of preferences, substitution patterns, seasonality, and elasticities to price discounts. It can thus be used to guide marketing campaigns and promotion strategies. Overall, being able to accurately predict demand will often translate into both increasing revenue and decreasing costs.

In recent years, the ability to collect large volumes of granular data in real time has disrupted the way that decisions are made in several industries, and retail is no exception. Both brick-and-mortar and online retailers routinely record precious information about their transactions and their customers. Traditional features include prices, discounts, volume of sales, and planogram arrangements (i.e., where products are located on the shelves or on the website). A recent trend has emerged in which retailers are also collecting modern data sources, such as foot-traffic information, clickstream data, social-media activities, and dwell times (i.e., how long customers stay in the store or browse each website page). The amount of available data is beyond limits. For example, as of 2017, Walmart was processing 2.5 petabytes of data every hour, coming from 200 streams of internal and external data sources.² These enormous volumes of data offer opportunities to develop data-driven methods that guide operational decisions. These methods are often based on machine learning and AI and are now used by many retailers across a wide range of verticals.

While these data can be used with several goals in mind, a traditional application is demand prediction. In fact, a large number of studies in both academia and industry have focused on developing data-driven demand-prediction methods.³ At the same time, it is hard to find a practical guide that outlines the different steps to follow and the mistakes to avoid. This book aims to bridge this gap by providing a practical guide for retail demand prediction along with detailed coding documentation. The codes are written in a generic fashion, so that one can readily use them in various business settings.

In this book, we will investigate the topic of demand prediction from a practical perspective. We will cover the entire end-to-end process, starting from data collection all the way to testing and visualization. While we cannot consider all existing approaches in the context of demand prediction, we will cover and implement more than a dozen different methods. The methods we consider are relatively simple and leverage classical machine-learning techniques. We will not cover time-series methods, which are also commonly used in the context of demand prediction (we will still briefly mention this type of methods along with relevant references and point out the differences with the methods we cover). We will discuss how to test the prediction accuracy of the different methods and visualize the results. Finally, we

²<https://www.forbes.com/sites/bernardmarr/2017/01/23/really-big-data-at-walmart-real-time-insights-from-their-40-petabyte-data-cloud/?sh=1fee8d416c10>.

³See e.g., Chase (2013).

will present several implementation details that are important in practice. Overall, this book aims to provide a basic step-by-step guide on how to predict demand in retail environments. The process includes implementing the different methods using a dataset inspired by actual retail data. Conveniently, we illustrate all the concepts and methods using the same dataset. We divide the process into the following modules:

- Data processing and modeling factors
- Common demand prediction methods (including feature selection and regularization)
- Tree-based methods
- Clustering techniques
- Evaluation and visualization
- Extensions (the Prophet method and advanced topics)

The content of this book is modular, hence allowing one to complete most modules in isolation. After mastering the content of this book, readers will be equipped with the basic skills related to data-driven demand prediction. The material is not tied to a specific type of retailer and can be applied to a multitude of retail applications. Of course, depending on the setting under consideration, several adjustments will be required (such adjustments as well as several more advanced topics are beyond the scope of this book and will be discussed in the conclusion section). Ultimately, the material discussed in this book can be useful for retailers who have access to historical sales data and are interested in predicting the future demand for their products.

1.2 Dataset

The data we use to guide our demand prediction process was originally provided by an online electronics retailer. For confidentiality purposes, we have anonymized the data and performed a series of slight modifications. The dataset can be downloaded in a csv format by using the following link:⁴

<https://demandpredictionbook.com>

This dataset reports the weekly sales of a tech-gadget e-commerce retailer over a period of 100 weeks, from October 2016 to September 2018. It includes the weekly sales of 44 items, also called stock-keeping units (SKUs), as well as diverse information on these SKUs.

We highlight that the demand prediction process presented in this book can be applied to various settings and is not restricted to tech-gadget e-commerce retailers. Specifically, it can be applied to e-commerce and brick-and-mortar retailers alike,

⁴If this link does not work, an alternative link is <https://demandprediction.github.io/>.

across several verticals. Of course, depending on the business setting under consideration, one needs to slightly adapt the methods and the set of features used for demand prediction. We aim to keep the treatment as generic as possible and agnostic to the specific application.

As discussed, retailers routinely collect and store data related to customer transactions. The raw data often consists of a collection of isolated transactions. For example, each row in the raw dataset can correspond to a specific transaction that includes several fields, such as timestamp, price, store (in the case of a centralized dataset across multiple stores), promotion information, loyalty card information (if available), SKU-related features (e.g., color, brand, size), and customer information (e.g., past purchases, clicks). The first step is to aggregate the transaction-level data into a more compact dataset. For example, one can aggregate the data at the day or week level. In this case, all the transactions that occur on the same day (or week) will be merged into a single observation. A second dimension for data aggregation is to decide whether to leave all the SKUs in their own right or to combine different SKUs (e.g., at the brand level or even at the category or subcategory level). Finally, a third common aggregation dimension is to decide whether to aggregate the data from different stores or to consider the data from each store separately. The correct way of aggregating the data depends on the context, on the size of the available data, and on the data variation. A more granular approach will preserve intrinsic characteristics of the data, whereas a more aggregated approach will allow us to reduce the noise in the data (at the expense of not retaining some of the characteristics). For example, there is a clear trade-off between aggregating at the day versus week level. A weekly aggregation will lose the intra-week variation (e.g., sales are often higher on weekends than on weekdays), but it will average over a larger number of transactions for each observation and, hence, result in a less noisy dataset. Unfortunately, there is no one-fits-all answer to this question. In this book, we will consider the case where the data is aggregated at the week-SKU level (i.e., each row in our dataset corresponds to a specific SKU for a specific week).

The code used to implement the different prediction methods and test their performance on the dataset will be presented, along with detailed explanations. Complete Jupyter notebooks are also available in the companion website.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 1/Introduction.ipynb
- data_raw.csv

While the code can be run with any Python development environment, one possible suggestion for new users is to leverage free tools such as Google Colab.⁵ We provide basic Python setup guidelines in the companion website. We highlight that to fully leverage the learning experience presented in this book, it is highly recommended to have some fundamental coding Python skills. One can easily find

⁵<https://colab.research.google.com/notebooks/intro.ipynb>.

	week	sku	weekly_sales	feat_main_page	color	price	vendor	functionality
0	2016-10-31	1	135.0	True	black	10.16	6	06.Mobile phone accessories
1	2016-11-07	1	102.0	True	black	9.86	6	06.Mobile phone accessories
2	2016-11-14	1	110.0	True	black	10.24	6	06.Mobile phone accessories
3	2016-11-21	1	127.0	True	black	8.27	6	06.Mobile phone accessories
4	2016-11-28	1	84.0	True	black	8.83	6	06.Mobile phone accessories
...	...	...	...	...	...	...	...	...
4395	2018-08-27	44	20.0	False	black	53.99	6	09.Smartphone stands
4396	2018-09-03	44	14.0	False	NaN	52.99	6	09.Smartphone stands
4397	2018-09-10	44	22.0	True	black	44.99	6	09.Smartphone stands
4398	2018-09-17	44	28.0	True	NaN	42.99	6	09.Smartphone stands
4399	2018-09-24	44	26.0	True	black	43.45	6	09.Smartphone stands

4400 rows × 8 columns

Fig. 1.1 Raw dataset exploration

tutorials, online courses, or books to acquire such Python skills.⁶ For a general reference on how to leverage data science for business applications, we refer the reader to this book.⁷

Let's start by taking a look at the dataset and performing high-level exploratory analyses. We next import the pandas library:

```
import pandas as pd
sales = pd.read_csv('data_raw.csv', parse_dates=['week'])
sale
```

The output is presented in Fig. 1.1. As we can see, the dataset comprises of 4400 rows and eight columns. Each row corresponds to a SKU-week pair (44 SKUs for 100 weeks), whereas each column corresponds to a feature.

The features of this dataset are described below (from left to right in Fig. 1.1):

- *Week*: The dataset covers all weeks from 2016-10-31 to 2018-09-24. In total, we have 100 weeks (i.e., approximately two years of data). We identify each week by the corresponding Monday.
- *SKU*: There are 44 SKUs, indexed from 1 to 44. In total, the dataset has $44 \times 100 = 4400$ rows.
- *Featured on the main page*: To boost the sales of specific products, the marketing team may decide to broaden their visibility by featuring these products on the website's homepage. We then record for each week and SKU whether this was the case (i.e., binary indicator).

⁶See, e.g., Matthes (2019), Downey (2012), and McKinney (2012).

⁷Provost and Fawcett (2013).

Table 1.1 Summary statistics for the numerical variables (weekly sales and price)

Metric	Weekly_sales	Price
mean	83.05	44.43
standard deviation	288.00	42.50
min	0	2.39
max	7512	227.72

- *Color*: There are nine different colors among the products: black, gold, pink, blue, red, grey, green, white, and purple. In addition, two SKUs have the value “none” for this feature, meaning that their color is not defined. Specifically, either the product is multi-color or it does not have a specific color (e.g., internal parts of a computer often do not have a specific color) or the data is missing.
- *Price*: The price is fixed for each item during a given week.⁸ The pricing team can adjust the price on a weekly basis based on various considerations (e.g., promotional events, excess amounts of inventory).
- *Vendor*: The company acts as a retailer for electronics brands. The vendor variable refers to the product brand. The SKUs in our dataset span 10 different vendors. For confidentiality reasons, we have masked the vendors’ names and indexed them from 1 to 10.
- *Functionality*: The functionality is the main function or description of the SKU. There are 12 different functionality values in our dataset. For illustration purposes, we assign names of different product categories to each functionality (these labels are meant to be illustrative and do not correspond to the true labels in the original data). Specifically, our 12 different functionalities correspond to the following categories: streaming sticks, portable smartphone chargers, Bluetooth speakers, selfie sticks, Bluetooth tracker, mobile phone accessories, headphones, digital pencils, smartphone stands, virtual reality headset, fitness trackers, and flash drives.

Finally, we also have access to the *weekly_sales* variable, which is the number of items sold during the focal week for the corresponding SKU. This is the variable to be predicted, often called the target or outcome variable.

The first column on the left simply corresponds to the index of the row in the dataset. In Tables 1.1–1.3, we present high-level summary statistics of the variables included in the dataset. It is common to compute and inspect the main statistics (mean, standard deviation, minimum, and maximum) of each variable in the dataset. In Fig. 1.2, we plot the average sales (top panel) and price (bottom panel) time series, where the average is taken over all 44 SKUs. Finally, in Fig. 1.3, we compare the weekly sales of two SKUs with different weekly sales patterns (both in terms of magnitude and volatility). We encourage the reader to create a habit of conducting data exploration (i.e., generating several relevant plots and examining the variables’

⁸If the price (or any other variable) varies during the week, one can compute the resulting weighted average value (where the weights can be based on the sale volumes or on the revenues).

Table 1.2 Breakdown of values for the variable “featuring on the main page”

Feat_Main_Page	Count
False	2825
True	1575

Table 1.3 Breakdown of values for the color variable

Color	Count
black	1691
blue	700
red	500
green	400
grey	300
white	200
none	200
gold	199
purple	100
pink	100



Fig. 1.2 Average sales and price time series

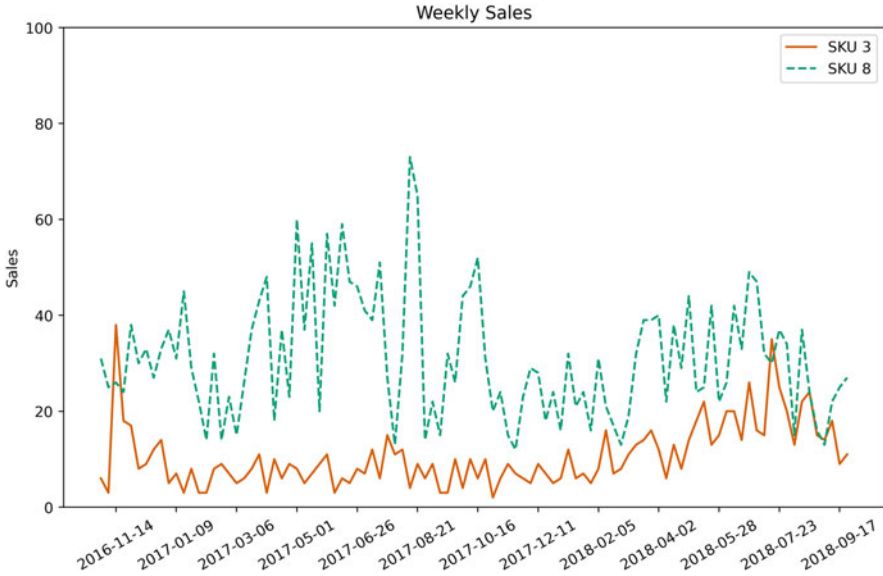


Fig. 1.3 Weekly sales for two specific SKUs (SKUs 3 and 8)

distribution) prior to any further analysis. All codes used to generate the tables and figures below can be found in the data exploration notebook:

<https://demandpredictionbook.com>

As we can see from Fig. 1.2, the average weekly sales and price time series are good illustrations of the variation observed in the sales and price variables. At this point, it is unclear how these two variables will interact. However, it should be clear that the price value affects the sales, so that a higher price typically leads to a lower level of sales. More formally, one can potentially compute the correlation value (e.g., Pearson correlation coefficient) between these two variables. Throughout this book, we will assume that demand and sales refer to the same concept (and we will use these two terms interchangeably). In practice, this means that the retailer can satisfy all the demand by having sufficient inventory. In several applications, however, this assumption may not be satisfied, so that inventory is limited. In such cases, sales will be equal to the minimum between demand and available inventory. Since our goal is to estimate demand by observing sales, it would create an additional complication, called data censoring (i.e., the fact that the observed sales are a truncated version of the actual consumer demand). Several techniques exist to address this issue but are beyond the scope of this book. A brief discussion on this topic is reported in the conclusion section, along with several references.

In Fig. 1.3, we plot the weekly sales for two specific SKUs (SKU 3 and SKU 8). As we can see, both the magnitude and the variance of the weekly sales can be substantially different from one SKU to another.

1.3 Objective and Scope

In this section, we first discuss the concepts of training and test sets that allow us to systematically split the data before estimating and evaluating any specific method. We then present four common prediction accuracy metrics, which can be used to assess the performance of any demand prediction method. Finally, we outline the specific goal and application related to the associated dataset.

1.3.1 Training and Test Data

To create a credible way for model evaluation, a common procedure is to split the available data into two sets: training and test. One can do so in (at least) two possible ways: random split and time-based split.

- *Random split*: As the name indicates, the split is performed in a random fashion. One advantage of this method is that we can perform multiple splits. If we construct several random training-test splits, then it allows us to use a cross-validation procedure (i.e., a resampling procedure used to evaluate predictive models on a limited data sample) and compute confidence intervals when comparing different methods. Having the ability to perform several splits will ultimately increase the robustness and confidence of the results. However, by performing a random split, we lose the temporal structure of the data.

Remark: In practice, the split may also incorporate additional requirements. For example, one may want to balance the training and test sets across relevant factors (e.g., preserving the same number of offered SKUs or the same proportion of promotional events). These additional requirements can be incorporated when designing a balanced split. This type of splits is called stratified sampling and can be done by using the following `StratifiedShuffleSplit` function in the `sklearn` package.⁹ Note that stratified split is more appropriate when using a random split and less for a time-based split.

- *Time-based split*: This split relies on separating the data based on a specific date. One advantage of this method is that it allows us to retain the temporal structure and chronology of the data. It is also in harmony with the task of demand prediction that aims to predict future sales by using historical data. It can thus be important to preserve the time ordering. An extension of the time-based split is to consider a sliding window, where the first T_1 time periods are used for training and the next T_2 time periods are used for testing. Then, one can slide the window forward in time in order to obtain several training-test datasets.

⁹https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.StratifiedShuffleSplit.html.

Remark: When the different SKUs are introduced at different dates in the data (which is often the case in practice), one needs to be careful when applying a time-based split. Two alternatives come to mind: (i) splitting the data for each item separately or (ii) using an absolute uniform date to split the data.

In this book, we will perform a time-based split.

1.3.2 Prediction Accuracy Metrics

Our goal is to use the historical data to predict future demand as accurately as possible. To measure the demand accuracy, several metrics exist. Four commonly used metrics to monitor the predictive performance are the following:

- R-squared (R^2)
- Mean absolute percentage error (MAPE)
- Mean absolute error (MAE)
- Mean squared error (MSE)

The above metrics aim to capture the accuracy of the predicted values relative to the realized values (in our case, actual sales). We note that no metric is perfect and that many other metrics have been proposed. For conciseness, we will mainly focus on the R^2 (although most of our qualitative results also hold for the other three metrics). In fact, each metric has its pros and cons, so considering several metrics is often desirable. For each prediction method, we will split our data into a training set and a test set and compute the R^2 on the test set, which is called the out-of-sample (OOS) R^2 . More details about this splitting procedure will be discussed in the sequel.

The R^2 formula is given by:

$$R^2 = 1 - \frac{RSS}{TSS}.$$

where the residual sum of squares (RSS) is given by $\sum_{i=1}^I \sum_{t=1}^T (r_{i,t} - \hat{r}_{i,t})^2$ and the total sum of squares (TSS) is given by $\sum_{i=1}^I \sum_{t=1}^T (r_{i,t} - \bar{r})^2$. Here, the index i is across all the SKUs in the dataset, whereas the index t corresponds to time periods. The variable $r_{i,t}$ corresponds to the actual sales value for SKU i at time t , $\hat{r}_{i,t}$ the forecasted value, and $\bar{r}$ the average value across all SKUs and all time periods. All three quantities should be computed based on the same dataset (e.g., on the test set).

The above formula leads to:

$$R^2 = 1 - \frac{\sum_{i=1}^I \sum_{t=1}^T (r_{i,t} - \hat{r}_{i,t})^2}{\sum_{i=1}^I \sum_{t=1}^T (r_{i,t} - \bar{r})^2}.$$

The value of R^2 is capped by one but can have any value below one. Obviously, the closer the R^2 is to 1, the better the performance is.

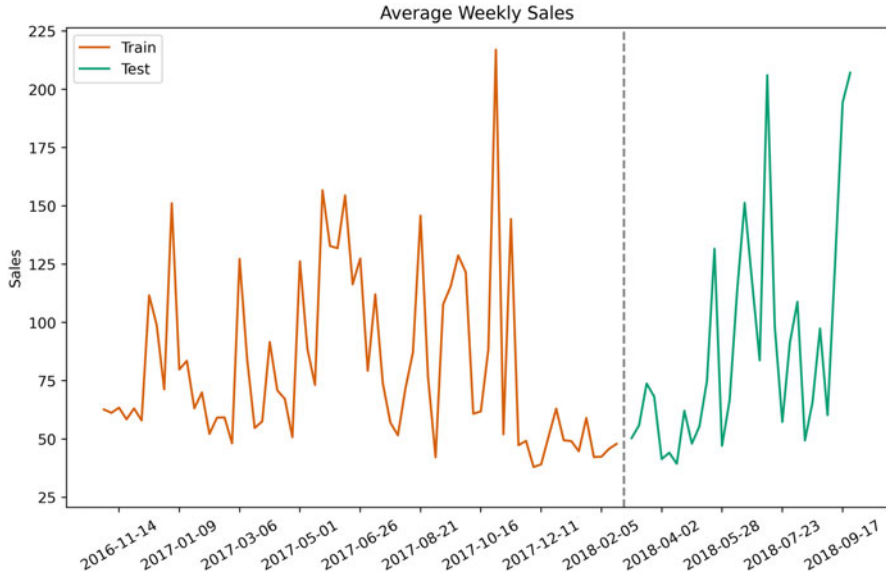


Fig. 1.4 Average weekly sales. The training (test) set is on the left (right) of the vertical line

Remark: We note that it is possible to find negative values for the OOS R^2 . It can happen when the predictions are of low quality and perform worse than the average out-of-sample value. Since we do not know the average value over the test set up front, it is thus possible to obtain negative R^2 values when using the test set.

We highlight that instead of predicting demand, one may be interested in predicting events related to demand values. For example, it is common to predict the probability that demand exceeds (or is below) a specific value or a set of values. Such considerations become relevant when making capacity decisions.

1.3.3 Application

From now on, our goal is to help a retail manager who is interested in predicting future sales (or demand) of the various SKUs offered. We will use the first 70 weeks as our training data. Our goal is to predict the sales for the last 30 weeks (testing data) and attain the highest possible R^2 value. Note that we use a ratio of 70:30 for splitting our data (while this is a common ratio, other ratios such as 75:25 and 2/3:1/3 are also often used). The specific ratio to be used depends on the size of the data and is often guided by a trial-and-error procedure. In Fig. 1.4, we plot the average weekly sales (the vertical line corresponds to the separation between the training and test sets).

References

- Chase, C.W., 2013. Demand-driven forecasting: a structured approach to forecasting. John Wiley & Sons.
- Downey, A., 2012. Think Python. O'Reilly Media, Inc.
- Matthes, E., 2019. Python crash course: A hands-on, project-based introduction to programming. no starch press.
- McKinney, W., 2012. Python for data analysis: Data wrangling with Pandas, NumPy, and IPython. O'Reilly Media, Inc.
- Provost, F. and Fawcett, T., 2013. Data Science for Business: What you need to know about data mining and data-analytic thinking. O'Reilly Media, Inc.

Chapter 2

Data Pre-Processing and Modeling Factors



Before implementing a statistical or a machine-learning method, it is crucial to process the raw data in order to extract as much predictive power as possible from the features available in the data. We will discuss several key concepts from data pre-processing and feature engineering that are relevant for demand prediction. The final processed dataset is also provided on the website under *data_processed.csv*.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 2/Data Pre-Processing and Modeling Factors.ipynb
- data_raw.csv

2.1 Dealing with Missing Data

Missing data is a very common issue in data science. Whether it comes from a human mistake or from the data acquisition process, there are several ways to deal with NaN values. Below are three simple practical methods, from the easiest to the hardest:

1. Simply deleting the rows with missing data.
2. Replacing the missing data by a fixed value (e.g., 0, 1, median value of the column, average value of the column).
3. Imputing the missing values by using business rules or by developing a predictive model (i.e., estimating a data-driven model to predict the missing values).

We start by identifying which features have missing data in our dataset:

	week	sku	weekly_sales	feat_main_page	color	price	vendor	functionality
802	2016-11-14	9	54.0	True	NaN	139.44	9	11.Fitness trackers
803	2016-11-21	9	71.0	True	NaN	141.16	9	11.Fitness trackers
4133	2017-06-19	42	4.0	False	NaN	27.33	10	09.Smartphone stands
4196	2018-09-03	42	8.0	False	NaN	42.99	10	09.Smartphone stands
4197	2018-09-10	42	14.0	True	NaN	42.99	10	09.Smartphone stands
4200	2016-10-31	43	5.0	True	NaN	109.99	9	11.Fitness trackers
4314	2017-02-06	44	5.0	False	NaN	53.99	6	09.Smartphone stands
4391	2018-07-30	44	34.0	True	NaN	41.99	6	09.Smartphone stands
4396	2018-09-03	44	14.0	False	NaN	52.99	6	09.Smartphone stands
4398	2018-09-17	44	28.0	True	NaN	42.99	6	09.Smartphone stands

Fig. 2.1 Observations with missing data for the feature color

```
sales.isna().any()
```

We obtain the following output:

```

week                False
sku                 False
weekly_sales        False
feat_main_page      False
color               True
price              False
vendor             False
functionality       False

```

As we can see, the feature `color` is the only one that contains missing data. We next dive deeper to identify the observations with missing values:

```
sales[sales['color'].isnull()]
```

The output of the above command is presented in Fig. 2.1.

We identify four SKUs (SKUs 9, 42, 43, and 44) with at least one missing value for the feature `color`. For illustration purposes, we focus on SKU 44 to see what are the most recurring colors for this specific product:

```
sales[sales.sku == 44]['color'].value_counts(dropna=False)
```

Remark: The command `dropna=False` allows us to count the number of NaN values (the default is set to True).

Here is the output we obtain:

```
black  96  
NaN    4
```

In this case, the missing values seem to come from a simple omission. A reasonable inference is that SKU 44 is only sold in the black color. We observe the same pattern for SKUs 9, 42, and 43.

Method 1: Deleting rows with missing values

```
sales.dropna()
```

This is naturally the fastest method, but it reduces the size of the dataset. Thus, it is often desirable to avoid this method, especially for small datasets (and when the number of missing values is significant).

We next discuss in more detail the second and third methods mentioned above.

Method 2: Manual replacement method

One can perform a manual replacement for each missing value:

```
sales.at[1, 'color'] = 'black'  
sales.at[3, 'color'] = 'black'  
sales.at[8, 'color'] = 'black'  
sales.at[85, 'color'] = 'black'
```

This process can be tedious if we were to repeat it for a large number of rows with missing data. We next consider the alternative method, which is based on systematically imputing values by using libraries or customized functions.

Method 3: Imputation libraries

We next present an imputation library that simplifies the process:

```
import numpy as np  
from sklearn.impute import SimpleImputer  
imputer = SimpleImputer(missing_values=np.nan,  
                        strategy='most_frequent')
```

One can find more information about this library in the `sklearn SimpleImputer` documentation.¹ We apply the `most_frequent` strategy,

¹https://scikit-learn.org/stable/modules/generated/sklearn.impute.SimpleImputer.html#sklearn.impute.SimpleImputer.fit_transform.

which imputes the missing values by using the most common value observed in the dataset.

For each SKU with missing data, we first fit the imputer and then apply it to the rows that contain missing data:

```
imputer.fit(sales[sales.sku==44][['sku', 'color']])

imputer.transform(sales[(sales.sku==44) & (sales.color.isna() )][['sku', 'color']])
#apply it on missing rows
```

To incorporate these modifications to the dataset, we iterate through the index of missing values for each SKU and apply the appropriate replacement:

```
missing_idx_44 = sales[(sales.color.isna()) & (sales.sku==44)].index.values
```

The above command returns the following:

```
[4314 4391 4396 4398]
```

Then, for each of these indices, we impute the missing values as follows:

```
for i in missing_idx_44:
    sales.at[i, 'color'] = imputer.transform(sales[(sales.sku==44) & (sales.color.isna() )][['sku', 'color']])[0,1]
```

After repeating this process for SKUs 9, 42, and 43 (omitted for conciseness), we can verify that there are no missing color values anymore:

```
sales[sales['color'].isnull()]
```

Remark: In our case, the missing data seem to come from data collection anomalies, and we can make simple assumptions to solve the issue. However, one can imagine a situation with different colors for a SKU, with several missing values. In such a case, whether to replace it with the most frequent value is an important question, and one must carefully design the appropriate imputing strategy. One can directly specify such a strategy into the imputer (e.g., mean, median, most_frequent, constant). As discussed, for more sophisticated cases, it is possible to develop a prediction method (e.g., a regression or a classification model) to be used as the imputer of missing data values. For example, the k-nearest neighbors algorithm

(k-NN)² is commonly used to identify “neighbors” of observations with missing data and then replace the missing values by the mean values from the identified neighbors.

2.2 Testing for Outliers

Another key pre-processing step is to inspect and (potentially) remove abnormal data points, also called outliers. There are various statistical methods to detect outliers. However, removing outliers or modifying their value should be done with care and requires understanding both the data collection process and the business context.

Below we define a function that will compute for each SKU the mean and standard deviation of a selected set of features. To identify statistically abnormal data points, one can compute lower and upper thresholds. For example, we can set the lower and upper thresholds to be $mean - k * standard\ deviation$ and $mean + k * standard\ deviation$, respectively. Then, if the observation is outside these boundaries, we will flag it as an outlier. We note that an alternative way to detect outliers is by using a specific quantile (e.g., 90- or 99-percentile).

```
def check_outliers(df, features, k=5):
    data = df.copy()
    for f in features:
        data['outlier_'+f] = data.groupby('sku')[f].transform(
            lambda x: (x > (x.mean()+k*x.std())) | (x < (x.mean()-k*x.std()))
        )
    return(data)
```

The above code performs a groupby operation on each SKU and tests whether the observation is outside the boundaries. The result of the test is a Boolean variable that will be stored in the column `outlier_<name of feature>`.

```
data.groupby('sku')[f].transform(
    lambda x: (x > (x.mean()+k*x.std())) | (x < (x.mean()-k*x.std())) )
```

We perform the outlier check on our dataset by focusing on the price feature and on the weekly_sales. Here, we decide to use k=5 to define the outlier boundaries.

```
df = check_outliers(sales, ['price', 'weekly_sales'], 5)
df[df.outlier_price]
```

²<https://scikit-learn.org/stable/modules/generated/sklearn.impute.KNNImputer.html>.

The output for the price variable looks as follows (Table 2.1):

As we can see, we identify six outliers for the price feature and 26 outliers for the target variable (weekly_sales). Of course, if we decrease the value of k , more outlier candidates will be identified. We next output the first six outliers (out of the 26 identified) in terms of weekly_sales (Table 2.2):

```
df[df.outlier_weekly_sales]
```

After identifying potential outlier candidates, the next step is to make sure that these observations are indeed abnormal and cannot be attributed to an explainable phenomenon. More precisely, one needs to carefully check whether these values are anomalies (and should be removed or modified) or whether they are actual data that should be retained. Statistically abnormal values can often come from legitimate reasons (e.g., a hefty promotional campaign, a large tourist group shopping in the store). Actual outliers, however, can be created by errors during the data collection process (e.g., mistake when pulling out the data), when the event occurs (selling the item at the wrong price or using the wrong discount code), but also by external events that one cannot use in a prediction model (e.g., a celebrity spontaneously endorsing a product and thus creating a major demand surge).

A good practice is to carefully inspect each outlier candidate and try to retrieve the reasons behind the unconventional values. One can do so by leveraging business expertise, examining the presence of external events, and by interviewing domain experts. For example, a price can be much lower than the average price for a special promotional event, but a price cannot be negative. Ultimately, one needs to use business intuition and make a judgment call. If all the outlier candidates can be explained with rational reasons, then one can proceed forward. If, however, some of them appear to be real outliers, then one needs to address the issue. A first option is to remove the identified anomalous observations (or sometimes even all the observations related to a specific SKU). A second option is to modify the abnormal values by replacing them with a more moderate value, such as the average or median value over the past few weeks or a specific quartile value. In our dataset, we decide to retain all the observations and not remove or modify any of the outlier candidates identified above.

2.3 Accounting for Time Effects

The impact of the temporal dimension on retail demand can be modeled by including seasonality features. Generally, there are four types of time series components: trend, seasonal variations, cyclical fluctuations, and irregular variations.³ In the context of retail demand prediction, it is common to focus only on the following two variables:

³https://link.springer.com/referenceworkentry/10.1007/978-0-387-32833-1_401.

Table 2.1 Outliers for the price variable

week	sku	weekly_sales	feat_main_page	color	price	vendor	functionality	outlier_price	outlier_weekly_sales
5/8/17	10	9	TRUE	white	130.89	9	10. VR headset	TRUE	FALSE
12/5/16	12	8	FALSE	black	135.91	6	01. Streaming sticks	TRUE	FALSE
1/15/18	29	11	FALSE	grey	170.76	6	06.Mobile phone accessories	TRUE	FALSE
8/6/18	40	51	FALSE	black	33.08	5	06.Mobile phone accessories	TRUE	FALSE
6/12/17	42	3	FALSE	black	87.98	10	09.Smartphone stands	TRUE	FALSE
4/16/18	44	2	TRUE	black	112.83	6	09.Smartphone stands	TRUE	FALSE

- *Trend*: This variable captures long-term demand movements. We operationalize the demand trend through the year associated with the observation. We extract the year from the `week` feature and normalize it by subtracting the minimum value (2016). An alternative way to capture the trend is by having a cumulative time variable (i.e., the total number of weeks from the beginning of the dataset).
- *Seasonality*: This is a categorical feature that measures the monthly (or weekly) effect on sales.

We construct the above two variables as follows:

```
sales['trend'] = sales['week'].dt.year - 2016
sales['month'] = sales['week'].dt.month
```

Note that we consider that each week belongs to the month associated with the Monday of the focal week.

We then use a one-hot encoding method on the month variable to include these binary features in our predictive models. We will use the `get_dummies` function which creates 11 features out of the 12 possible values for the calendar months, each containing an indicator for a specific month.

Remark: When using a linear regression, we only need 11 features to represent the 12 calendar months. Indeed, not belonging to any of the first 11 months is equivalent to belonging to the 12th month (we use the month of January as our baseline). We use the command `drop_first=True` below to implement this operation. More information on this topic can be found in the `pandas` documentation.⁴ It is worth highlighting that we only need to perform this operation when using a linear regression method (most alternative predictive methods, such as tree-based models, do not require removing one of the encoded variables).

```
sales = pd.get_dummies(data=sales, columns=['month'], drop_first = True)
```

Table 2.3 presents an example of an observation before and after the one-hot encoding of the *month* variable:

Note that the seasonality can be modeled at different levels. If we were to model the seasonality at the week level, we would create 51 dummy variables (each for a different calendar week minus the first week that we would drop) instead of 11 monthly dummy variables. Another possibility is to model the seasonality at the quarter level (to capture the four seasons). The right way to model seasonality depends on the context under consideration and is often driven by business expertise.

⁴https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.get_dummies.html.

Table 2.3 Example of an observation before and after the one-hot encoding of the *month* variable

Before one-hot encoding	After one-hot encoding
week 2018-09-24	week 2018-09-24
sku 44	sku 44
weekly_sales 26	weekly_sales 26
feat_main_page True	feat_main_page True
color black	color black
price 43.45	price 43.45
vendor 6	vendor 6
functionality 09.Smartphone	Functionality 09.Smartphone
stands	stands
trend 2	trend 2
month 9	month_2 0
	month_3 0
	month_4 0
	month_5 0
	month_6 0
	month_7 0
	month_8 0
	month_9 1
	month_10 0
	month_11 0
	month_12 0

2.4 Price and Lag-Prices

The price is definitely one of the most important factors that drive demand in a competitive market such as electronics e-commerce. This is why the company decided to adjust the price on a weekly basis to remain competitive and boost revenues and profits.

First, the price of a SKU in the current week affects the sales of this SKU during that week. For example, customers are comparing prices and may end up purchasing the product from this specific retailer if the price was lower relative to its competitors.

Second, the prices of previous weeks (often called lag-prices) can also have an impact on current sales. Indeed, several consumers adopt a deal-seeking behavior and will adjust their purchases based on the offered discounts. Thus, lowering the prices in previous weeks (e.g., weeks W-1 and W-2) may potentially reduce the sales observed in week W, because some customers may have already purchased the product in one of the previous weeks. The lag-order (which we call M) is the parameter that models the number of past prices that affect current demand. The value of M can be estimated directly from the data and often depends on products’ characteristics, such as perishability and technology obsolescence.

For the above reason, it is often important to include lag-prices as predictors in the demand function. In our case, we use `price-1` and `price-2` (i.e., the price of the SKU in the previous 2 weeks) as predictors in addition to the current price. This selection of lag order is based on both business knowledge and predictive power

(i.e., we want to generate the best possible prediction). For conciseness, we do not report the details of this selection process. At a high level, we consider several combinations of lag prices and compare the prediction accuracy of each option. We ultimately select the best combination, which, in our case, is to include `price-1` and `price-2`. Depending on the context, one can naturally consider including a different number of lag prices (e.g., the four last prices). An alternative feature (instead of lag prices) can be the time elapsed from the last promotional event.

To create the lag-prices features, we use the `shift` function:

```
## Lag prices
sales['price-1'] = sales.groupby(['sku'])['price'].shift(1)
sales['price-2'] = sales.groupby(['sku'])['price'].shift(2)
sales.dropna(subset=['price-1', 'price-2'], inplace=True)
sales.head()
```

The command `shift(1)` applied to the price variable returns the price value from the previous week. More information on the `shift` command can be found in the pandas documentation.⁵ We note that the values of the lag-prices for the first 2 weeks are not well defined. One potential way to fix this issue is to simply remove the first 2 weeks (for all SKUs) and start our dataset from the third week. In the above code, the command `dropna` removes from the dataset the rows with a blank cell for either `price-1` or `price-2`. In this case, it thus omits the first 2 weeks, so that our final dataset will include 98 weeks. An alternative way is to retain the first 2 weeks and assume that the lag-prices are equal to the average price or to the price in the first week.

Remark: To enhance clarity, we next modify the columns' order to assign the price and the two lag-prices to the first three columns. Here are the relevant commands:

```
## Put lag-prices next to the price column
#price
col = sales.pop('price') #pop deletes the column
sales.insert(3, col.name, col) #insert a column at a specific position
pos_price=sales.columns.get_loc('price') #get position of column
#p-1
col = sales.pop('price-1')
sales.insert(pos_price+1, col.name, col)
#p-2
col = sales.pop('price-2')
sales.insert(pos_price+2, col.name, col)
#plot
sales.head()
```

⁵<https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.shift.html>.

The `pop()` function deletes a column from a data frame. The deleted column is then stored in the variable `col`, so that we can add it back to the data frame in another position. Next, the `insert` function moves the focal column to the desired position by using the defined column name. Finally, we insert the lag-price columns next to the price column (`pos_price+1` and `pos_price+2`) for legibility purposes.

Remark: When considering the lag prices as predictive features to predict current demand, one needs to ensure that the price values are known at the time of prediction. For example, when predicting the demand far into the future, the values of the past prices are not always known in advance. In several retail applications, however, prices are set well in advance (e.g., in the previous quarter), so this is not an issue. If prices are not known in advance, one cannot use past prices as predictive features.

2.5 Featured on Main Page

As mentioned before, the company can decide to boost the visibility of specific products by featuring them on the main website's homepage (typically for 1 full week). Being featured on the homepage naturally increases awareness and may also steer indecisive consumers toward the featured products.

Originally `feat_main_page` is a Boolean variable (i.e., with True or False values). Unfortunately, we cannot directly use such variables with `sklearn`. Thus, we decide to make this variable numerical by assigning a value of 1 to the SKU-week pairs that are featured on the main page and 0 to others:

```
sales['feat_main_page'] = sales.feat_main_page.astype('int')
```

Remark: One may take this analysis to the next level by considering the impact of featuring a SKU on the home page on the sales of similar SKUs (e.g., to capture cannibalization effects),⁶ or on future sales of the same SKU. For simplicity and conciseness, we will not dive deeper into this topic and only consider a static binary variable for each SKU.

⁶See Srinivasan et al. (2005).

2.6 Item Descriptive Features

As we did for the month variables, we next perform a one-hot encoding on the functionality, color, and vendor variables. The code is reported below:

```
sales=pd.get_dummies(data=sales,
                      columns=['functionality','color','vendor'],
                      drop_first = True)

sales.head()
```

At this point, for each SKU in our dataset, we have access to the functionality, color, and vendor values. More generally, if additional relevant information on the SKUs is available (e.g., size, country of origin), one can include them in a similar fashion.

2.7 Additional Features

Depending on the business setting under consideration, several additional features can be included in demand prediction models. In e-commerce applications, for example, one can include data related to the customer journey, such as clicks, searches, dwell time (i.e., how long the customer spent on each webpage), and cookies information. In brick-and-mortar applications, one can include the location of the item on the shelf (if available), prices of other SKUs, and promotion-vehicle information (e.g., in-store flyers, endcap displays, TV advertisements). In some applications, one can also consider adding external data sources, such as Google Trend, social media, weather, and macroeconomics factors. Finding the right set of relevant features to accurately predict demand is often considered as an art and is informed by domain expertise.

In our setting, we will keep things simple and consider only the set of features discussed above. However, one can apply a similar process and use the same prediction models we will cover in the following sections for settings that include additional features. A common extension is to consider adding the prices of other SKUs as predictors (i.e., the price of SKU i is used to predict the demand of SKU j). Including these variables would allow us to capture the potential substitution and complementarity of the different SKUs.⁷

⁷See, e.g., Pindyck and Rubinfeld (2018), Cohen and Perakis (2020).

2.8 Scaling

When dealing with features that have different ranges of values, it can often be desirable to scale (or normalize) the features in the dataset, so they all lie in a similar range. Scaling data can also decrease the running time of the learning algorithm. Another advantage of scaling the features is to make the estimated coefficients easier to compare and interpret. Below, we present two common ways to scale features:

- Standard scaling will scale a feature x to a normalized version z with mean 0 and standard deviation 1.⁸

$$z = \frac{x - \mu}{\sigma},$$

where μ and σ are the mean and standard deviation of the feature x , respectively. Specifically, the average and standard deviation can be computed either for each SKU separately or jointly for all the SKUs (this choice depends on the context and on the data variation). This scaling can be performed using the following code:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(data)
scaler.transform(data)
```

- Min Max scaling will scale a feature x to a normalized version z that takes values between 0 and 1.⁹

$$z = \frac{x - \min(x)}{\max(x) - \min(x)},$$

where the minimum and maximum functions can either be taken for each SKU separately or jointly for all the SKUs. This scaling can be performed using the following code:

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
scaler.fit(data)
scaler.transform(data)
```

In the context of this book (demand prediction in retail), and based on our dataset, scaling the features does not significantly impact the predictive performance of most of the methods we will present. However, for various contexts and datasets, scaling

⁸<https://sklearn.org/modules/generated/sklearn.preprocessing.StandardScaler.html>.

⁹<https://sklearn.org/modules/generated/sklearn.preprocessing.MinMaxScaler.html>.

the features may be important and can sometimes drastically improve the results (especially for tree-based models and clustering methods).

2.9 Sorting and Exporting the Dataset

We sort the values by SKU and by week. The processed dataset can then be exported. The exported dataset is also provided on the companion website.

```
Sales = sales.sort_values(by=['sku', 'week'])
sales.to_csv('data_processed.csv', index=False) #we don't need the index
```

At this stage, we have a fully processed dataset, and we are ready to proceed with the step of predicting demand.

References

- Cohen MC, Perakis G (2020) Optimizing promotions for multiple items in supermarkets. *Channel Strategies and Marketing Mix in a Connected World*, 71–97 (Springer).
- Pindyck RS, Rubinfeld DL (2018) *Microeconomics*.
- Srinivasan, S. R., S. Ramakrishnan, S. Grasman. 2005. Incorporating cannibalization models into demand forecasting. *Marketing Intelligence & Planning*.

Chapter 3

Common Demand Prediction Methods



One common demand prediction method relies on applying the ordinary least squares (OLS) method (either with or without regularization, as we will discuss below). The question then is what should be the right level of data aggregation. One extreme option is to estimate a different model for each SKU. In our case, this translates into estimating 44 regression specifications in parallel. Each SKU will then have a different set of estimated coefficients. We will refer to this method as the decentralized approach. The second extreme option is to combine the data across all SKUs and estimate a single model for all SKUs. We will call this method the centralized approach. There are advantages and disadvantages to either approach. For example, the centralized approach is faster to train and is less prone to overfitting. However, this approach is often too simple and does not accurately capture important SKU-specific characteristics (e.g., certain SKUs are more sensitive to price promotions, specific SKUs are more seasonal). In practice, however, several options, which are less extreme, are also considered. For example, one could estimate a single model for all SKUs, with a different intercept coefficient for each SKU or even a different price coefficient for each SKU. More generally, one could decide that a subset of features should be estimated at the aggregate level (e.g., trend and seasonality), whereas another subset of features should be estimated at the SKU level (e.g., price). Finding the right level of data aggregation for each feature highly depends on the context and on the data (we will formally discuss this trade-off in Chap. 7). In this section, we will implement and compare the decentralized and centralized approaches, as well as consider several alternative options.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 3/Common Demand Prediction Methods.ipynb

3.1 Primer: Basic Linear Regression for One SKU

We start by building a simple predictive model for one specific SKU (SKU 11).¹ This will allow us to discuss various concepts and implementation details in a simple setup before extending our treatment to the entire dataset in order to build a demand prediction model for all 44 SKUs.

We filter the observations that belong to SKU 11 and select all the features we created so far except for `week`, `sku`, and `weekly_sales`.

```
data = sales[sales.sku==11].sort_values(by=['week'])
colnames = [i for i in data.columns if i not in
            ['week', 'weekly_sales', 'sku']]
```

We define `X_primer` as the data frame that contains all our features and `y_primer` as our target variable (i.e., weekly sales or demand).

```
X_primer = data[colnames]
y_primer = data.weekly_sales
```

In a demand prediction model, we want to predict the future sales based on historical sales. As discussed, we need to perform a time-based split to create a training set and a test set.

- The training set contains the data from November 2016 to February 2018 (i.e., 68 weeks², 70% of the data).
- The test set contains the data from March 2018 to September 2018 (i.e., 30 weeks, 30% of the data).

Remark: As a reminder, the raw dataset includes 100 weeks, whereas the processed dataset has 98 weeks (we removed the first 2 weeks to avoid missing values of lag-prices).

```
X_train_primer, X_test_primer = np.split(X_primer, [68])
y_train_primer, y_test_primer = np.split(y_primer, [68])
```

¹SKU 11 was chosen because it illustrates well the results obtained on the entire dataset. We highlight that the model performance may significantly vary from one SKU to another.

²The final dataset contains 98 weeks. See Chap. 2.4 for more details.

Remarks:

- The dataset we provided is already sorted by week; one should make sure that this is the case before using the command `np.split()`.
- Since we want to assign a specific number of weeks to the training set, it is more convenient to use the `np.split` function. To have a temporal split made with specific proportions, one can use the `train_test_split` function from `sklearn`³ (one needs to use the command `shuffle=False` in order to retain the data temporality).

```
from sklearn.model_selection import train_test_split

X_train_primer, X_test_primer = train_test_split(X_primer,
                                                shuffle=False,
                                                train_size=0.70)
y_train_primer, y_test_primer = train_test_split(y_primer,
                                                shuffle=False,
                                                train_size=0.70)
```

We use the OLS method by relying on the `statsmodels` package,⁴ as follows:

```
from statsmodels.regression.linear_model import OLS
model = OLS(y_train_primer, X_train_primer) #model definition
model = model.fit() #model training
y_pred_primer = list(model.predict(X_test_primer))
```

We next evaluate our model performance by computing the OOS R^2 and MSE:

```
from sklearn.metrics import r2_score, mean_squared_error
print('OOS R2:', round(r2_score(y_test_primer, np.array(
    y_pred_primer)), 3))
print('OOS MSE:', round(mean_squared_error(y_test_primer,
    np.array(y_pred_primer)), 3))
```

For simplicity, we have rounded the metrics to three decimals. Here are the results we obtain:

```
OOS R2: 0.309
OOS MSE: 3725.488
```

³https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html.

⁴https://www.statsmodels.org/dev/generated/statsmodels.regression.linear_model.OLS.html.

As we can see, the value of the R^2 is not particularly high, and there is definitely room for improvement. These results will serve as a baseline for the feature selection and regularization methods presented later in this section.

3.2 Structuring the Dataset

We now consider extending the treatment to all 44 SKUs. The first step is to structure our dataset so as to ease the model evaluation process. We define `skuData` to be a dictionary containing `X` (the data frame that contains all our features) and `y` (our `weekly_sales` target variable) for each SKU. This will allow us to easily build a model for each SKU (i.e., the decentralized approach) or a single model for all SKUs (i.e., the centralized approach). The code looks as follows:

```
skuSet = list(sales.sku.unique()) #list of sku id
skuData = {}
colnames = [i for i in sales.columns if i not in
            ['week', 'weekly_sales', 'sku']]
#removing dates, target variable and SKU number
for i in skuSet:
    df_i = sales[sales.sku == i] #build a dataframe for each sku
    #for each sku, we fill the dictionary with the features and target
    variable
    skuData[i] = {'X': df_i[colnames].values,
                  'y': df_i.weekly_sales.values}
```

For each SKU, we need to apply a time-based split. We can thus create another dictionary that contains the split data, namely, the train and test sets for each SKU:

```
X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:

    X_train_i, X_test_i = np.split(skuData[i]['X'], [68]) #split for X
    y_train_i, y_test_i = np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

y_test += list(y_test_i) #creating the complete training array
y_train += list(y_train_i) #creating the complete testing array
```

Once the dataset is properly structured, one can finally start predicting demand. We start by considering the centralized approach.

3.3 Centralized Approach

As discussed, the centralized approach consists of training a single linear regression model by simultaneously using the observations from all SKUs. Thus, the linear regression specification follows the following equation:

$$\text{weekly sales} = \beta_{\text{intercept}} + \beta_{\text{price}} * X_{\text{price}} + \beta_{\text{price-1}} * X_{\text{price-1}} + \dots + \beta_{\text{vendor10}} * X_{\text{vendor10}} + \varepsilon.$$

Here, $\beta_{\text{intercept}}$ corresponds to the intercept, ε is the error term, and the β coefficients correspond to all the features selected.

We next build the appropriate dataset for the centralized approach:

```
X_cen_train=X_dict[skuSet[0]]['train'] #initialization with item 0
X_cen_test = X_dict[skuSet[0]]['test']

for i in skuSet[1:]: #Iteration over items
    #concatenation of training sets
    X_cen_train=np.concatenate((X_cen_train, X_dict[i]['train']),
                               axis = 0)
    #concatenation of test sets
    X_cen_test = np.concatenate((X_cen_test, X_dict[i]['test']),
                                axis = 0)
```

Specifically, we concatenate the training data across all SKUs to build a centralized training set. We then apply the same logic to the centralized test set.

We can now fit a single linear regression using the centralized training set and compute the OOS R^2 and MSE.

```
from sklearn.linear_model import LinearRegression

model_cen = LinearRegression().fit(X_cen_train, y_train)
print('OOS R2:', round(r2_score(y_test, model_cen.predict(
    X_cen_test)), 3))
print('OOS MSE:', round(mean_squared_error(y_test, model_cen.
    predict(X_cen_test)), 3))
```

The results are given by:

```
OOS R2: 0.114
OOS MSE: 98086.301
```

Remark: We note that throughout this book, we sometimes use the `LinearRegression` class from the `sklearn` library, whereas other times we rely on the `OLS` class from `statsmodels` to create linear regressions. These packages will yield the same results in the vast majority of cases (in some edge cases, which are outside the scope of this book, results may differ because of implementation differences). However, supplemental functions make it interesting to learn how to use both methods depending on one's needs (e.g., outputting summary statistics is easier with `statsmodel`, whereas cross validation is easier with `sklearn`).

We may want to keep track of the running time of estimating the above model. To do so, we import the `time` library.

```
import time
tZero=time.time() #time at the beginning

...# insert code to run

t = time.time()-tZero #difference of time between tZero and time at
    the end of the code
print('Time to compute:',round(t,3), ' sec')
```

The output is given by:

```
Time to compute: 0.203 sec
```

As we can see, the computation time for the centralized OLS approach is very low. However, the prediction accuracy is also very low (0.114 OOS R^2), given that the centralized approach imposes a uniform structure for all SKUs. When the SKUs have different characteristics (e.g., different categories, brands), this approach may not be appropriate.

Remark: Depending on the machine and the computing environment used to run the different models, one may naturally get a different running time. One should not focus on the exact value of the computing time but rather on its magnitude.

3.4 Decentralized Approach

As discussed, under the decentralized approach, we estimate a different linear regression model for each SKU. Namely, we assume that the weekly sales of each SKU $i = 1, 2, \dots, 44$ behave according to the following equation (the coefficients of each SKU are estimated by using only the observations associated with this SKU):

$$\text{weekly sales}_i = \beta_{\text{price } i} * X_{\text{price } i} + \beta_{\text{price}-1 \ i} * X_{\text{price}-1 \ i} + \dots + \beta_{\text{vendor10 } i} * X_{\text{vendor10 } i} + \varepsilon_i.$$

Our goal is to estimate the β coefficients for each SKU. The code proceeds as follows:

```
tZero=time.time()

y_pred = []
skuModels = {}

for i in skuSet:
    #one model for each item, fitted on training set
    model_i = OLS(y_dict[i]['train'], X_dict[i]['train'], hasconst = False)
    skuModels[i] = model_i.fit()

    #compute and concatenate prediction of the model i on item i
    y_pred += list(skuModels[i].predict(X_dict[i]['test']))

#computing overall performance metrics on y_pred and y_test:
print('OOS R2:', round(r2_score(y_test, np.array(y_pred)), 3))
print('OOS MSE:', round(mean_squared_error(y_test, np.array(y_pred)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec')
```

The results (OOS R^2 , MSE, and running time) are given by:

```
OOS R2: 0.517
OOS MSE: 53537.475
Time to compute: 0.065 sec
```

In this case, the decentralized approach clearly outperforms the centralized approach in terms of out-of-sample prediction accuracy. Specifically, we obtain a 354% (resp. 45%) increase (resp. decrease) in the OOS R^2 (resp. MSE) relative to the centralized OLS approach. This finding supports the importance of item-specific characteristics in capturing demand patterns. In different applications, however, it may be possible that the centralized approach outperforms the decentralized approach, depending on the heterogeneity of the SKUs and the data quality. This phenomenon is driven by the well-known bias-variance trade-off in supervised learning,⁵ which presents the conflict between trying to reduce bias (i.e., inability of the model to perfectly capture the relationship between the features and the target variable, such as assuming a linear relationship) and reducing variance (i.e.,

⁵Hastie et al. (2009).

sensitivity of the model performance to fluctuations in the training dataset). On the one hand, the centralized approach pools together data from multiple SKUs to fit a single model, thus reducing the variance of the model at the cost of increasing the bias from model misspecification for each individual SKU. On the other hand, the decentralized approach fits a different model for each SKU, thus reducing the bias from model-misspecification, while potentially increasing the variance due to the limited amount of data for each SKU.

3.5 Feature Selection and Regularization

So far, the models we estimated did not include either feature selection or regularization. In this section, we investigate these two concepts. Feature selection and regularization are important steps that should follow data pre-processing. When a dataset includes a large number of features, it is likely that not all of them are relevant and pertinent in terms of predicting demand. Identifying the key features that bear the most predictive power has several major benefits, such as the following:

- decreasing computational and data-acquisition costs,
- increasing the interpretability of the model, and
- potentially reducing the overfitting issue.

The feature selection process is based on fitting a model with a subset of features out of the available ones. This process is guided by several criteria, such as model accuracy and processing speed.

Regularization is another way to improve models, often referred to as shrinkage methods. The main intent of regularization methods is to update the estimated coefficients of the prediction model to simplify the model (by reducing the number of estimated coefficients), and ultimately, reducing the risk of overfitting.

In this section, we will illustrate the concepts of subset selection and regularization by using the basic model built for SKU 11 before extending the treatment to all 44 SKUs.

3.5.1 *Subset Selection*

3.5.1.1 Presentation of Subset Selection

The concept of subset selection aims to identify the best-performing subset of features by estimating various models, each using a different subset of d features, out of the p features ($d < p$). It then compares these different models using a specific criterion. Since we want to compare models with a different number of features, we cannot simply use the in-sample R^2 or the RSS as metrics. Indeed, a model with a larger number of predictors will always have a higher in sample R^2 and a lower RSS

relative to a model with a smaller number of features. Common criteria used in this context are the Akaike Information Criterion (AIC), the Bayesian Information Criterion (BIC), Cp, and the adjusted R^2 . We next discuss in great detail the AIC and BIC.

There are three common methods to perform feature selection: best, forward stepwise, and backward stepwise feature selection. The best subset selection method consists of trying all model possibilities that use d features out of the p available features in the data, with the value of d ranging between 1 and p and finding the best combination. The major drawback of this method is the potentially enormous computing time. Indeed, this method requires testing all possible combinations of features even in cases where some irrelevant features can be easily identified. The best subset selection method is often very time-consuming to find even on relatively small datasets. On the other hand, stepwise methods reduce the number of models to test by exploring different combinations in a unidirectional way (forward or backward).

The forward method starts by fitting all one-feature models (i.e., models with a single feature). Once the best option is identified, the first feature is selected. The next step involves considering all two-feature models (where the first feature is the one identified in the first step). More precisely, the method will explore all pairs of features that contain the previously selected feature. The pair of features leading to the best performing model will then be selected. The process continues in the same fashion. At the end of the process, we obtain p models (where p is the total number of features), where each model was built by using a different number of features. We then select the best performing model along with its features.

The backward method adopts a similar spirit, except that it proceeds by removing features one at a time, starting from the initial model that contains all the available features. For conciseness, we will only illustrate the forward method along with its detailed implementation.

3.5.1.2 AIC and BIC

Recall that we study the relationship between the weekly sales and our various features using the following equation:

$$\begin{aligned} \text{weekly sales} = & \beta_{\text{intercept}} + \beta_{\text{price}} \times X_{\text{price}} + \beta_{\text{price}-1} \times X_{\text{price}-1} + \dots + \beta_{\text{vendor10}} \\ & * X_{\text{vendor10}} + \varepsilon. \end{aligned}$$

The general version of the AIC formula is based on maximum likelihood. For simplicity, we will use a simplified version of the AIC. Specifically, we discard constant terms of the formula because we are only interested in comparing the differences in the AIC between several models and we assume that the residual error (ε) follows a normal distribution. In this case, the AIC is given by:

$$AIC = \frac{(RSS + 2d\hat{\sigma}^2)}{n\hat{\sigma}^2}$$

where d is the number of predictors, n is the number of samples, $\hat{\sigma}^2$ is an estimate of the variance of ε (obtained from a model that contains all the features), and RSS is the residual sum of squares (defined in Chap. 1).

The AIC formula bears the following intuition: increasing the number of features (i.e., the value of d) will have two effects:

- Improving the model performance and thus reducing the RSS value, which will result in a decreased AIC.
- Increasing the penalty term, $2d\hat{\sigma}^2$, which will increase the AIC.

The purpose of the AIC is to compare the impact of these two effects. In order to obtain a smaller AIC, one would only add new features if the decrease in the RSS value exceeds the increase in the penalty term.

Similarly, we define the BIC as follows:

$$BIC = \frac{(RSS + \log(n)d\hat{\sigma}^2)}{n\hat{\sigma}^2}.$$

The only difference between the AIC and the BIC is that the factor 2 is replaced by $\log(n)$ in the penalty term. Since n is the number of observations, it means that for large values of n , BIC will penalize more heavily the increase in the number of features d relative to the AIC.

We next present a function that performs a forward feature selection process. In this function, p is the number of features that one wishes to select (input decided by the user). The function then computes the selected metric (AIC by default) while using only a single feature. After selecting the feature leading to the smallest AIC value, the function will add an additional feature to the previously selected one and compute the new AIC value. This process will go on until we reach a subset of p selected features.

```
def forward_stepwise_selection(data,p,metric='aic'):
    """
    Perform selection of p features on a dataset

    p: highest number of features used to train each model
    metric: aic (default) or bic

    Returns best trained model and list of features selected
    """
```

(continued)

```

available_features = set(data.columns)
selected = []
current_score, best_new_score = 1000, 1000

overall_selected = []
while available_features and current_score == best_new_score and
    len(selected) < p:

    scores_with_candidates = []
    for candidate in available_features:

        selected_candidates = selected.copy()

        selected_candidates.append(candidate)
        # print(selected_candidates)
        if metric == 'aic':
            score = OLS(y_train_primer,
                        X_train_primer[selected_candidates]
                        ).fit().aic
        elif metric == 'bic':
            score = OLS(y_train_primer,
                        X_train_primer[selected_candidates]
                        ).fit().bic
        else:
            print('Metric not supported')
            break
        scores_with_candidates.append((score, candidate))
    scores_with_candidates.sort(reverse=True)
    best_new_score, best_candidate = scores_with_candidates.
        pop()

    #remove the selected feature from available features:
    available_features.remove(best_candidate)
    selected.append(best_candidate) #add it to our list of
        features selected

    # display results at each stage
    print(str(len(selected)) + '. : ', selected, best_new_score)

    #if this model generates an improvement we record it
    if current_score > best_new_score:
        current_score = best_new_score
        overall_selected = selected.copy()

model = OLS(y_train_primer, X_train_primer[overall_selected]).fit()
return(model, overall_selected)

```

The above function returns the trained model on selected features as well as the subset of features leading to the best result for the selected metric.

When using AIC (in the example below, we use $p = 10$ features), the command is as follows:

```

model,selected = forward_stepwise_selection(X_train_primer,10,'aic')

print(selected)
print(model.aic)

```

The output of the three selected features along with the resulting AIC value is given by:

```

1. : ['vendor_10'] 718.7550022483708

2. : ['vendor_10', 'price'] 711.9902300153797

3. : ['vendor_10', 'price', 'month_5'] 709.5003738038225

4. : ['vendor_10', 'price', 'month_5', 'month_7'] 708.9123972788229

5. : ['vendor_10', 'price', 'month_5', 'month_7', 'color_blue']
708.9123972788229

6. : ['vendor_10', 'price', 'month_5', 'month_7', 'color_blue',
'color_gold'] 708.9123972788229

```

As we can see, the smallest AIC value of 708.91 is obtained when using the four following features: ['vendor_10', 'price', 'month_5', 'month_7'] .

We next apply the demand prediction model and compute the OOS R^2 and MSE.

```

model = OLS(y_train_primer, X_train_primer[selected])
model = model.fit()
y_pred_primer = list(model.predict(X_test_primer[selected]))
from sklearn.metrics import r2_score, mean_squared_error
print('OOS R2:',round(r2_score(y_test_primer, np.array
(y_pred_primer)),3))
print('OOS MSE:', round(mean_squared_error(y_test_primer,
np.array(y_pred_primer)),3))

```

The results are given by:

```

OOS R2: 0.306
OOS MSE: 3740.378

```

When using BIC (with $p = 6$ features), the command is as follows:

```

model,selected = forward_stepwise_selection(X_train_primer,6,'bic')

print(selected)
print(model.bic)

```

The output of the three selected features along with the resulting BIC value is given by:

```
1. : ['vendor_10'] 720.9745099535469

2. : ['vendor_10', 'price'] 716.4292454257319

3. : ['vendor_10', 'price', 'month_5'] 716.1588969193508

4. : ['vendor_10', 'price', 'month_5', 'color_blue']
716.1588969193508

5. : ['vendor_10', 'price', 'month_5', 'color_blue', 'color_gold']
716.1588969193508

6. : ['vendor_10', 'price', 'month_5', 'color_blue', 'color_gold',
'color_green'] 716.1588969193508
```

As we can see, the smallest BIC value of 716.16 is obtained when using the three following features: ['vendor_10', 'price', 'month_5'].

As before, we apply the demand prediction model and compute the OOS R^2 and MSE.

```
model = OLS(y_train_primer, X_train_primer[selected])
model = model.fit()
y_pred_primer = list(model.predict(X_test_primer[selected]))
from sklearn.metrics import r2_score, mean_squared_error
print('OOS R2:', round(r2_score(y_test_primer, np.array
(y_pred_primer)), 3))
print('OOS MSE:', round(mean_squared_error(y_test_primer,
np.array(y_pred_primer)), 3))
```

The results are given by:

```
OOS R2: 0.327
OOS MSE: 3625.572
```

It is worth mentioning that we obtain better results (in terms of prediction accuracy) if we follow the selection based on the BIC even if it means using three features instead of four (as obtained based on AIC).

While performing forward selection is much more efficient than testing all possible combinations of features, it is not an optimal method. In the forward stepwise selection process, all the models are built based on the features selected in the previous step, which means that the sequence of features selected by the algorithm will depend on the first selected features. As discussed, one can apply a similar method based on a backward stepwise selection process, starting from a model that contains all the available features, and iteratively computing the AIC or

BIC after removing a single feature at a time. This process is then repeated until the best performing model is obtained.

An alternative, albeit less common approach, is to perform the feature selection by looking at the statistical significance of the estimated parameters for the different features. When looking at the estimated parameters, one may want to pay particular attention to their sign and their statistical significance. For example, the price coefficient is expected to be negative, whereas the coefficient for the featured on the main page variable is expected to be positive. In addition, one can look at the statistical significance of the estimated parameters (captured by the p-value).⁶ When the p-value is low (e.g., below 0.05), it means that the estimated coefficient is statistically significant. We highlight that this method is less common in predictive analytics tasks, such as demand prediction, and more appropriate in tasks related to causal inference and econometrics.

In summary, the main intent of feature selection is to simplify the model and reduce the risk of overfitting. We next discuss several classical approaches to perform regularization and elaborate on their ability to improve our prediction models.

3.5.2 Lasso Regularization

3.5.2.1 Lasso for One SKU

Our first regularization method is called Lasso (least absolute shrinkage and selection operator) and falls under shrinkage methods. Shrinkage methods use all the features along with an algorithm to optimize the coefficient value assigned to each feature.

To compute the coefficient for each feature, the Lasso method optimizes the coefficients in a similar way as OLS, while including an additive penalty with the goal of making some coefficients to zero. Specifically, Lasso penalizes the residual sum of squares (RSS) as well as the absolute value of the coefficients (β_j) as follows:

$$\min_{\beta_1, \dots, \beta_p} \sum_{i=1}^n \left(r_i - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p |\beta_j|,$$

where p is the number of features, n is the number of SKUs, r_i is the actual sales for SKU i , β_j is the coefficient of feature j , and α is a tuning parameter (we provide more detail on how to select the tuning parameter below). The resulting features with non-zero coefficients are considered selected by the Lasso method.

We use the `sklearn` Lasso library,⁷ as follows:

⁶For more details on the concept of p-value, see e.g., Definition 8.3.26 of Casella and Berger (2002).

⁷https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html.

```
from sklearn import linear_model
lasso_model = linear_model.Lasso(alpha=1.0)
lasso_model.fit(X_train_primer, y_train_primer)
```

Since Lasso is the first regularization method we discuss, we will first present the implementation details for one specific SKU (SKU 11), before extending it to our decentralized approach (the next regularization methods will be directly presented for the decentralized approach, without detailing the process for SKU 11 to avoid repetitions).

The parameter α captures the weight of the penalty term assigned to the regularization. Increasing the value of α gives more weight to the second term (value of the estimated coefficients) and, thus, will reduce the resulting number of features selected by Lasso. In practice, to identify the optimal value of α , one can potentially test different values (e.g., by using a random or a grid search procedure) and compare the performance of the different options. In Fig. 3.1, we present the results for different values of α (the y-axis is the OOS R^2 and the x-axis is the logarithm of α). As we can see from the figure, in this case, we observe a fairly stable performance across different values of α (the performance decreases when α starts becoming larger). In what follows, we use a value of α equal to 1.0.

We will present a more rigorous hyperparameter tuning process in Chap. 4.

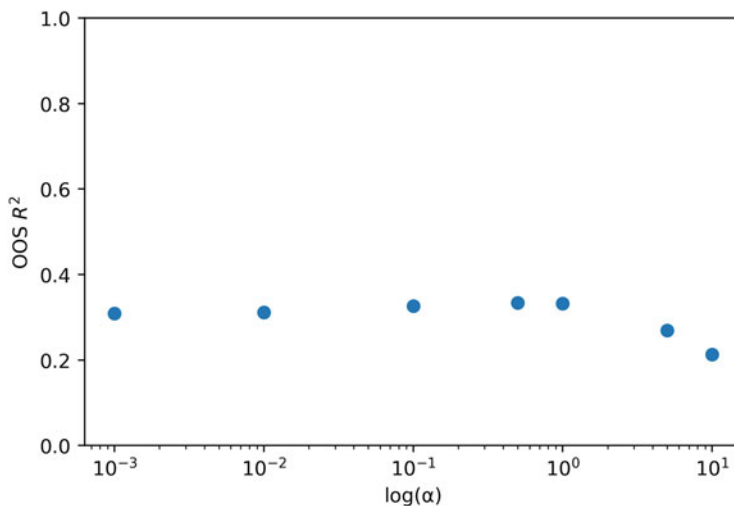


Fig. 3.1 Prediction performance on SKU 11 for different values of α

```
lasso_df=pd.DataFrame(list(zip(X_primer.columns,
                              lasso_model.coef_)),
                      columns=['feature', 'coefficient'])

#zip iterates on tuples and brings items together

lasso_df[lasso_df.coefficient !=0]
```

In Table 3.1, we report the estimated values of the different coefficients when we set the value of α to either 0.5 or 1.0.

As we increase the value of α , we decrease the number of features selected by the Lasso method, namely, a higher α corresponds to a stricter feature selection.

Next, we want to assess the model performance by computing the OOS R^2 and MSE:

```
y_pred_primer = list(lasso_model.predict(X_test_primer))
print('OOS R2:', round(r2_score(y_test_primer, np.array
                                (y_pred_primer)), 3))
print('OOS MSE:', round(mean_squared_error(y_test_primer,
                                             np.array(y_pred_primer)), 3))
```

The results are given by:

```
OOS R2: 0.332
OOS MSE: 3601.29
```

As we can see, we obtain a significant performance increase with six Lasso selected features relative to the model that uses all the features (for SKU 11). By doing so, we also improve the model interpretability (and the computational cost) without decreasing the prediction accuracy. Specifically, we obtain a 7.8% (resp. 3.3%) improvement in the OOS R^2 (resp. MSE) relative to the OLS model based on

Table 3.1 List of Lasso-selected estimated coefficients with $\alpha = 0.5$ and $\alpha = 1.0$

Feature	Coefficients $\alpha = 0.5$	Coefficient $\alpha = 1.0$
price	-6.61	-6.41
price-1	-2.45	-2.16
feat_main_page	8.02	—
trend	3.36	1.31
month_5	37.80	28.73
month_6	9.93	—
month_7	24.31	16.77
month_9	-16.89	-8.89
month_10	0.22	—
OOS R^2	0.334	0.332

all the features for SKU 11. We next extend this process to the decentralized approach applied to all 44 SKUs.

3.5.2.2 Decentralized Lasso

Given Lasso's satisfactory performance for SKU 11, we consider a Lasso-decentralized model for all SKUs. For each SKU, Lasso will perform a feature regularization, select the most relevant set of features, and build a linear regression out of these selected features.

```
tZero=time.time()

y_pred = []

skuModelsLasso = {}

for i in skuSet:

    model_i = linear_model.Lasso(alpha=0.1)
    skuModelsLasso[i] = model_i.fit(X_dict[i]['train'],y_dict[i]
                                   ['train'])
    y_pred += list(skuModelsLasso[i].predict(X_dict[i]['test']))

print('OOS R2:',round(r2_score(y_test, np.array(y_pred)),3))
print('OOS MSE:', round(mean_squared_error(y_test, np.array
(y_pred)),3))

t = time.time()-tZero
print('Time to compute:',round(t,3), ' sec')
```

The results are given by:

```
OOS R2: 0.523
OOS MSE: 52833.745
Time to compute: 0.037 sec
```

Interestingly, we find that the Lasso-decentralized model (slightly) outperforms the OLS-decentralized model while speeding up the training process. As discussed, it also potentially enhances the model interpretability.

3.5.3 Ridge Regularization

3.5.3.1 Ridge Model

Our second regularization method, Ridge, is similar to Lasso. Specifically, Ridge penalizes based on the squared value of the coefficients instead of using the absolute values. Namely, it solves the following optimization problem:

$$\min_{\beta_1, \dots, \beta_p} \sum_{i=1}^n \left(r_i - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha \sum_{j=1}^p \beta_j^2.$$

At a high level, the Ridge method assumes that the coefficients close to 0 are irrelevant (even if they are not reaching the exact value of 0). Qualitatively, it implies that a Ridge model will still use all the features and assign them coefficients based on their importance.

More details can be found in the `sklearn` Ridge documentation.⁸

```
from sklearn.linear_model import Ridge
ridge = Ridge(alpha=1.0)
ridge.fit(X_train_primer, y_train_primer)
```

As mentioned earlier, we will directly apply the Ridge regularization method to all SKUs under the decentralized approach.

3.5.3.2 Decentralized Ridge

The implementation details are similar to Lasso and are reported below.

```
tZero=time.time()
y_pred = []
skuModelsRidge = {}

for i in skuSet:
    model_i = Ridge(alpha=1)
    skuModelsRidge[i] = model_i.fit(X_dict[i]['train'], y_dict[i]
    ['train'])
    y_pred += list(skuModelsRidge[i].predict(X_dict[i]['test']))
```

(continued)

⁸https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html.

```
print('OOS R2:', round(r2_score(y_test, np.array(y_pred)), 3))
print('OOS MSE:', round(mean_squared_error(y_test, np.array(y_pred)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec')
```

The results are given by:

```
OOS R2: 0.571
OOS MSE: 47518.987
Time to compute: 0.145 sec
```

For our dataset, we find that the Ridge-decentralized outperforms the Lasso-decentralized in terms of OOS R^2 and MSE.

3.5.4 Elastic Net Regularization

3.5.4.1 Elastic Net Model

Finally, our third regularization method, Elastic Net, combines both Lasso and Ridge and can be tuned using two penalty parameters:

$$\min_{\beta_1, \dots, \beta_p} \sum_{i=1}^n \left(r_i - \sum_{j=1}^p \beta_j X_{ij} \right)^2 + \alpha_1 \sum_{j=1}^p |\beta_j| + \alpha_2 \sum_{j=1}^p \beta_j^2.$$

More precisely, Elastic Net combines both regularization types: L1 (used in Lasso) and L2 (used in Ridge). More details can be found in the Elastic Net Regression documentation.⁹

Here is the code for this case:

```
from sklearn.linear_model import ElasticNet
elastic = ElasticNet(alpha=0.05, l1_ratio=0.7)
elastic.fit(X_train_primer, y_train_primer)
```

We highlight that the parameters used by `sklearn` in the Elastic Net library (α and $l1_ratio$) are different from the ones used in the above equation (α_1 and α_2). Specifically, the relationship between the two sets of parameters is given by:

⁹https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.ElasticNet.html.

$$\alpha = \alpha_1 + \alpha_2,$$

$$l1_{ratio} = \frac{\alpha_1}{\alpha_1 + \alpha_2}.$$

We note that using $l1_{ratio} = 1$ (resp. 0) is equivalent to considering a Lasso (resp. Ridge) model. The choice of the parameter values follows a similar logic as for the Lasso method.

3.5.4.2 Decentralized Elastic Net

We next repeat the same procedure using Elastic Net to estimate the decentralized approach for all 44 SKUs.

```
tZero=time.time()
y_pred = []
skuModelsElastic = {}

for i in skuSet:
    elastic = ElasticNet(alpha= 0.05,l1_ratio=0.3)
    skuModelsElastic[i] = elastic.fit(X_dict[i]['train'],y_dict
                                     [i]['train'])
    y_pred += list(skuModelsElastic[i].predict(X_dict[i]['test']))
print('OOS R2:',round(r2_score(y_test, np.array(y_pred)),3))
print('OOS MSE:', round(mean_squared_error(y_test, np.array
(y_pred)),3))
t = time.time()-tZero
print('Time to compute:',round(t,3), ' sec')
```

The results are given by:

```
OOS R2: 0.585
OOS MSE: 45992.958
Time to compute 0.064 sec
```

As we can see, the OOS prediction accuracy further improves. Overall, one can consider different types of regularization and select the best model in terms of prediction accuracy. It is even possible to select the best type of regularization (with the best penalty hyperparameters) for each SKU.

The feature selection methods presented in this section can help reduce the number of features used in the demand prediction model. Another common approach related to feature selection is to transform the large set of initial features into a smaller number of synthesized features that still contain most of the relevant information present in the large initial set. This type of approach is known as dimensionality-reduction methods, such as Principal Component Analysis (PCA).¹⁰

¹⁰Hastie et al. (2009).

3.6 Log Transformations

It is common to use log transformations to “smoothen” skewed distributions (i.e., distributions for which the data points are more concentrated toward one side). Such transformations can help better interpret patterns in the data. In addition, these transformations can help satisfy the statistical assumptions required by the mathematical models. For example, in a linear regression, the residual errors are assumed to be normally distributed, so that a skewed distribution can potentially lead to unreliable estimated values in terms of statistical significance.

To identify the variables on which to perform a log-transformation, one can rely on data visualization. The idea is to identify skewness patterns in the distribution or a linear relationship between the transformed variables. This often can provide intuition on which transformations to perform. These transformations should then be applied before estimating the prediction model. Ultimately, it may be a good strategy to base the log-transformation decisions on both the data visualization patterns and the performance metrics. We next discuss two types of log-transformation.

3.6.1 *Log-Transformation on the Price Variable*

In several retail datasets, when plotting the histogram of prices, we observe that the distribution follows a log-normal pattern, implying that the logarithm of prices (also called log-prices) follows a Normal distribution. In this case, it may be desirable to perform a log-transformation on the price variable. This pattern is clearly confirmed in our dataset when we plot the histogram of prices and log-prices (see Figs. 3.2 and 3.3).

The code used to plot Figs. 3.2 and 3.3 is reported as follows. These figures show the distributions of prices and log-prices, for all SKUs and all weeks.

```
df_plots=pd.DataFrame()
df_plots['sales']=sales['weekly_sales']
df_plots['price']=sales['price']
df_plots['logprice']=np.log(sales['price'])

import seaborn as sns
import matplotlib.pyplot as plt
color=sns.color_palette(palette='colorblind')[3]

plt.figure(figsize=(15,6))
graph = sns.histplot(data=df_plots, x='price', bins=100,
                    color=color)
plt.title('Distribution of prices', fontsize=15)
```

(continued)

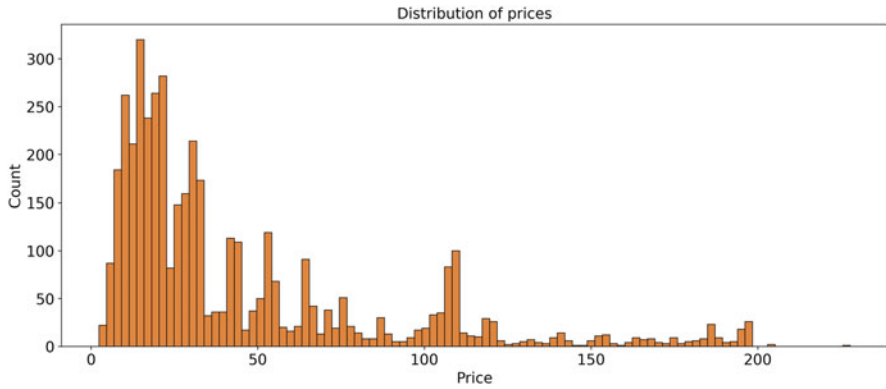


Fig. 3.2 Distribution of prices for all SKUs and all weeks



Fig. 3.3 Distribution of log-prices for all SKUs and all weeks

```
plt.xlabel('Price')
plt.show()

plt.figure(figsize=(15,6))
graph = sns.histplot(data=df_plots, x='logprice', bins=100,
                    color=color)
plt.title('Distribution of Log-prices', fontsize=15)
plt.xlabel('Log-price')
plt.show()
```

In addition, we can plot the sales as a function of prices in an aggregated fashion (i.e., by summing up the sales for all 44 SKUs). The results are presented in Figs. 3.4 and 3.5 when we use the price and the log-price, respectively. The overall impression

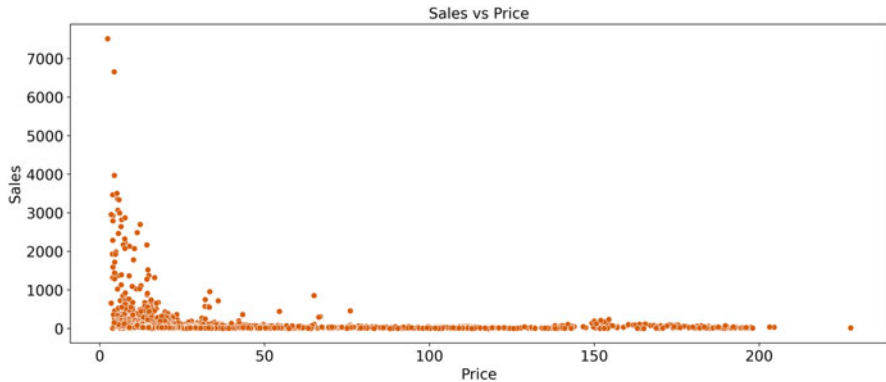


Fig. 3.4 Sales versus price



Fig. 3.5 Sales versus log-price

is that a linear regression is more appropriate for the log-price than for the price. This comforts us in our decision to consider a model with log-prices. The code for Figs. 3.4 and 3.5 is presented below.

```
plt.figure(figsize=(15,6))
plt.title('Sales vs Price', fontsize=15)
sns.scatterplot(data=df_plots, x='price', y='sales', color='orange')
plt.xlabel('Price')
plt.ylabel('Sales')
plt.show()
```

(continued)

```
plt.figure(figsize=(15,6))
plt.title('Sales vs Log-price', fontsize=15)
sns.scatterplot(data=df_plots, x='logprice', y='sales',
               color=color)
plt.xlabel('Log-price')
plt.ylabel('Sales')
plt.show()
```

When comparing Figs. 3.4 and 3.5, one can infer that applying a log-transformation on the price variable is desirable. Indeed, the relationship between the sales and the log-prices seems to be closer to a linear pattern relative to the relationship between the sales and the prices. Once again, the goal here is to develop intuitions, which should then be duly checked by using quantitative methods. A good way to further validate is to compare the prediction accuracy with and without performing the log-transformation, as we will discuss later.

Note that we have focused our discussion on potentially applying a log-transformation to the current price. A similar approach can be used to decide whether applying a log-transformation on the lag prices as well as on other variables. We next discuss the strategy of applying a log-transformation on the target variable (i.e., weekly sales).

3.6.2 Log-Transformation on the Target Variable

Another common approach is to apply a log-transformation on the target variable. In this case, performing a log-transformation can sometimes help better capture trends and data patterns. The distributions of the sales in our dataset are depicted in Figs. 3.6 and 3.7. Once again, we can see a skewness behavior in the distribution.

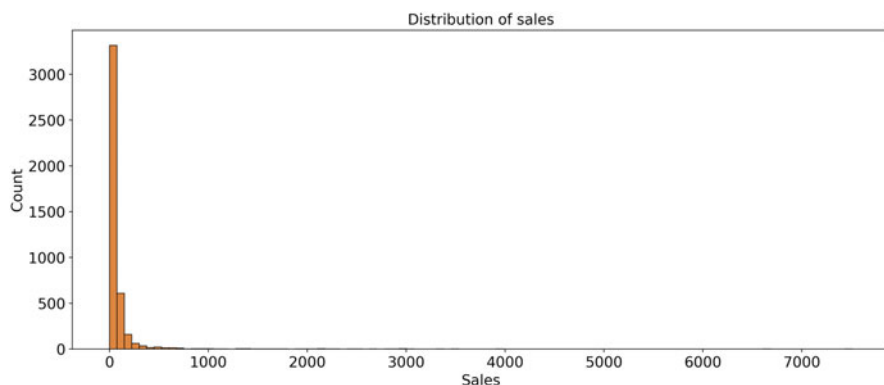


Fig. 3.6 Distribution of sales for all SKUs and all weeks

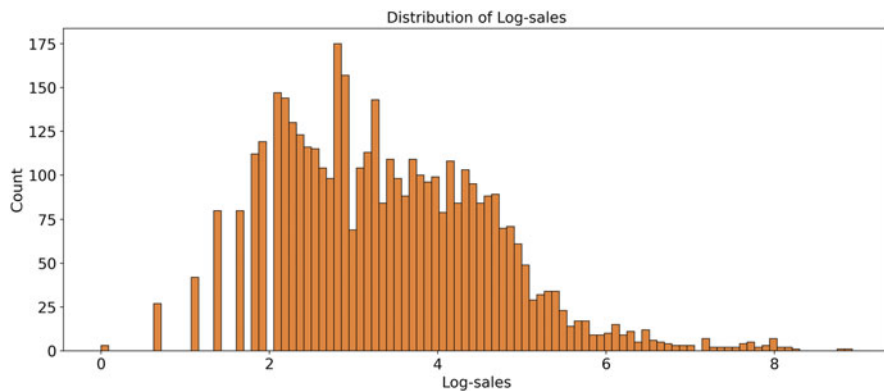


Fig. 3.7 Distribution of log-sales for all SKUs and all weeks



Fig. 3.8 Log-sales versus log-price

The plot of log-sales against log-prices is also presented in Fig. 3.8, suggesting that applying a log-transformation to the target variable may be beneficial. We highlight that these results should be interpreted only as potential signals to the fact that a log-transformation might be helpful, but they do not mean it will necessarily be beneficial. To reach a firm conclusion, one needs to perform the transformation and run the models.

```
df_plots['logsales']=np.log(sales['weekly_sales']+1)
```

Remark: When applying the logarithm function to a variable, one needs to be careful with the zero values to avoid creating infinite values. In our case, this is not a significant issue given that we do not often observe zero weekly sales. Specifically, we have only three zero weekly sales, hence corresponding to 0.07% of our

observations. To address this issue, we apply the logarithm function to `weekly_sales + 1` (instead of `weekly_sales`). Another common way is to use a smaller additive constant (e.g., 0.001) or to simply remove the observations with zero values (assuming they represent a small portion of the dataset).

3.6.3 Transformations and Prediction Accuracy

After applying log-transformations on the prices (and lag-prices) and sales, we can consider estimating the following three models:

Linear model

$$\begin{aligned} \text{weekly sales}_i = & \beta_{\text{intercept } i} + \beta_{\text{price } i} \times X_{\text{price } i} + \beta_{\text{price-1 } i} \times X_{\text{price-1 } i} + \beta_{\text{price-2 } i} \\ & \times X_{\text{price-2 } i} + \dots + \beta_{\text{vendor10 } i} \times X_{\text{vendor10 } i} + \varepsilon_i \end{aligned}$$

Log-linear model

$$\begin{aligned} \text{weekly sales}_i = & \beta_{\text{intercept } i} + \beta_{\text{price } i} \times \log(X_{\text{price } i}) + \beta_{\text{price-1 } i} \times \log(X_{\text{price-1 } i}) \\ & + \beta_{\text{price-2 } i} \times \log(X_{\text{price-2 } i}) + \dots + \beta_{\text{vendor10 } i} \times X_{\text{vendor10 } i} + \varepsilon_i \end{aligned}$$

Log-log model

$$\begin{aligned} \log(\text{weekly sales}_i) = & \beta_{\text{intercept } i} + \beta_{\text{price } i} \times \log(X_{\text{price } i}) + \beta_{\text{price-1 } i} \\ & \times \log(X_{\text{price-1 } i}) + \beta_{\text{price-2 } i} \times \log(X_{\text{price-2 } i}) + \dots \\ & + \beta_{\text{vendor10 } i} \times X_{\text{vendor10 } i} + \varepsilon_i \end{aligned}$$

The linear model assumes that the weekly sales variable depends linearly on the prices. The log-linear model assumes that the weekly sales variable depends linearly on the logarithm of the prices. Finally, the log-log model assumes that the logarithm of the sales depends linearly on the logarithm of the prices. Each of these three models can be estimated by applying an OLS. As discussed, it is also possible to consider a model where the log-transformation is applied only on the current price but not on the lag-prices, and vice-versa. As we can see, applying log-transformations allows us to turn an additive demand model to a multiplicative model.

We next dive into the code for the log-linear model. First, we apply the log-transformations to the data. Then, we structure the data and finally, we estimate the parameters by considering the decentralized approach.

```
sales_log_lin = sales.copy()
sales_log_lin['price'] = np.log(sales_log_lin['price'])
sales_log_lin['price-1'] = np.log(sales_log_lin['price-1'])
sales_log_lin['price-2'] = np.log(sales_log_lin['price-2'])
```

The code for the log-log model follows the exact same logic and is omitted for conciseness. The only difference is that we perform the log-transformation on the sales, and then structure and estimate the model parameters.

```
sales_log_log = sales.copy()
sales_log_log['price'] = np.log(sales_log_log['price'])
sales_log_log['price-1'] = np.log(sales_log_log['price-1'])
sales_log_log['price-2'] = np.log(sales_log_log['price-2'])
sales_log_log['weekly_sales'] = np.log(sales_log_log
    ['weekly_sales'] + 1)
```

When computing the OOS R^2 for this configuration (i.e., log-linear model), it is important not to forget to take the exponential of the predicted values in order to retrieve the predicted demand values (as opposed to the logarithm of the sales values). Note that the variable `y_test` contains the weekly sales (and not the logarithm of the weekly sales).

```
## Don't forget to take the exponential of the predicted values
y_pred = np.exp(y_pred_log) - 1
y_test = np.exp(y_test_log) - 1
## Then compute the R2
print('OOS R2:', r2_score(y_test, np.array(y_pred)))
```

The OOS accuracy results for all three models are presented in Table 3.2.

In our case, we find that the log-linear model yields the highest prediction accuracy. Of course, depending on the business setting and on the dataset, any of the three above models can yield the best performance. A common strategy is to test all three models and select the best one. Another (more time-consuming) possibility is to try all three models for each SKU and select the best model for each specific SKU.

Remark: Fig. 3.8 seems to suggest that the log-log model would work better than the log-linear model, although it did not in terms of prediction accuracy. One

Table 3.2 Comparison of OOS R^2 for models with and without log-transformations

Model	OOS R^2
Linear (decentralized)	0.517
Log-linear (decentralized)	0.558
Log-log (decentralized)	0.200

possible reason is that the impact of other variables (e.g., lag-prices, seasonality) was not captured in this plot. This is why it is important to check intuitions from data visualization by formally estimating and comparing the prediction accuracy of the different models. Modeling can be considered as an art, and while some rules can provide good intuitions, there are no general rules for feature engineering.

Finally, we highlight that additional (non-linear) transformations exist and can be potentially considered. Examples are polynomials (e.g., quadratic), square root, or a Box-Cox transformation (i.e., power function).

3.7 Centralized Approach with SKU-Fixed Effects

As we previously saw, the centralized OLS approach yields a lower performance relative to the decentralized approach. This is probably due to the model's inability to capture SKU-specific characteristics. A potential approach is to consider a centralized model where each SKU has a different intercept parameter (but other estimated parameters are the same). These parameters are often referred to as “SKU-fixed effects” and aim to capture the time-invariant differences in weekly sales among the different SKUs. The other parameters (e.g., price and seasonality), however, are still estimated in a centralized fashion. Intuitively, we apply the decentralized approach on several features and the centralized logic on other features. In this case, the model is given by:

$$\begin{aligned} \text{weekly sales} = & \beta_{\text{intercept}} + \beta_{\text{price}} \times X_{\text{price}} + \beta_{\text{price-1}} \times X_{\text{price-1}} + \dots + \beta_{\text{vendor10}} \\ & \times X_{\text{vendor10}} + \beta_{\text{sku1}} \times X_{\text{sku1}} + \dots + \beta_{\text{sku44}} \times X_{\text{sku44}} + \varepsilon. \end{aligned}$$

Each of the $X_{\text{sku } i}$ is a binary variable, equal to 1 for the rows corresponding to SKU i and 0 otherwise. The code to build such features proceeds as follows:

```
#copy the sales dataset into sales_fe_sku
sales_fe_sku = sales.copy()

#encoding of the sku feature
sales_fe_sku = pd.get_dummies(data=sales_fe_sku, columns=['sku'])

#we need to keep the sku feature to build the dictionaries later
sales_fe_sku['sku'] = sales['sku']
```

In the centralized model, we previously had a total of 48 features. In the centralized approach with SKU-fixed effects, we have added one binary feature for each of the 44 SKUs, hence reaching a total of 92 features.

We can leverage the same code as before to build the dictionary for sales_fe_sku (“fe” stands for fixed effects) instead of the sales dataset, split the data, and finally evaluate the centralized with sku-fixed effects model.

```
skuSet = list(sales_fe_sku.sku.unique()) #the SKU numbers do not
change
skuData = {}
colnames = [i for i in sales_fe_sku.columns if i not in
['week', 'weekly_sales', 'sku']]
for i in skuSet:
    df_i = sales_fe_sku[sales_fe_sku.sku == i]
    skuData[i] = {'X': df_i[colnames].values,
                  'y': df_i.weekly_sales.values}

X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:
    X_train_i, X_test_i = np.split(skuData[i]['X'], [68]) #split for X
    y_train_i, y_test_i = np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling
    dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i)
    y_train += list(y_train_i)
```

Once the relevant training and test datasets are created, we can estimate the model parameters by using the training set and evaluate the performance on the test set.

```
import time
tZero=time.time()

X_cen_train=X_dict[skuSet[0]]['train'] #initialization with item 0
X_cen_test = X_dict[skuSet[0]]['test']

for i in skuSet[1:]: #Iteration over items
    X_cen_train=np.concatenate((X_cen_train, X_dict[i]
                                ['train']), axis = 0)
    X_cen_test = np.concatenate((X_cen_test, X_dict[i] ['test']),
                                axis = 0)
```

(continued)

```

model_cen = LinearRegression().fit(X_cen_train, y_train)
print('OOS R2:', round(r2_score(y_test, model_cen.predict(
    X_cen_test)), 3))
print('OOS MSE:', round(mean_squared_error(y_test, model_cen.
    predict(X_cen_test)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec.')

```

The results are given by:

```

OOS R2: 0.298
OOS MSE: 77763.005
Time to compute: 0.046 sec

```

We can then print the values of the estimated coefficients:

```

fixed_effect_sku =
    pd.DataFrame(zip(colnames_fe_sku, model_cen.coef_),
        columns=['feature', 'coef'])

```

Here is a snapshot of the estimated coefficients (we only report the coefficients for the price variables, featured in the main page, and trend) (Table 3.3):

As we can see, based on our dataset, adding the binary variables sku_i (for $i = 1, \dots, 44$) improves the prediction accuracy relative to the centralized approach. One can interpret these sku_i coefficients as the average correction in the weekly sales values for each SKU relative to the average sales across all SKUs. At a high level, when the SKUs follow different sales patterns, adding SKU-fixed effects often improves the model prediction. More generally, one can consider adding different types of fixed effects, such as category, sub-category, and brand. For example, all SKUs from the same brand may have the same intercept. The right way of setting the granularity and structure of the fixed effects highly depends on the business context and on the data.

Table 3.3 Estimated coefficients for the centralized approach with SKU-fixed effects

Feature	Coefficient
price	-1.71
price-1	-0.16
price-2	0.87
feat_main_page	61.5
trend	-1.88

3.8 Centralized Approach with Price-Fixed Effects

An alternative extension of the centralized model involves incorporating price-fixed effects, that is, considering a different price coefficient for each SKU:

$$\text{weekly sales} = \beta_{\text{price}} \times X_{\text{price}} + \beta_{\text{price}-1} \times X_{\text{price}-1} + \dots + \beta_{\text{vendor}10} \times X_{\text{vendor}10} \\ + \beta_{\text{price}1} \times X_{\text{price}1} + \dots + \beta_{\text{price}44} \times X_{\text{price}44} + \varepsilon.$$

Each $X_{\text{price } i}$ is a variable equal to the price of SKU i for the rows corresponding to SKU i and 0 otherwise. This model partially captures the complementary and substitution effects between different SKUs. We illustrate how to construct the price-fixed effects variables for three SKUs in Table 3.4.

From a practical perspective, one can construct the price-fixed effect variables by multiplying the `price` feature with the binary variables `sku_i` by using the following for loop:

```
sales_fe_price = sales.copy()

for i in range(1,45):
    sales_fe_price['price_fixed_effect_'+str(i)] =
        sales_fe_sku['price']*sales_fe_sku['sku_'+str(i)]
```

One can then check that we have achieved our desired goal by inspecting the price-fixed effect values for SKU 5 (the output is presented in Table 3.5):

```
sales_fe_price[sales_fe_price.sku==5] [
    ['price_fixed_effect_5', 'price_fixed_effect_6'] ]
```

The column `price_fixed_effect_5` contains the price values for SKU 5, while the column `price_fixed_effect_6` only contains zeros since the price-fixed effect of SKU 6 should not be applied to SKU 5. Similarly, we can display these values for SKU 6 (the output is presented in Table 3.6):

Table 3.4 Example of price-fixed effects variables for three SKUs

SKU	price	Price-fixed effect 1	Price fixed effect 2	Price fixed effect 3
1	10.6	10.6	0	0
2	15.8	0	15.8	0
3	8.5	0	0	8.5

Table 3.5 Price-fixed effect values for SKU 5

Week	SKU	price_fixed_effect_5	price_fixed_effect_6
2016-11-14	5	8.08	0.0
2016-11-21	5	8.99	0.0
2016-11-28	5	8.10	0.0
2016-12-05	5	6.18	0.0
2016-12-12	5	8.06	0.0
...	...	...	...
2018-08-27	5	10.09	0.0
2018-08-27	5	10.99	0.0
2018-09-10	5	8.99	0.0
2018-09-17	5	8.99	0.0
2018-09-17	5	10.99	0.0

Table 3.6 Price-fixed effect values for SKU 6

Week	SKU	price_fixed_effect_5	price_fixed_effect_6
2016-11-14	6	0.0	20.99
2016-11-21	6	0.0	34.18
2016-11-28	6	0.0	20.94
2016-12-05	6	0.0	19.86
2016-12-12	6	0.0	17.10
...	...	...	...
2018-08-27	6	0.0	41.99
2018-08-27	6	0.0	41.99
2018-09-10	6	0.0	35.22
2018-09-17	6	0.0	63.99
2018-09-17	6	0.0	43.99

```
sales_fe_price[sales_fe_price.sku==6] [
    ['week', 'sku', 'price_fixed_effect_5', 'price_fixed_effect_6'] ]
```

As in the case of SKU-fixed effects, we build the new dataset `sales_fe_price`. We add the price variable to the list of features to remove given that we are already using the `price_fixed_effect` variables for each SKU.

```

X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:
    X_train_i,X_test_i=np.split(skuData[i]['X'], [68]) #split for X
    y_train_i,y_test_i=np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling
                dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i)
    y_train += list(y_train_i)

```

```

X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:
    X_train_i,X_test_i=np.split(skuData[i]['X'], [68]) #split for X
    y_train_i,y_test_i=np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling
                dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i)
    y_train += list(y_train_i)

```

Once the relevant training and test datasets are created, we can estimate the model parameters by using the training set and evaluate the performance on the test set.

```

tZero=time.time()

X_cen_train=X_dict[skuSet[0]]['train'] #initialization with item 0
X_cen_test=X_dict[skuSet[0]]['test']

```

(continued)

```

for i in skuSet[1:]: #Iteration over items
    X_cen_train = np.concatenate((X_cen_train, X_dict[i]
                                  ['train']), axis = 0)
    X_cen_test = np.concatenate((X_cen_test, X_dict[i]['test']),
                                axis = 0)

model_cen = LinearRegression().fit(X_cen_train, y_train)
print('OOS R2:', round(r2_score(y_test, model_cen.predict(
    X_cen_test)), 3))
print('OOS MSE:', round(mean_squared_error(y_test,
    model_cen.predict(X_cen_test)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec.')

```

The results are given by:

```

OOS R2: 0.367
OOS MSE: 70077.111
Time to compute: 0.044 sec

```

As we can see, we obtain that in our dataset, incorporating price-fixed effects leads to a better prediction accuracy relative to a model with SKU-fixed effects. We then naturally explore the approach that combines both types of fixed effects.

3.9 Centralized Approach with SKU-Price-Fixed Effects

If we combine both SKU- and price-fixed effects, the specification becomes

$$\begin{aligned}
 \text{weekly sales} = & \beta_{\text{price}} \times X_{\text{price}} + \beta_{\text{price}-1} \times X_{\text{price}-1} + \dots + \beta_{\text{vendor}10} \times X_{\text{vendor}10} \\
 & + \beta_{\text{sku}1} \times X_{\text{sku}1} + \dots + \beta_{\text{sku}44} \times X_{\text{sku}44} + \beta_{\text{price}1} \times X_{\text{price}1} + \dots \\
 & + \beta_{\text{price}44} \times X_{\text{price}44} + \varepsilon.
 \end{aligned}$$

Starting from the dataset with SKU-fixed effects, we just need to add the price-fixed effects.

```

sales_fe_sku_price = sales_fe_sku.copy()
for i in range(1,45):
    sales_fe_sku_price['price_fixed_effect_'+str(i)] =
        sales_fe_sku_price['price']*sales_fe_sku_price
        ['sku_'+str(i)]
sales_fe_sku_price.head(5)

```

Once again, we build the dataset using the `sales_fe` dataset with the new columns and perform the evaluation of the updated centralized model:

```
skuSet = list(sales.sku.unique()) #the SKU numbers do not change
skuData = {}
colnames =
    [i for i in sales_fe_sku_price.columns
     if i not in ['week', 'weekly_sales', 'sku', 'price']]
for i in skuSet:
    df_i = sales_fe_sku_price[sales_fe_sku_price.sku == i]
    skuData[i] = {'X': df_i[colnames].values,
                  'y': df_i.weekly_sales.values}
```

```
X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:
    X_train_i, X_test_i = np.split(skuData[i]['X'], [68]) #split for X
    y_train_i, y_test_i = np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling
                dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i)
    y_train += list(y_train_i)
```

```
tZero=time.time()

X_cen_train=X_dict[skuSet[0]]['train'] #initialization with item 0
X_cen_test=X_dict[skuSet[0]]['test']

for i in skuSet[1:]: #Iteration over items
    X_cen_train=np.concatenate((X_cen_train, X_dict[i]
                                ['train']), axis=0)
    X_cen_test=np.concatenate((X_cen_test, X_dict[i]['test']),
                               axis=0)
```

(continued)

```

model_cen = LinearRegression(fit_intercept=True).fit
              (X_cen_train, y_train)
print('OOS R2:', round(r2_score(y_test, model_cen.predict
              (X_cen_test)), 3))
print('OOS MSE:', round(mean_squared_error(y_test, model_cen.
              predict(X_cen_test)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec.')

```

The results are given by:

```

OOS R2: 0.58
OOS MSE: 46528.813
Time to compute: 0.07 sec

```

We find that the model that combines both types of fixed effects yields a greater performance relative to the previous models. Given the importance of the price variable, it seems that the price-fixed effects allow us to capture a significant part of the predictive power. Indeed, it is likely that the effect of price on demand is different across SKUs. More generally, one can explore adding fixed effects for each subset of features. A method to identify the optimal subset of features that should be estimated at the aggregate level (i.e., using a centralized approach) versus at the SKU level (i.e., a decentralized approach) will be discussed in Chap. 7.

3.10 Decentralized Approach with Aggregated Seasonality

A common approach in several retail applications is to estimate all the features at the SKU level (i.e., using a decentralized approach) except for the seasonality coefficients. There are many practical settings where the seasonality affects the demand of the different products in a similar fashion (e.g., all the products in the ice cream category admit a higher consumption in the summer). The method we present below allows us to leverage the data from all the SKUs to jointly estimate the coefficients of the seasonal parameters, while still estimating the coefficients related to other features at the SKU level.

In our case, we model the seasonality using monthly variables (i.e., each calendar month has its own additive effect on demand). In this case, the demand equation (assuming a linear regression specification) for each SKU i can be written as follows:

$$\begin{aligned}
 \text{weekly sales}_i = & \beta_{\text{price } i} * X_{\text{price } i} + \beta_{\text{price}-1 i} * X_{\text{price}-1 i} + \dots + \beta_{\text{vendor10 } i} \\
 & * X_{\text{vendor10 } i} + \beta_{\text{month } 1} * X_{\text{month } 1} + \dots + \beta_{\text{month } 12} * X_{\text{month } 12} + \varepsilon.
 \end{aligned}$$

As discussed, the month coefficients are the same for all SKUs. We highlight that the same approach can be applied when the seasonality is estimated at the week level (i.e., a different coefficient for each of the 52 calendar weeks). We can implement this model and estimate its coefficients by performing similar operations as our previous fixed effects models.

At a high level, we implement a centralized approach with fixed effects for all the features except monthly variables. We start by defining all the features that we want to estimate at the SKU level (i.e., all the features except monthly seasonality coefficients):

```
colnames_to_fix = [i for i in sales.columns if i not in
                   ['week', 'weekly_sales', 'sku',
                    'month_2', 'month_3', 'month_4', 'month_5',
                    'Month_6', 'month_7', 'month_8', 'month_9',
                    'month_10', 'month_11', 'month_12']]
```

We then create a new dataset for this model, which we call `sales_seasonality`. For each of the identified features (i.e., all features but seasonality), we create 44 SKU-fixed effects by following the same procedure as in the previous parts.

```
sales_seasonality = sales_fe_sku.copy()

for feature in colnames_to_fix:
    for i in range(1,45):
        sales_seasonality[str(feature)+'_fixed_effect_'+str(i)] =
            sales_seasonality[feature]*sales_seasonality
                                ['sku_'+str(i)]
sales_seasonality.head(5)
```

The above code returns a sales dataset with SKU-fixed effects for all features. For example,

- `feat_main_page_fixed_effect_9` captures the impact of being featured on the main page, specifically for the weekly sales of SKU 9.
- `functionality_02.portable_smartphone_chargers_fixed_effect_22` represents the impact of the portable smartphone chargers functionality on the weekly sales, specifically for SKU 22. In our case, only the `week`, `weekly_sales`, `sku`, and `month` variables should not have fixed effects.

The next step is to structure our training and test sets. As before, we remove `week`, `weekly_sale`, and `sku` from the training features. In this case, we also need to remove the original features that were created at the SKU level to avoid duplicating features. For example, we need to remove the original price feature since

we created 44 fixed effects versions of this feature (in the code below, we call the removed features `colnames_to_fix`):

```
skuSet = list(sales.sku.unique()) #the SKU numbers do not change
skuData_seasonality = {}
colnames_seasonality =
    [i for i in sales_seasonality.columns
     if i not in ['week', 'weekly_sales', 'sku']
     and i not in colnames_to_fix]
for i in skuSet:
    df_i = sales_seasonality[sales_seasonality.sku == i]
    skuData_seasonality[i] = {'X': df_i[colnames_seasonality].values,
                              'y': df_i.weekly_sales.values}
```

The rest of the process is exactly the same as before, and thus omitted for conciseness.

The results are given by:

```
OOS R2: 0.616
OOS MSE: 42516.749
Time to compute: 2.658 sec
```

Interestingly, the decentralized approach with seasonality coefficients estimated at the aggregate level returns the best performance we have obtained so far. Recall that the only difference between this model and the decentralized approach (which led to a 0.517 OOS R^2) is the level at which we consider monthly variables. The improved performance helps validate our hypothesis that in our dataset, the seasonality seems to affect the demand of the various SKUs homogeneously. As a result, pooling the observations across all SKUs allows us to estimate the seasonality coefficients more accurately. Considering different aggregation levels to estimate certain features is a powerful tool that can often improve prediction accuracy. We will present a more advanced data-driven method to systematically find the right level of data aggregation in Sect. 7.2 in Chap. 7.

3.11 Summary and Next Steps

We observe that in our dataset, the decentralized approach clearly outperforms the centralized approach (in terms of OOS R^2 and MSE). However, the decentralized approach happens to be three times longer to train and to evaluate relative to the centralized counterpart (given the small size of our dataset, the computing times remain below 0.1 second, but this could become an issue for larger-scale settings). It is also likely that the decentralized approach would yield a lower performance for SKUs with fewer observations relative to SKUs with a larger number of

observations. We have also observed that it is possible to improve the prediction accuracy by applying feature regularization and feature engineering (e.g., log-transformations).

We next aim to further investigate the following aspects:

- In Chap. 4, we consider machine-learning techniques, and in particular tree-based methods. Given the small size of our dataset, we will not consider any deep-learning-based techniques (such techniques will be briefly discussed in Chap. 7).
- In the decentralized model, we have considered each SKU in its own right without leveraging potential similarities among certain SKUs. In fact, it is likely that the demand for similar items (e.g., different TVs with the same size and specifications) would share similar patterns. Another example is the same products with different colors or sizes. It is not clear how to take this into account by using the decentralized approach. In Chap. 5, we discuss how to leverage and implement clustering algorithms that will group similar SKUs together and estimate a demand prediction model for each cluster of SKUs. By doing so, we will build models that are in-between the centralized and decentralized approaches.

References

- Casella, G., R. Berger, 2002, *Statistical Inference*, second edition.
- Hastie, T., Tibshirani, R. and Friedman, J., 2009. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media.

Chapter 4

Tree-Based Methods



In this section, we explore tree-based methods for demand prediction. These techniques are widely used given their strong predictive power. There is often a trade-off between using a simple Decision Tree versus more complex methods, in terms of simplicity, interpretability, and prediction accuracy.

We will consider three types of methods: Decision Tree, Random Forest, and Gradient Boosted Tree. These methods are commonly used in machine learning, mainly for classification tasks. In this book, we will leverage these methods for demand prediction. We will apply these methods under both the centralized and decentralized approaches.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 4/Tree-Based Methods.ipynb

Before starting this section, we remind the reader of the Python code for data pre-processing.

```
sales=pd.read_csv('data_processed.csv')

## Structure by SKU
skuSet = list(sales.sku.unique())
skuData = {}
colnames = [i for i in sales.columns if i not in
             ['week', 'weekly_sales', 'sku']]
for i in skuSet:
    df_i = sales[sales.sku == i]
    skuData[i] = {'X': df_i[colnames].values,
                  'Y': df_i.weekly_sales.values}
```

(continued)

```

## Decentralized Structure
X_dict = {}
y_dict = {}

skuModels = {}
y_pred = []
y_test = []
y_train = []

for i in skuSet:
    X_train_i, X_test_i = np.split(skuData[i]['X'], [68]) #split for X
    y_train_i, y_test_i = np.split(skuData[i]['y'], [68]) #split for y

    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i) #creating the complete training array
    y_train += list(y_train_i) #creating the complete testing array

## Centralized Structure
X_cen_train = X_dict[skuSet[0]]['train'] #initialization with
                                     item 0
X_cen_test = X_dict[skuSet[0]]['test']

for i in skuSet[1:]: #Iteration over items
    #Bringing together the training set
    X_cen_train = np.concatenate((X_cen_train, X_dict[i]['train']),
                                  axis = 0)
    X_cen_test = np.concatenate((X_cen_test, X_dict[i]['test']),
                                  axis = 0)

```

4.1 Decision Tree

A Decision Tree is a collection of decision nodes, connected by branches, extending downward from the root node to terminating (or leaf) nodes. Decision Trees are a popular classification method but can also be used as a regression method.¹ The major advantages of a Decision Tree are the relative ease of understanding the concept (and its interpretability), the speed of training, the convenience of handling missing values, and the robustness to outliers. A major drawback is related to the high variance commonly associated with the results, meaning that the results are often very sensitive to small changes in the training data. In short, a Decision Tree

¹Hastie et al. (2009).

might significantly change even when we replace only a couple of samples in the training set.

We will use the `DecisionTreeRegressor` model of `sklearn`.² We start by considering the centralized approach (i.e., estimating a single joint Decision Tree for all 44 SKUs).

4.1.1 Centralized Decision Tree

An important aspect of Decision Trees is that one should specify several model hyperparameters. In this section, we will seek to find the optimal values for the following two hyperparameters:

- `max_features`: This is the number of features to consider in each decision node. For example, our dataset contains 45 predictors. If the `max_features` is set to 10, each time we need to find the best split (i.e., identifying the optimal parameters of the decision node and which parameter to use as the predictor), we randomly select 10 features and decide which one is the best feature to use among these (and not among the total list of 45). In the next decision node, we select another draw of 10 features, and so on. We use this technique to mitigate the potential overfitting issue. We will test all the values between 2 and 45 (the total number of features in our dataset).
- `max_depth`: This corresponds to the maximum depth of the tree (i.e., the length of the longest path from the root node to a leaf node). A depth that is too large can lead to overfitting, so we want to specify an appropriate `max_depth` value.

Note that there are additional hyperparameters, such as the minimum number of points in a leaf and the maximum number of leaves, just to name a few. All the hyperparameters are detailed in the documentation of the `sklearn` package. One needs to fine-tune the different model parameters to achieve a strong predictive performance, while mitigating the overfitting issue. In this book, we decided to focus on the two hyperparameters, `max_features` and `max_depth`, because they allow us to introduce two interesting concepts. However, one can further expand on this topic and consider additional hyperparameters as detailed in the `sklearn` documentation.

This raises the following question: How should one select the above two hyperparameters? We will discuss one approach but of course several approaches exist.

²<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeRegressor.html>.

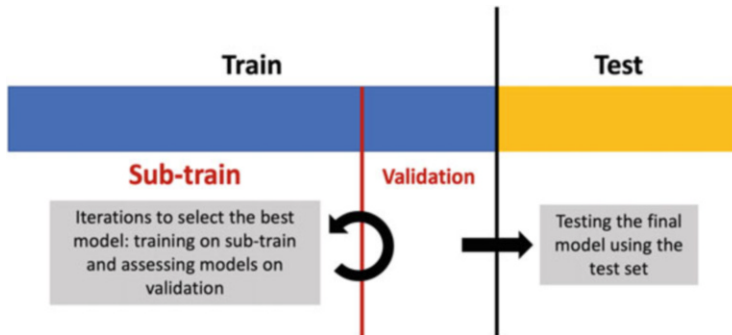


Fig. 4.1 Splitting the training set into two subsets

4.1.1.1 Selecting the Parameters

To mitigate the risk of overfitting, we can split the training set into two subsets (e.g., by using a time-based split so as to preserve the temporal structure of the data). The training set is split into a training subset (sub-train) and a test subset (often called a validation set). The models are trained on the sub-training set, and the validation set is used to select the hyperparameters. Once the hyperparameters are selected, we can train the final model using the entire training set and record the performance on the test set. This procedure is very standard in a multitude of settings.³ An illustration of this procedure is presented in Fig. 4.1.

The process proceeds in three steps:

Step 1: Building and structuring the datasets

- **Train:** Split the train set into sub-train and validation
 - We perform a 70–30 split of the training data: 70% for sub-train (48 weeks) and 30% for validation (20 weeks).
 - We also keep in mind that the entire training dataset will be used later in the process.
- **Test:** The test set remains the same as before, that is, the dataset used to compute the OOS R^2 for all previous models.

Step 2: Iterations to select the best model

- We want to compare the performance of the models with different hyperparameter values. To this end, we proceed as follows:
 - For each combination of hyperparameters, we train a model using the sub-train data and compute the R^2 on the validation set.

³Hastie et al. (2009).

- We select the hyperparameters that lead to the best R^2 on the validation set. These parameters are said to be optimal because they optimize our metric of reference (R^2).
- The iterations will be based on the different tested combinations of hyperparameters. Depending on the type of model and on the resulting computing time, we will perform either an exhaustive grid search or a random search.

Step 3: Testing the final model using fresh data

- Training the model:
 - We use the hyperparameters previously selected (the ones that led to the best model in Step 2).
 - We re-train the model using the entire training set (i.e., sub-train set plus validation set).
 - This model will be later called the best model or the final model.
- Computing the OOS R^2 on the test set:
 - The data is called “fresh” because it has not been used either to train the model or to determine the hyperparameter values.

We next present the code. We will first split and structure the training set.

```
x_dict_subsplit = {}
y_dict_subsplit = {}

y_validation = []
y_subtrain = []

for i in skuSet:
    X_train_i, X_test_i = np.split(X_dict[i]['train'], [48])
                                # 70%*70%*98=48 weeks
    y_train_i, y_test_i = np.split(y_dict[i]['train'], [48])

    x_dict_subsplit[i] = {'train': X_train_i, 'test': X_test_i}
                                #filling dictionary
    y_dict_subsplit[i] = {'train': y_train_i, 'test': y_test_i}

    y_validation += list(y_test_i) #creating the complete training
                                array
    y_subtrain += list(y_train_i) #creating the complete testing
                                array
```

We then structure the centralized dataset. We reorder the rows to group them by SKU. This will enable us to identify which rows are related to which SKU.

```

X_cen_subtrain = X_dict_subsplit[skuSet[0]]['train']
#initialization with item 0
X_cen_validation = X_dict_subsplit[skuSet[0]]['test']
#X_cen_test_sub

for i in skuSet[1:]: #Iteration over items
    X_cen_subtrain = np.concatenate((X_cen_subtrain,
                                     X_dict_subsplit[i]['train']), axis = 0)
    X_cen_validation = np.concatenate((X_cen_validation,
                                       X_dict_subsplit[i]['test']), axis = 0)

```

We then perform iterations to select the best model and we finally test it on fresh data. We conduct a random search with 50 iterations to test different combinations of parameters.

Note that we conduct a random search instead of a grid (or exhaustive) search. As opposed to a grid search, where all combinations of parameters are tested within a specific increment, a random search consists in randomly selecting combinations of parameters at each step and assessing the performance of each combination. This method typically allows testing parameters from larger ranges and requires a reduced computing power. In addition, when compared to a grid search, this technique has been proven to yield good results while testing a smaller number of combinations, and way better results for the same number of combinations tested. More details can be found in the literature.⁴

The number of iterations is chosen to be 50 as a trade-off between computing speed and performance of the selection, but one may potentially test alternative values. For replicability purposes, we set the random seed to a specific value (`random.seed(5)`). This will ensure that the results are consistent when running the same code at different times or on different computers. The choice of the random seed value was driven by the desire to obtain illustrative results.

```

import random
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score

max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
params=[]
maximum_score=0

## Random selection of parameters to test
random.seed(5)

```

(continued)

⁴Bergstra and Bengio (2012).

```

mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)

## Iterations to select best model
for i in range (50):
    print('model number:', i+1)
    #selection of parameters to test
    mf = mf_[i]
    md = md_[i]
    print(' parameters:', [mf,md])
    #model
    DT_cen = DecisionTreeRegressor(max_features=mf,
                                   max_depth=md,
                                   random_state=0
                                   ).fit(X_cen_subtrain, y_subtrain)
    score=r2_score(y_validation, DT_cen.predict(X_cen_validation))
    print(' R2:', score)
    #compare performances on validation data
    if score > maximum_score:
        params = [mf,md]
        maximum_score = score

## Test on fresh data
mf,md=params
DT_cen = DecisionTreeRegressor(max_features=mf,
                               max_depth=md,
                               random_state=0
                               ).fit(X_cen_train, y_train)
oos_r2=r2_score(y_test, DT_cen.predict(X_cen_test))

print('\nBest Model:')
print('Parameters:', params)
print('Validation R2:', maximum_score)
print('OOS R2:', oos_r2)

```

Similarly, the hyperparameter `max_features` introduces some randomness into the model. To ensure consistent comparisons in terms of hyperparameters, we set the random state to a specific value (by using the command `random_state=0`).

As discussed, the best model is the one that is using the optimal identified hyperparameters and trained using the entire training dataset. In our case, the best model is characterized by the following parameter values and performance:

```

Best Model:
Parameters: [17, 4]
Validation R2: 0.570
OOS R2: 0.159

```

Remark: A common extension of the previous method is to use a k-fold cross-validation procedure. The idea is to split the training set into k folds of equal size.

The parameter k represents the number of groups that the dataset is to be split into. Then, $k-1$ groups are used as a training subset and the last fold as a testing subset. This process is repeated k times (for all train-test combinations), and we compare the average R^2 (over the k values). We then record the OOS R^2 (on the test set) by using the best model hyperparameters obtained from the previous step. To preserve the temporal structure of the data and for conciseness, we will not implement this method, but this approach is widely used in practice (especially for non-time-series data). For more details on cross-validation, see the `sklearn` documentation.⁵

We next further analyze the above best model. We first run it separately to assess its computing speed. We then plot a Decision Tree to interpret the estimated model.

4.1.1.2 Focusing on the Best Model

We want to assess the computing speed of the above model.

```
tZero=time.time()

DT_cen = DecisionTreeRegressor(max_features=17,
                               max_depth=4,
                               random_state=0
                              ).fit(X_cen_train, y_train)
print('OOS R2:', round(r2_score(y_test,
                                DT_cen.predict(X_cen_test)), 3))

t = time.time() - tZero
print('Time to compute:', round(t, 3), ' sec')
```

The results are given by:

```
OOS R2: 0.159
Time to compute: 0.009 sec
```

The OOS R^2 obtained with this method is 0.159, which is quite low. However, a significant advantage of this model is its low computing time.

4.1.1.3 Example of a Plotted Tree

A significant advantage of Decision Tree is the interpretability of the estimated model. To illustrate this point, we plot a Decision Tree in Fig. 4.2. We intentionally use a low value for the parameter `max_depth` to render the tree more interpretable

⁵https://scikit-learn.org/stable/modules/cross_validation.html.

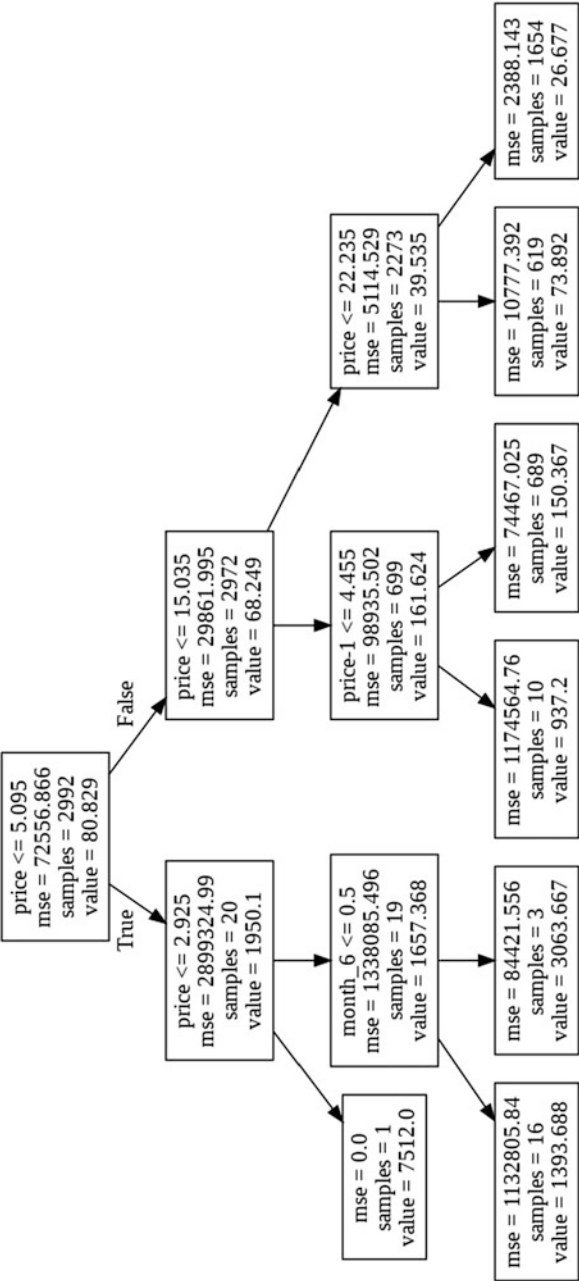


Fig. 4.2 Illustration of a Decision Tree (centralized approach)

with a compact visualization. The code to generate this visualization is presented below. We first report the prediction accuracy of this Decision Tree.

```
DT_cen_visualization = DecisionTreeRegressor(max_features=43,
                                              max_depth=3,
                                              random_state=0
                                              ).fit(X_cen_train,
                                                    y_train)
print('OOS R2', r2_score(y_test, DT_cen_visualization.predict(
    X_cen_test)))
```

The performance is given by:

OOS R2: 0.118

We highlight that the prediction accuracy is comparable with the model we identified in our previous analysis, but it enables a more compact visualization.

```
import matplotlib.pyplot as plt
from sklearn.tree import plot_tree

## Print the tree
plt.figure(figsize=(15,8), dpi=400)
plot_tree(DT_cen_visualization, feature_names = colnames)
plt.savefig("visualization_decision_tree.png",
            bbox_inches='tight')
plt.show()
```

The plot in Fig. 4.2 provides a good illustration of the interpretability advantage of Decision Trees. In this example, we infer that the price is the major discriminant feature (5/6 splits are determined based on the price or the lag-price).

At the same time, we can also understand one of the main drawbacks of this method, namely, its high variance. As mentioned before, a Decision Tree might substantially vary when we replace a small number of samples in the training set. This is due to the binary nature of the decision nodes. Consider for example that we have a testing observation with a price equal to 5.09. With the current tree (see Fig. 4.2), this observation will be assigned to the left side of the tree. By adding a few training observations, it is clearly possible that the decision criterion $\text{price} \leq 5.095$ is replaced by $\text{price} \leq 5.085$ (which can be perceived as a very minor change). In this case, the above testing observation would now be assigned to the right side of the tree with a much lower predicted sales value.

On the flip side, the above binary nature allows Decision Trees to be robust to outliers. Consider for example that we have a SKU with a very large (erroneous) price value (e.g., \$100,000). This large value is likely to come from an error in the

data collection. For Decision Trees, the impact of such a high price value will be limited (e.g., relative to a linear regression).

4.1.2 *Decentralized Decision Tree*

We now turn our attention to the decentralized approach (i.e., estimating a separate Decision Tree for each SKU). The code to estimate a decentralized Decision Tree goes as follows:

```
mf,md=[## Input parameters ##]
y_pred = []

for i in skuSet:
    model_i = DecisionTreeRegressor(max_features=mf,
                                    max_depth=md,
                                    random_state=0
                                    ).fit(X_dict[i]['train'],
                                          y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))

oos_r2=r2_score(y_test, np.array(y_pred))
```

We next select the hyperparameter values and assess the performance of the final model. The process is similar to the centralized Decision Tree.

4.1.2.1 **Selecting the Parameters**

We run a loop to identify the best parameter values in a similar fashion.

```
max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
params=[]
maximum_score=0

#selection of parameters to test
random.seed(5)
mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)

## Selection of the best model
for i in range(50):
    print('model number:',i+1)
```

(continued)

```

#selection of parameters to test
mf = mf_[i]
md = md_[i]
print(' Parameters:', [mf,md])
#model
y_pred = []
for i in skuSet:
    model_i = DecisionTreeRegressor(max_features=mf,
                                    max_depth=md,
                                    random_state=0
                                    ).fit(X_dict_subsplit[i]['train'],
                                          y_dict_subsplit[i]['train'])
    y_pred += list(model_i.predict(X_dict_subsplit[i]['test']))
score=r2_score(y_validation, np.array(y_pred))
#compare performances on validation data
if score>maximum_score:
    params = [mf,md]
    maximum_score = score

## Test on fresh data
mf,md=params
y_pred = []
for i in skuSet:
    model_i = DecisionTreeRegressor(max_features=mf,
                                    max_depth=md,
                                    random_state=0
                                    ).fit(X_dict[i]['train'],
                                          y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))
oos_r2=r2_score(y_test, np.array(y_pred))

print('\nBest Model:')
print('Parameters:',params)
print('Validation R2:',maximum_score)
print('OOS R2:', oos_r2)

```

The results are given by:

```

Best Model:
Parameters: [14, 4]
Validation R2: 0.685
OOS R2: 0.399

```

Once again, we want to assess the computing speed of this model, so we run it separately.

4.1.2.2 Focusing on the Best Model

```
tZero=time.time()

y_pred = []
for i in skuSet:
    model_i = DecisionTreeRegressor(max_features=14,
                                    max_depth=4,
                                    random_state=0
                                    ).fit(X_dict[i]['train'],
                                          y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))

print('OOS R2:',round(r2_score(y_test, np.array(y_pred)),3))

t = time.time()-tZero
print('Time to compute:',round(t,3), ' sec')
```

The results are given by:

```
OOS R2: 0.399
Time to compute: 0.037 sec
```

The conclusion is clear: the results are significantly better, and the computing time is only twice higher (but still very reasonable) relative to the centralized Decision Tree approach.

We highlight that the decentralized OLS model takes only 0.065 second to run and yields an OOS R^2 that is higher (0.52 versus 0.40). In addition, as mentioned before, Decision Trees suffer from a high variance. The method presented in the next section is a common approach that aims to fix the high variance issue.

4.2 Random Forest

The Random Forest method builds multiple Decision Trees and averages their predicted outcomes to deliver a more accurate and stable prediction.⁶ Since it consists of multiple Decision Trees, the Random Forest method will typically have a lower variance but will require a longer training time. In this section, we use the `RandomTreeRegressor` model of `sklearn`.⁷

Once again, the parameters that we decide to fine-tune are the following:

⁶See Hastie et al. (2009).

⁷<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>.

- `max_features` (see IV/1.1).
- `max_depth` (see IV/1.1).

Remark: An additional important parameter of a Random Forest is the number of trees used in the model (called `n_estimators`). A common pattern for this parameter is that using a higher number of trees will lead to a better performance. However, the performance improvement saturates at some point, that is, the marginal gain from adding an additional tree to the model decreases with the number of trees. By default, the Random Forest model of `sklearn` is assigned 100 trees, which is generally beyond the saturation point. More information on this topic can be found in the `sklearn` documentation.

Once again, we start our analysis with the centralized approach.

4.2.1 Centralized Random Forest

The code to estimate a centralized Random Forest proceeds as follows:

```
mf,md,ne=[## Input parameters ##]
RF_cen = RandomForestRegressor(max_features=mf,
                               max_depth=md,
                               random_state=0
                              ).fit(X_cen_train, y_train)
oos_r2=r2_score(y_test, RF_cen.predict(X_cen_test))
```

We next discuss how to identify the hyperparameters (i.e., the `params_rf` value in the above code) and assess the performance of the final model.

4.2.1.1 Selecting the Parameters

The code is presented below.

```
from sklearn.ensemble import RandomForestRegressor

max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
params=[]
maximum_score=0

#selection of parameters to test
random.seed(5)
```

(continued)

```

mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)

## Iterations to select best model
for i in range (50):
    print('Model number:', i+1)
    #selection of parameters to test
    mf = mf_[i]
    md = md_[i]
    #ne = ne_[i]
    print(' Parameters:', [mf, md])
    #model
    RF_cen = RandomForestRegressor(max_features=mf, max_depth=md,
    random_state=0).fit(X_cen_subtrain, y_subtrain)
    score=r2_score(y_validation, RF_cen.predict(X_cen_validation))
    print(' R2:', score)
    #compare performances on validation data
    if score > maximum_score:
        params = [mf, md]
        maximum_score = score

## Test on fresh data
mf, md=params
RF_cen = RandomForestRegressor(max_features=mf, max_depth=md,
    random_state=0).fit(X_cen_train, y_train)
oos_r2=r2_score(y_test, RF_cen.predict(X_cen_test))

print('\nBest Model:')
print('Parameters:', params)
print('Validation R2:', maximum_score)
print('OOS R2:', oos_r2)

```

The results are given by:

```

Best Model:
Parameters: [31, 4]
Validation R2: 0.457
OOS R2: 0.272

```

4.2.1.2 Focusing on the Best Model

```
tZero=time.time()

RF_cen = RandomForestRegressor(max_features=31,
                               max_depth=4,
                               random_state=0).fit(X_cen_train,
                                                    y_train)

print('OOS R2:',round(r2_score(y_test,
                               RF_cen.predict(X_cen_test)),3))

t = time.time()-tZero
print("Time to compute:",round(t,3)," sec")
```

The results are given by:

```
OOS R2: 0.272
Time to compute: 0.413 sec
```

As with previous methods, the results based on the centralized approach are not particularly promising. Thus, we consider implementing the Random Forest method based on the decentralized approach (i.e., estimating a Random Forest for each of the 44 SKUs).

4.2.2 Decentralized Random Forest

The code to run a decentralized Random Forest model goes as follows:

```
mf,md,ne=[## Input parameters ##]
y_pred = []
for i in skuSet:
    model_i = RandomForestRegressor(max_features=mf,
                                    max_depth=md,
                                    random_state=0
                                    ).fit(X_dict[i]['train'],
                                          y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))
oos_r2=r2_score(y_test, np.array(y_pred))
```

4.2.2.1 Selecting the Parameters

```

max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
params=[]
maximum_score=0

#selection of parameters to test
random.seed(5)
mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)

## Iterations to select best model
for i in range (50):
    print('Model number:', i+1)
    #selection of parameters to test
    mf = mf_[i]
    md = md_[i]
    print(' Parameters:', [mf,md])
    #model
    y_pred = []
    for i in skuSet:
        model_i = RandomForestRegressor(max_features=mf,
                                         max_depth=md,
                                         random_state=0).fit(X_dict_subsplit[i]['train'] ,
                                                             y_dict_subsplit[i]['train'])
        y_pred += list(model_i.predict(X_dict_subsplit[i]['test']))
    score=r2_score(y_validation, np.array(y_pred))
    #compare performances on validation data
    if score > maximum_score:
        params = [mf,md]
        maximum_score = score

## Test on fresh data
mf,md=params
y_pred = []
for i in skuSet:
    model_i = RandomForestRegressor(max_features=mf, max_depth=md,
                                    random_state=0).fit(X_dict[i]['train'] , y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))
oos_r2=r2_score(y_test, np.array(y_pred))

print('\nBest Model:')
print('Parameters:',params)
print('Validation R2:', maximum_score)
print('OOS R2:', oos_r2)

```

The results are given by:

```
Best Model:
Parameters: [44, 8]
Validation R2: 0.573
OOS R2: 0.559
```

4.2.2.2 Focusing on the Best Model

```
y_pred = []
for i in skuSet:
    model_i = RandomForestRegressor(max_features=44,
                                    max_depth=8,
                                    random_state=0).fit
                                    (X_dict[i]['train'],
                                     y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))

print('OOS R2:', round(r2_score(y_test, np.array(y_pred)), 3))

t = time.time() - tZero
print("Time to compute:", round(t, 3), " sec")
```

The results are given by:

```
OOS R2: 0.559
Time to compute: 5.811 sec
```

As we can see, the required computing time has significantly increased. For applications that involve a large number of observations and features, this can be a critical issue, and in such cases, the decentralized Random Forest may not be an appropriate method. At the same time, the OOS R^2 is higher relative to the previous methods we considered, and thus this seems promising. In the next section, we study an improved version of Random Forest, with the goal of increasing further the OOS R^2 .

4.3 Gradient-Boosted Tree

Gradient-Boosted Tree (or Gradient Boosting) is another method that trains multiple trees for prediction.⁸ The basic idea is that newly grown trees can learn from the errors of the previous ones. This generally leads to an improved performance.

⁸See Hastie et al. (2009), Chen and Guestrin (2016).

In our case, the small size of our dataset mitigates this potential advantage. We use the `GradientBoostingRegressor` model of `sklearn`.⁹

The parameters to tune are the following:

- `max_features` (see IV/1.1).
- `max_depth` (see IV/1.1).
- `learning_rate`—The learning rate parameter shrinks the contribution of each tree. A higher learning rate corresponds to a more aggressive and faster learning process from one tree to the next but will make the model more prone to overfitting. In our implementation below, we consider `learning_rate` with values `[0.01, 0.05, 0.1, 0.5]`.

As before, we start our analysis with the centralized approach.

4.3.1 Centralized Gradient Boosted Tree

The code to estimate a centralized Gradient-Boosted Tree proceeds as follows:

```
mf,md,ne,lr=params #input the parameters
GB_cen = GradientBoostingRegressor(max_features=mf,
                                   max_depth=md,
                                   learning_rate=lr,
                                   random_state=0).fit(X_cen_train,
                                                       y_train)
oos_r2=r2_score(y_test, GB_cen.predict(X_cen_test))
```

We next select the hyperparameters and assess the performance of the final model.

4.3.1.1 Selecting the Parameters

```
max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
learning_rate_ = [0.01, 0.05, 0.1, 0.5]
params=[]
maximum_score=0
```

(continued)

⁹<https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>.

```

#selection of parameters to test
random.seed(5)
mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)
lr_ = random.choices(learning_rate_, k=50)

from sklearn.ensemble import GradientBoostingRegressor
## Iterations to select best model
for i in range(50):
    print('Model number:', i+1)
    #selection of parameters to test
    mf = mf_[i]
    md = md_[i]
    lr = lr_[i]
    print('Parameters:', [mf, md, lr])
    #model
    GB_cen = GradientBoostingRegressor(max_features=mf,
                                       max_depth=md,
                                       learning_rate=lr,
                                       random_state=0).fit(
                                           X_cen_subtrain,
                                           y_subtrain)

    score=r2_score(y_validation, GB_cen.predict(X_cen_validation))
    print(' R2:', score)
    #compare performances on validation data
    if score > maximum_score:
        params = [mf, md, lr]
        maximum_score = score

## Test on fresh data
mf, md, lr=params
GB_cen = GradientBoostingRegressor(max_features=mf,
                                   max_depth=md,
                                   learning_rate=lr,
                                   random_state=0).fit(X_cen_train,
                                                       y_train)

oos_r2=r2_score(y_test, GB_cen.predict(X_cen_test))

print('\nBest Model:')
print('Parameters:', params)
print('Validation R2:', maximum_score)
print('OOS R2:', oos_r2)

```

The results are given by:

```

Best Model:
Parameters: [14, 7, 0.5]
Validation R2: 0.476
OOS R2: 0.223

```

With the best parameter values, we also assess the computing speed, and compare the Gradient-Boosted Tree relative to other tree-based methods.

4.3.1.2 Focusing on the Best Model

```
tZero=time.time()

GB_cen = GradientBoostingRegressor(max_features=14,
                                   max_depth=7,
                                   learning_rate=0.5,
                                   random_state=0).fit
                                   (X_cen_train, y_train)

print('OOS R2:',round(r2_score(y_test, GB_cen.predict
                              (X_cen_test)),3))

t = time.time()-tZero
print("Time to compute:",round(t,3)," sec")
```

The results are given by:

```
OOS R2: 0.223
Time to compute: 0.380 sec
```

We next consider implementing this method in a decentralized fashion.

4.3.2 Decentralized Gradient-Boosted Tree

The code to estimate a decentralized Gradient-Boosted Tree proceeds as follows:

```
mf,md,ne,lr=[## Input parameters ##]
y_pred = []
for i in skuSet:
    model_i = GradientBoostingRegressor(max_features=mf,
                                        max_depth=md,
                                        learning_rate=lr,
                                        random_state=0
                                        ).fit(X_dict[i]['train'],
                                            y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))
oos_r2=r2_score(y_test, np.array(y_pred))
```

4.3.2.1 Fine-tuning the Parameters

```

max_features_ = list(range(2,45))
max_depth_ = list(range(2,10))
learning_rate_ = [0.01, 0.05, 0.1, 0.5]
params=[]
maximum_score=0

#selection of parameters to test
random.seed(5)
mf_ = random.choices(max_features_, k=50)
md_ = random.choices(max_depth_, k=50)
lr_ = random.choices(learning_rate_, k=50)

## Iterations to select best model
for i in range(50):
    print('Model number:', i+1)
    #selection of parameters to test
    mf = mf_[i]
    md = md_[i]
    lr = lr_[i]
    print(' Parameters:', [mf,md,lr])
    #model
    y_pred = []
    for i in skuSet:
        model_i = GradientBoostingRegressor(max_features=mf,
                                             max_depth=md,
                                             learning_rate=lr,
                                             random_state=0).fit
                                             (X_dict_subsplit[i]
                                              ['train'],
                                              y_dict_subsplit[i]
                                              ['train'])

        y_pred += list(model_i.predict(X_dict_subsplit[i]['test']))
    score=r2_score(y_validation, np.array(y_pred))
    print(' R2:', score)
    #compare performances on validation data
    if score > maximum_score:
        params = [mf,md,lr]
        maximum_score = score

## Test on fresh data
mf,md,lr=params
y_pred = []
for i in skuSet:
    model_i = GradientBoostingRegressor(max_features=mf,
                                         max_depth=md,
                                         learning_rate=lr,

```

(continued)

```

                                random_state=0).fit
                                (X_dict[i]['train'],
y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))
oos_r2=r2_score(y_test, np.array(y_pred))

print('\nBest Model:')
print('Parameters:',params)
print('Validation R2:',maximum_score)
print('OOS R2:', oos_r2)

```

The results are given by:

```

Best Model:
Parameters: [31, 4, 0.5]
Validation R2: 0.607
OOS R2: 0.497

```

4.3.2.2 Focusing on the Best Model

```

tZero=time.time()
y_pred = []
for i in skuSet:
    model_i = GradientBoostingRegressor(max_features=31,
                                        max_depth=4,
                                        learning_rate=0.5,
                                        random_state=0).fit
                                        (X_dict[i]['train'],
y_dict[i]['train'])
    y_pred += list(model_i.predict(X_dict[i]['test']))

print('OOS R2:',round(r2_score(y_test, np.array(y_pred)),3))

t = time.time()-tZero
print("Time to compute:",round(t,3)," sec")

```

The results are given by:

```

OOS R2: 0.497
Time to compute: 1.421 sec

```

We next compare all the methods in terms of OOS R^2 and in terms of running time.

Table 4.1 OOS R^2 and running time for tree-based methods

Model	OOS R^2	Running time (sec)
Centralized Decision Tree	0.159	0.009
Decentralized Decision Tree	0.399	0.037
Centralized Random Forest	0.272	0.413
Decentralized Random Forest	0.559	5.811
Centralized Gradient-Boosted Tree	0.223	0.380
Decentralized Gradient-Boosted Tree	0.497	1.421

4.4 Methods Comparison

We compare all the tree-based methods we implemented using our dataset in terms of OOS R^2 and running time in Table 4.1. For each method, we select the final model by finding the best performing values of the hyperparameters.

As expected, the Decision Tree yields a lower performance in terms of prediction accuracy relative to Random Forest and Gradient-Boosted Tree. In our case, the best results are achieved by the Random Forest method. The results of the Gradient Boosted-Tree are not as good, which potentially indicates that we do not have a large enough dataset to fully leverage the power of this method. For this reason, we will not consider alternative methods that typically require a large dataset, such as deep-learning techniques. If running time is a concern, one may advocate for an alternative method depending on the size of the dataset and the available computing power.

References

Bergstra, J. and Bengio, Y., 2012. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2).

Chen, T., Guestrin, C., 2016, XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 785–794).

Hastie, T., Tibshirani, R. and Friedman, J., 2009. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media.

Chapter 5

Clustering Techniques



As discussed in Chap. 3, there are advantages and disadvantages when aggregating data across SKUs. Aggregating sales data across all SKUs reduces the noise and allows the model to rely on a larger number of observations but will overlook the fact that different SKUs have differing characteristics. There is also a clear trade-off between performance and running time. Based on this observation, one may want to consider a compromise between the two extreme approaches (centralized and decentralized) by aggregating a group of similar SKUs together. A natural way to do so is by using clustering techniques. We discuss two common clustering techniques: k-means and DBSCAN. Other clustering techniques (e.g., hierarchical clustering, OPTICS) can potentially be applied in a similar fashion.

The associated files for this section can be found in the following website:

<https://demandpredictionbook.com>

- 5/Clustering Techniques.ipynb

5.1 K-means Clustering

5.1.1 Description of K-means Clustering

The k-means clustering method proceeds in the four following steps.¹ The objective is to aggregate similar observations (or records).

- *Step 1:* We randomly assign k records to be the initial centers or means of the clusters (k is the number of clusters and must be determined in advance).

¹See Hastie et al. (2009).

Step 1: Assign clusters**Step 2:** Run a separate model for each cluster

Fig. 5.1 Illustration of the implementation of the k-means clustering method

- *Step 2:* For each record, we find the nearest center (according to a distance metric, such as the Euclidean distance). In this context, the clusters are sets of records with the same nearest center.
- *Step 3:* We now have k clusters. For each of the k clusters, we compute the new cluster center, based on the records present in each cluster.
- *Step 4:* Repeat Steps 2 and 3 until convergence (i.e., when the clusters do not change anymore between two successive iterations) or termination (i.e., requiring the algorithm to stop after a large number of predetermined iterations).

In our case, the idea is to create groups of SKUs that share similar characteristics and will be assigned to the same cluster. To this end, we apply the k-means clustering methods on the 44 SKUs in our dataset.

As presented in Fig. 5.1, our objective is to divide the SKUs into k clusters and subsequently estimate a demand prediction model (e.g., OLS) for each cluster. We proceed as follows:

1. Run the k-means method using the average values of the price and weekly sales of each SKU: We first create a table where the rows are the different SKUs, and the columns are the average values of the price and weekly sales variables. We note that it is possible to use different features (and possibly improve the results). In each cell, we compute the average value of the feature for the focal SKU over the training period. Then, since the scale of the different features has a significant impact on the distance function (which is used to find the closest center), we scale the values.² In the context of clustering, scaling the features is often important. Table 5.1 illustrates this data for SKU 1 and the price and weekly sales features.

²Scaling techniques are presented in Section II/8. Here, we use a Min Max scaler (specifically, we use the `MinMaxScaler` from `sklearn`).

Table 5.1 Example of the table that contains the average values of the predictors

	Price (scaled)	Weekly sales (scaled)
SKU 1	Scaled average price of SKU 1 over the training period	Scaled average weekly sales of SKU 1 over the training period
...		

We next apply the k-means clustering method on this table to aggregate similar SKUs. Following this process, each SKU is assigned to one specific cluster (the clusters identified by k-means are non-overlapping by design). The parameter k is a parameter of the model, and we will detail later how to select its value, along with the features used to perform clustering.

2. Estimate a centralized OLS for each cluster: For each cluster, we estimate an OLS regression (using the observations from all the SKUs assigned to the cluster). This is equivalent to assuming that all the SKUs in the same cluster have the same demand prediction model.

We next dive into the implementation details. We use the k-means model based on the `sklearn` library. More information is available in the `sklearn` documentation.³

```
from sklearn.cluster import KMeans
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import r2_score
scaler = MinMaxScaler()
```

First, we aggregate similar SKUs into clusters.

```
z = [## Input number of clusters ##]

#Clustering
X_clus = np.zeros((len(skuSet), 2))
count = 0
for sku in skuSet:
    X_clus[count, :] =
        np.mean(
            np.concatenate((
                np.array([ [i] for i in X_dict[sku]['train'][:,0] ]),
                np.array([ [i] for i in y_dict[sku]['train'] ])),
                axis=1),
            axis = 0 )
```

(continued)

³<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>.

```

count += 1
X_clus = scaler.fit_transform(X_clus)
kmeans = KMeans(n_clusters=z, random_state=0).fit(X_clus)

```

To obtain the same results every time, we run the code, we set a specific random seed value for the initialization of the k-means clustering method. In the above code, we chose `random_state=0`. Interestingly, however, with the k-means function of `sklearn`, the random seed often has only a marginal impact on the initial centroids and on the final clusters. The initialization method is called k-means++, and more information about it is available in the `sklearn` documentation.

Second, we create a centralized dataset for each cluster and estimate an OLS regression for each cluster. We then create the `y_clus_test` variable with the SKUs in the corresponding order and compute the OOS R^2 .

```

#Loop
y_clus_pred = []
y_clus_test = []
for j in range(z):
    ##Get indices of items in cluster j
    clus_items = list(np.where(kmeans.labels_ == j) [0])
    ##Initialization
    #X
    X_clus_j_train = X_dict[skuSet[clus_items[0]]]['train']
    X_clus_j_test = X_dict[skuSet[clus_items[0]]]['test']
    #y
    y_clus_j_train = list(y_dict[skuSet[clus_items[0]]]['train'])
    y_clus_j_test = list(y_dict[skuSet[clus_items[0]]]['test'])
    ##Loop
    for idx in clus_items[1:]: #Iteration over items
        sku=skuSet[idx]
        #X
        #Bringing together the training set for the cluster
        X_clus_j_train = np.concatenate((X_clus_j_train, X_dict[sku]
                                         ['train']),
                                         axis = 0)
        X_clus_j_test = np.concatenate((X_clus_j_test, X_dict[sku]
                                         ['test']),
                                         axis = 0)
        #y
        y_clus_j_train += list(y_dict[sku]['train'])
        y_clus_j_test += list(y_dict[sku]['test'])
    ##Model
    model_clus_j = LinearRegression().fit(X_clus_j_train, y_clus_j_train)
    y_clus_pred += list(model_clus_j.predict(X_clus_j_test))
    y_clus_test += y_clus_j_test

#Results
print('OOS R2:', r2_score(y_clus_test, y_clus_pred))

```

Now that we understand better the method, we next discuss the following two design choices:

- Which features should we use for clustering the SKUs? Choosing the right set of features can be seen as an art. For example, one can use the predictors (e.g., prices, vendors, colors) and potentially including the weekly sales. More precisely, one can compute the average of historical values (e.g., in the last 3 months or in the last year). Another common approach is to use the standard deviation or variance of each predictor (e.g., the standard deviation of the price of each SKU in the last year). The standard deviation of the variables captures the variability (or stability), and it can be worthwhile to combine SKUs with the same level of variability across the different variables. We will detail the process for the price and weekly sales, but one can potentially replicate it while using a different combination of clustering features.
- How do we select the appropriate number of clusters (i.e., the value of k)? This number can be dictated by business constraints or determined by optimizing a specific objective function. In this section, we will naturally select the number of clusters that maximizes the R^2 on the validation set.

5.1.2 Clustering using Average Price and Weekly Sales

As discussed, to determine the best number of clusters, we can use the selection method described in Chap. 4. Specifically, we run a loop to select the best value of k , which ranges between 2 and 15 (these numbers are chosen by the user and depend on the business context and on the number of SKUs).

We highlight how to evaluate the clustering performance based on the out-of-sample prediction accuracy (in our case, by computing the OOS R^2), which is our ultimate goal. In traditional clustering settings, however, several other metrics and approaches are used for clustering evaluation, such as the elbow method.⁴

Here is the code for the iterative procedure to find the best number of clusters:

```
num_clusters=0
maximum_score=-100
oos_r2=0

## Iterations to find optimal parameter
for z in range(2,15):
    #Clustering
    X_clus = np.zeros((len(skuSet), 2))
```

(continued)

⁴<https://towardsdatascience.com/clustering-evaluation-strategies-98a4006cfc>.

```

count = 0
for sku in skuSet:
    X_clus[count, :] = np.mean(
        np.concatenate((
            np.array([ [i] for i in X_dict_subsplit[sku]
                       ['train'][:,0] ] ),
            np.array([ [i] for i in y_dict_subsplit[sku]
                       ['train'] ] )),
            axis=1),
            axis = 0 )
    count += 1
X_clus = scaler.fit_transform(X_clus)
kmeans = KMeans(n_clusters=z, random_state=0).fit(X_clus)
#Loop
y_clus_pred = [] #y_clus_pred_sub
y_clus_validation = [] #y_clus_test_sub
for j in range(z):
    ##Get indices of items in cluster j
    clus_items = list(np.where(kmeans.labels_ == j) [0])
    ##Initialization
    #X_sub
    X_clus_j_subtrain=X_dict_subsplit[skuSet[clus_items[0]]]['train']
    X_clus_j_validation=X_dict_subsplit[skuSet[clus_items[0]]]['test']
    #y_sub
    y_clus_j_subtrain = list(y_dict_subsplit[skuSet[clus_items
        [0]] ['train']])
    y_clus_j_validation = list(y_dict_subsplit[skuSet
        [clus_items[0]] ['test']])
    ##Loop
    for idx in clus_items[1:]: #Iteration over items
        sku=skuSet[idx]
        #X_sub
        X_clus_j_subtrain = np.concatenate(
            (X_clus_j_subtrain, X_dict_subsplit[sku]
             ['train']), axis = 0)
        X_clus_j_validation = np.concatenate(
            (X_clus_j_validation, X_dict_subsplit[sku]
             ['test']), axis = 0)
        #y_sub
        y_clus_j_subtrain += list(y_dict_subsplit[sku] ['train'])
        y_clus_j_validation += list(y_dict_subsplit[sku] ['test'])
    ##Model
    model_clus_j_sub = LinearRegression().fit(X_clus_j_subtrain,
        y_clus_j_subtrain)
    y_clus_pred += list(model_clus_j_sub.predict(X_clus_j_validation))
    y_clus_validation += y_clus_j_validation

#Comparison of results
score=r2_score(y_clus_validation, y_clus_pred)
print('Number of clusters:', z, '- Validation R2:', score)
if score > maximum_score:
    num_clusters=z
    maximum_score = score

```

We next test the results using fresh data (i.e., we train the model on the entire training set and evaluate the performance on the test set).

```

z=num_clusters
#Clustering
d = len(colnames) #d is the number of columns
X_clus = np.zeros((len(skuSet), 2))
count = 0
for sku in skuSet:
    X_clus[count, :] = np.mean(
        np.concatenate((
            np.array( [ [i] for i in X_dict[sku] ['train']
                        [:,0] ] ),
            np.array( [ [i] for i in y_dict[sku] ['train'] ] ) ),
            axis=1),
            axis = 0 )
    count += 1
X_clus = scaler.fit_transform(X_clus)
kmeans = KMeans(n_clusters=z, random_state=0).fit(X_clus)
#Loop
y_clus_pred = []
y_clus_test = []
for j in range(z):
    ##Get indices of items in cluster j
    clus_items = list(np.where(kmeans.labels_ == j) [0])
    ##Initialization
    #X
    X_clus_j_train = X_dict[skuSet[clus_items[0]]] ['train']
    X_clus_j_test = X_dict[skuSet[clus_items[0]]] ['test']
    #y
    y_clus_j_train = list(y_dict[skuSet[clus_items[0]]] ['train'])
    y_clus_j_test = list(y_dict[skuSet[clus_items[0]]] ['test'])
    ##Loop
    for idx in clus_items[1:]: #Iteration over items
        sku=skuSet[idx]
        #X
        X_clus_j_train = np.concatenate( (X_clus_j_train, X_dict
                                           [sku] ['train']),
                                           axis = 0)
        X_clus_j_test = np.concatenate( (X_clus_j_test, X_dict
                                           [sku] ['test']),
                                           axis = 0)
        #y
        y_clus_j_train += list(y_dict[sku] ['train'])
        y_clus_j_test += list(y_dict[sku] ['test'])
    ##Model
    model_clus_j = LinearRegression().fit(X_clus_j_train,
                                           y_clus_j_train)
    y_clus_pred += list(model_clus_j.predict(X_clus_j_test))
    y_clus_test += y_clus_j_test

```

(continued)

```
#Results
oos_r2=r2_score(y_clus_test, y_clus_pred)

#### Print Results ####

print('\nBest Model:')
print('Number of clusters:', num_clusters)
print('Validation R2:', maximum_score)
print('OOS R2:', oos_r2)
```

The results are given by:

```
Best Model:
Number of clusters: 8
Validation R2: 0.270
OOS R2: 0.566
```

As we can see, the prediction accuracy is satisfactory. More precisely, it outperforms both the centralized-OLS and the decentralized-OLS approaches. Thus, this finding illustrates the fact that clustering is not a compromise between these two approaches but can often be a more effective method. We next investigate including additional features into the clustering step.

5.1.3 Adding Standard Deviations of the Clustering Features

Our next attempt is to include both the average and standard deviation of the price and weekly sales into the clustering method. The standard deviation is computed in a similar fashion as the average value by using the observations from the weeks used to train the model for each SKU (i.e., the sub-train dataset for the validation loop and the train dataset when assessing the performance of the final model). The code is the same as before, except that the clustering step needs to be slightly adapted, as presented below. The complete code is available on the companion website (<https://demandpredictionbook.com>).

```
z = [## Input number of clusters ##]

#Clustering
X_clus = np.zeros((len(skuSet), 4))
count = 0
for sku in skuSet:
    X_clus[count, :] = np.concatenate((
```

(continued)

```

np.mean(
    np.concatenate((
        np.array( [ [i] for i in X_dict[sku] ['train'][:,0] ] ),
        np.array( [ [i] for i in y_dict[sku] ['train'] ] ) ),
        axis=1),
        axis = 0 ),

np.std(
    np.concatenate((
        np.array( [ [i] for i in X_dict[sku] ['train'][:,0] ] ),
        np.array( [ [i] for i in y_dict[sku] ['train'] ] ) ),
        axis=1),
        axis = 0 )),
    axis=0)
count += 1
X_clus = scaler.fit_transform(X_clus)
kmeans = KMeans(n_clusters=z, random_state=0).fit(X_clus)

```

Interestingly, adding the standard deviation of the clustering features (i.e., price and weekly sales) improves the results:

```

Best Model:
Number of clusters: 5
Validation R2: 0.264
OOS R2: 0.560

```

In fact, the above results are the highest we could reach so far in terms of OOS R^2 . The best number of clusters is five. In addition, the computing time remains low:

```

OOS R2: 0.560
Time to compute 0.064

```

We next attempt to visualize the clusters. The code is provided below, and an illustrative plot is presented in Fig. 5.2 (weekly sales as a function of the price).

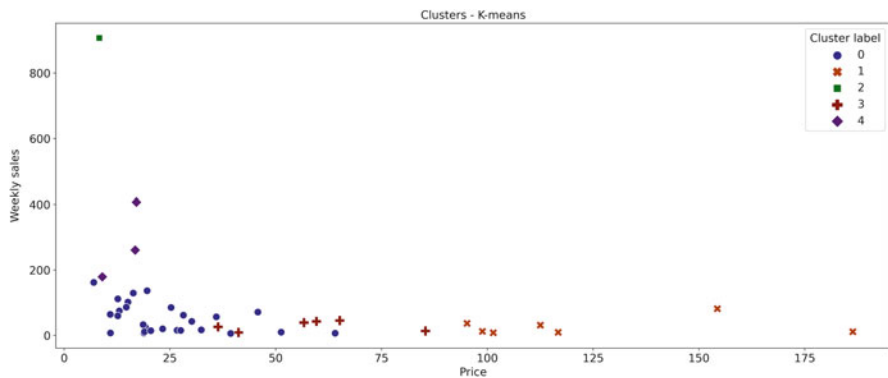


Fig. 5.2 Illustration of the clusters obtained from k-means

```

import matplotlib.pyplot as plt
import seaborn as sns

## Build dataframe
list_prices=[]
list_sales=[]
for sku in skuSet:
    list_prices.append(np.mean(X_dict[sku]['train'][:,0], axis = 0))
    list_sales.append(np.mean([ [i] for i in y_dict[sku]['train'] ]))
df_clus=pd.DataFrame()
df_clus['price']=list_prices
df_clus['weekly_sales']=list_sales
df_clus['Cluster label']=labels=kmeans.labels_
## Plot
plt.figure(figsize=(15,6))
graph = sns.scatterplot(data=df_clus, x='price', y='weekly_sales',
                        hue='Cluster label', style='Cluster label',
                        palette='dark',
                        size='Cluster label', sizes=(100, 200))
plt.title('Clusters - K-means', fontsize=15)
plt.xlabel('Price')
plt.ylabel('Weekly sales')
plt.show()

```

We find that both weekly sales and price are important features. Indeed, the five clusters on Fig. 5.2 are concentrated in distinct parts of the plot. Specifically, we can infer the following:

- Cluster 0 contains 27 SKUs with low weekly sales and low prices.
- Cluster 1 contains 7 SKUs with low weekly sales and high prices.
- Cluster 2 contains a single SKU (SKU 25) with high weekly sales (and a low price).
- Cluster 3 contains 6 SKUs with low weekly sales and intermediate prices.
- Cluster 4 contains 3 SKUs with intermediate weekly sales and low prices.

These results show once again that aggregation can be powerful, when correctly applied. They also highlight the fact that certain features may be more relevant than others in terms of aggregation, and we will discuss this aspect in greater detail in Sect. 7.2 in Chap. 7.

We note that the above results do not directly generalize to all settings and all datasets. Instead, one needs to test several alternative approaches and identify the method and the set of features that yield the best performance.

As a reminder, since we only use historical (training) data for the clustering step, there is no information leakage in this process. Our results are summarized in Table 5.2.

Table 5.2 Summary k-means clustering results

Features	Best model	OOS R^2	Computing time (sec)
Average values	k=8	0.567	0.074
Avg. values and std. dev.	k=5	0.560	0.064



Fig. 5.3 Illustration of clusters obtained with DBSCAN (top) vs. k-means (bottom). Source: DBSCAN, Github by NSHipster. Retrieved on July 19, 2021, from <https://github.com/NSHipster/DBSCAN>

As mentioned, we find that the results of the clustering approach outperform the results we obtained with the decentralized approach. This suggests that for our dataset, aggregating several SKUs together can be powerful, and ultimately improve the demand prediction accuracy. In addition, we consistently find that the prediction accuracy is the highest when using a small number of clusters (i.e., between five and eight). We next consider an alternative clustering technique called DBSCAN.

5.2 DBSCAN Clustering

In this section, we consider an alternative clustering method called density-based spatial clustering of applications with noise (DBSCAN). We refer the reader to the original paper for more details about this method.⁵

5.2.1 Description of DBSCAN Clustering

At a high level, the core concept of this method is to identify regions of data points with high density that are separated from regions with a lower density. An illustration of the difference between DBSCAN (top panel) and k-means (bottom panel) clustering methods is provided in Fig. 5.3.⁶

The main advantage of this method is to discover clusters that can have arbitrary shapes. It will also allow for singleton clusters (i.e., clusters that contain a single element), as this technique does not necessarily aggregate each SKU with other SKUs. This means that SKUs with unique characteristics will potentially be not

⁵See Ester et al. (1996).

⁶Credit: <https://github.com/NSHipster/DBSCAN>.

aggregated, whereas SKUs with similar characteristics will be aggregated with other SKUs to better leverage the training data and reduce overfitting. This is less often the case when using k-means clustering, and the visualization of clusters in the previous section shows that this was a pattern in our dataset.

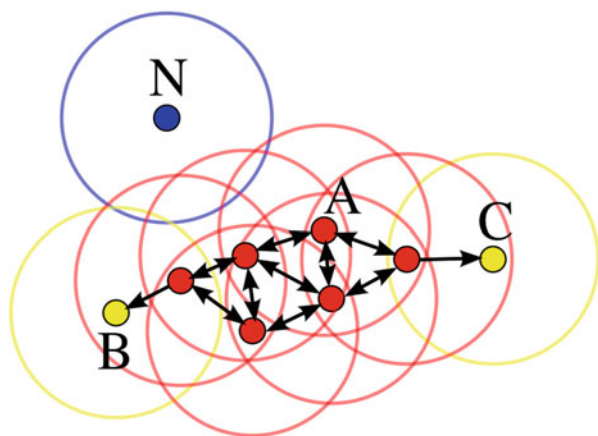
This method relies on two hyperparameters:

- `eps`—Maximum distance between two samples to be considered in the same neighborhood (i.e., what it means for observations to be considered close together).
- `min_samples`—Minimum number of neighbors required for an observation to be considered as a core point (including the observation itself). A point can be:
 - A core point if the observation has the minimum number of neighbors. If there are at least `min_samples` data points within a distance of `eps` to a given data point, this data point will be classified as a core point.
 - A border point if the observation is a neighbor to a core point (but not a core point). Border points are the points that have fewer than `min_samples` data points within a distance of `eps` but are in the neighborhood of a core point.
 - An outlier (or noisy point) if the observation is not a neighbor to any of the core points.

Fig. 5.4 provides a high-level illustration of the way DBSCAN works. This figure illustrates the DBSCAN clustering model with `min_samples=3`. In this figure, point A is a core point, points B and C are border points, and point N is a noisy point. The circles represent a neighborhood around a specific point with radius `eps` (i.e., all the points contained in the circle have a distance lower than `eps` from the center point).

The clustering process goes as follows:

Fig. 5.4 Illustration of the DBSCAN clustering
(Source: <https://en.wikipedia.org/wiki/File:DBSCAN-Illustration.svg>;
retrieved on 2021, July 12)



1. The algorithm starts with an arbitrary point that has not been visited yet and its neighborhood information is retrieved by using the `eps` parameter.
2. If the point contains `min_samples` within an `eps` neighborhood, then the point is labeled as a core point and the algorithm starts the cluster formation. Otherwise, the point is labeled as a noisy point. This point can later be found within an `eps` neighborhood of a different point and, thus, will potentially be part of a cluster.
3. If the point is found to be a core point, then the points within an `eps` neighborhood are part of the same cluster. So, all the points found within an `eps` neighborhood are added, along with their own `eps` neighborhood, if they are also core or border points.
4. The above process continues until the clusters are completely identified.
5. The process restarts with a new point that can be part of a new cluster or labeled as a noisy point. As a result, DBSCAN always converges but is not deterministic because the order of considering the different points is random.

We use the DBSCAN method by relying on the `sklearn` library.⁷

We proceed in the same way as we did for k-means. We first aggregate similar SKUs based on the average values of the price and weekly sales after scaling these variables,⁸ and then estimate a separate OLS regression for each cluster.

Remark: For illustration purposes, we use specific values of the hyperparameters (`eps = 0.05` and `min_samples = 3`) in this section. We also consider the average value of the price and the weekly sales as clustering features. In Sect. 5.2.2, we will discuss how to select the best values of the hyperparameters, and in Sect. 5.2.3, we will consider a different alternative for clustering features.

Step 1: Aggregating similar SKUs into clusters.

```
eps, ms = 0.05, 3

X_clus = np.zeros((len(skuSet), 2))
count = 0
for sku in skuSet:
    X_clus[count, :] = np.mean(
        np.concatenate((
            np.array([ [i] for i in X_dict[sku]['train'][:,0] ]),
            np.array([ [i] for i in y_dict[sku]['train'] ])),
            axis=1),
        axis=0)
    count += 1
X_clus = scaler.fit_transform(X_clus)
```

(continued)

⁷<https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html>.

⁸As before, we use the Min Max scaler.

```

from sklearn.cluster import DBSCAN
dbscan = DBSCAN(eps=eps, min_samples = ms) .fit(X_clus)

clusters_dbscan=dbscan.labels_
print(clusters_dbscan)

```

Unlike the k-means method, there is an additional step here. As previously mentioned, DBSCAN allows for singleton clusters. By default, the sklearn library assigns the value -1 to all singleton clusters (see Table 5.3 for an illustration). We thus want to re-label these -1 clusters and assign them specific values to be able to identify them in subsequent steps.

Below is the table presenting the initial cluster assigned to each SKU. As mentioned, all the noisy points are given the same label of -1 but correspond to different clusters (we thus need to re-label them).

In our dataset, when using $\text{eps} = 0.05$ and $\text{min_samples} = 3$, the DBSCAN method identifies three non-singleton clusters (each includes several SKUs). These three clusters account for 33 SKUs, and we have 11 singleton clusters (whose points can be considered as noisy points).

As discussed, we want to re-label the singleton clusters. We can do so as follows:

```

for i in range(len(clusters_dbscan)):
    if clusters_dbscan[i]==-1:
        clusters_dbscan[i]=max(clusters_dbscan)+1
print(clusters_dbscan)

```

Table 5.3 Examples of clusters obtained with DBSCAN (using $\text{eps} = 0.05$ and $\text{min_samples} = 3$)

SKU	1	2	3	4	5	6	7	8	9	10	11
label	0	1	2	0	0	0	0	-1	-1	-1	0
SKU	12	13	14	15	16	17	18	19	20	21	22
label	1	0	0	-1	0	0	2	2	0	0	0
SKU	23	24	25	26	27	28	29	30	31	32	33
label	0	0	-1	0	0	0	1	-1	-1	-1	-1
SKU	34	35	36	37	38	39	40	41	42	43	44
label	0	0	1	0	0	0	-1	0	0	-1	1

Table 5.4 Labels of the clusters after re-labelling

SKU	1	2	3	4	5	6	7	8	9	10	11
label	0	1	2	0	0	0	0	3	4	5	0
SKU	12	13	14	15	16	17	18	19	20	21	22
label	1	0	0	6	0	0	2	2	0	0	0
SKU	23	24	25	26	27	28	29	30	31	32	33
label	0	0	7	0	0	0	1	8	9	10	11
SKU	34	35	36	37	38	39	40	41	42	43	44
label	0	0	1	0	0	0	12	0	0	13	1

After performing the re-labelling, we obtain the labels presented in Table 5.4.

At this point, each cluster has its own label, and we can proceed to the next step.

Step 2: Estimating a separate OLS regression for each cluster.

```
#Loop
y_clus_pred = []
y_clus_test = []
for j in range(max(clusters_dbscan)+1):
    ##Get indices of items in cluster j
    clus_items = list(np.where(clusters_dbscan == j) [0])
    ##Initialization
    #X
    #initialization with first item of the cluster
    X_clus_j_train = X_dict[skuSet[clus_items[0]]]['train']
    X_clus_j_test = X_dict[skuSet[clus_items[0]]]['test']
    #y
    #initialization with first item of the cluster
    y_clus_j_train = list(y_dict[skuSet[clus_items[0]]]['train'])
    y_clus_j_test = list(y_dict[skuSet[clus_items[0]]]['test'])
    ##Loop
    for idx in clus_items[1:]: #Iteration over items
        sku=skuSet[idx]
        #X
        X_clus_j_train = np.concatenate((X_clus_j_train, X_dict
                                          [sku]['train']),
                                          axis = 0)
        X_clus_j_test = np.concatenate((X_clus_j_test, X_dict
                                         [sku]['test']),
                                         axis = 0)
        #y
        y_clus_j_train += list(y_dict[sku]['train'])
```

(continued)

```

        y_clus_j_test += list(y_dict[sku]['test'])
    ##Model
    model_clus_j = LinearRegression().fit(X_clus_j_train,
                                          y_clus_j_train)
    y_clus_pred += list(model_clus_j.predict(X_clus_j_test))
    y_clus_test += y_clus_j_test
#Results
oos_r2=r2_score(y_clus_test, y_clus_pred)

```

In the next section, we will fine-tune the hyperparameters (`eps` and `min_samples`) in order to obtain an improved prediction accuracy. We will use the same sub-splitting technique as discussed before to find the best performing model. As previously defined, the best model is the one that yields the highest R^2 value on the validation set. Given the large number of possible combinations of parameters, we will perform a random search (over 50 iterations) instead of an exhaustive search to reduce the computing time.

5.2.2 *Clustering using Average Price and Weekly Sales*

We proceed in the same way as we did for the k-means clustering method. We first run several iterations to identify the best model, and then test the resulting model using fresh data. We run a random search and consider the following ranges of parameters:

- `eps`: We consider values between 0.05 and 1 (with an increment of 0.05). Since the clustering features (price and weekly sales) are scaled using the Min Max scaler, it is very likely that the optimal value of `eps` lies in the above range.
- `min_sample`: We consider values between 2 and 15 (we note that 15 corresponds to roughly one-third of the total number of SKUs, but one can potentially also consider higher values).

The code for the clustering and for the linear regression estimation is presented below.

```

eps_values_ = list(np.arange(0.05,1,0.05))
min_samples_ = list(range(2,15))
params=[]
maximum_score=0
oos_r2=0

import random

```

(continued)

```

#selection of parameters to test
random.seed(5)
eps_ = random.choices(eps_values_, k=50)
ms_ = random.choices(min_samples_, k=50)

## Iterations to find optimal parameter
for i in range (50):
    print('Model number:', i+1)
    eps = eps_[i]
    ms = ms_[i]
    print(' Parameters:', [eps, ms])
    #Clustering
    X_clus = np.zeros((len(skuSet), 2))
    count = 0
    for sku in skuSet:
        X_clus[count, :] =
            np.mean( np.concatenate((
                np.array( [ [i] for i in X_dict_subsplit[sku]
                           ['train'][:,0] ] ),
                np.array( [ [i] for i in y_dict_subsplit[sku]
                           ['train'] ] ) ),
                    axis=1),
                axis = 0 )
        count += 1
    X_clus = scaler.fit_transform(X_clus)
    dbscan = DBSCAN(eps=eps, min_samples = ms) .fit(X_clus)
    clusters_dbscan=dbscan.labels_
    for i in range(len(clusters_dbscan)):
        if clusters_dbscan[i]==-1:
            clusters_dbscan[i]=max(clusters_dbscan)+1
    #Loop
    y_clus_pred = [] #y_clus_pred_sub
    y_clus_validation = [] #y_clus_test_sub
    for j in range(max(clusters_dbscan)+1):
        ##Get indices of items in cluster j
        clus_items = list(np.where(clusters_dbscan == j) [0])
        ##Initialization
        #X_sub
        X_clus_j_subtrain=X_dict_subsplit[skuSet[clus_items[0]]]['train']
        X_clus_j_validation=X_dict_subsplit[skuSet[clus_items[0]]]['test']
        #y_sub
        y_clus_j_subtrain = list(y_dict_subsplit[skuSet[clus_items
            [0]]]['train'])
        y_clus_j_validation = list(y_dict_subsplit[skuSet
            [clus_items[0]]]['test'])

```

(continued)

```

##Loop
for idx in clus_items[1:]: #Iteration over items
    sku=skuSet[idx]
    #X_sub
    X_clus_j_subtrain = np.concatenate((X_clus_j_subtrain,
                                         X_dict_subsplit[sku]
                                         ['train']),
                                         axis = 0)
    X_clus_j_validation = np.concatenate((X_clus_j_validation,
                                           X_dict_subsplit[sku]
                                           ['test']),
                                           axis = 0)

    #y_sub
    y_clus_j_subtrain += list(y_dict_subsplit[sku] ['train'])
    y_clus_j_validation += list(y_dict_subsplit[sku] ['test'])
##Model
model_clus_j_sub = LinearRegression().fit(X_clus_j_subtrain,
                                           y_clus_j_subtrain)
y_clus_pred += list(model_clus_j_sub.predict
                    (X_clus_j_validation))
y_clus_validation += y_clus_j_validation

#Comparison of results
score=r2_score(np.array(y_clus_validation), np.array
(y_clus_pred))
print(' Validation R2:', score)
if score > maximum_score:
    params = [eps,ms]
    maximum_score = score

```

We next estimate the model using the best parameter values identified above. As a reminder, we train the model on the entire training set and evaluate it on the test set.

```

eps, ms = params
#Clustering
X_clus = np.zeros((len(skuSet), 2))
count = 0
for sku in skuSet:
    X_clus[count, :] = np.mean(
        np.concatenate((
            np.array([ [i] for i in X_dict[sku] ['train'][:,0] ]),
            np.array([ [i] for i in y_dict[sku] ['train'] ])),
            axis=1),
        axis = 0)
    count += 1
X_clus = scaler.fit_transform(X_clus)
dbscan = DBSCAN(eps=eps, min_samples = ms).fit(X_clus)
clusters_dbscan=dbscan.labels_

```

(continued)

```

for i in range(len(clusters_dbscan)):
    if clusters_dbscan[i]==-1:
        clusters_dbscan[i]=max(clusters_dbscan)+1
#Loop
y_clus_pred = []
y_clus_test = []
for j in range(max(clusters_dbscan)+1):
    ##Get indices of items in cluster j
    clus_items = list(np.where(clusters_dbscan == j) [0])
    ##Initialization
    #X
    X_clus_j_train = X_dict[skuSet[clus_items[0]] ['train']]
    X_clus_j_test = X_dict[skuSet[clus_items[0]] ['test']]
    #y
    y_clus_j_train = list(y_dict[skuSet[clus_items[0]] ['train']])
    y_clus_j_test = list(y_dict[skuSet[clus_items[0]] ['test']])
    ##Loop
    for idx in clus_items[1:]: #Iteration over items
        sku=skuSet[idx]
        #X
        X_clus_j_train = np.concatenate((X_clus_j_train,
                                           X_dict[sku] ['train']),
                                           axis = 0)
        X_clus_j_test = np.concatenate((X_clus_j_test,
                                           X_dict[sku] ['test']),
                                           axis = 0)
        #y
        y_clus_j_train += list(y_dict[sku] ['train'])
        y_clus_j_test += list(y_dict[sku] ['test'])
    ##Model
    model_clus_j = LinearRegression().fit(X_clus_j_train,
                                           y_clus_j_train)
    y_clus_pred += list(model_clus_j.predict(X_clus_j_test))
    y_clus_test += y_clus_j_test
#Results
oos_r2=r2_score(y_clus_test, y_clus_pred)

#### Print Results ####

print('\nBest Model:')
print('Parameters:',params)
print('Validation R2:',maximum_score)
print('OOS R2:',oos_r2)

```

The results are given by:

```

Best Model:
Parameters: [0.2, 3]
Validation R2: 0.238
OOS R2: 0.544

```

In this case, we find that the R^2 on the test set is much higher than the value on the validation set. This suggests that the method does not overfit the data and that the better performance may be explained by the fact that we use a larger dataset to train the final model and compute the OOS R^2 . If we focus on this model, the running time is given by:

OOS R^2 : 0.544
Time to compute 0.036

Overall, the above clustering method yields a good performance along with a low running time. We next test incorporating the features' standard deviations into the clustering step.

Before doing so, we want to understand the formation of the clusters by visualizing them. An illustrative plot is presented in Fig. 5.5.

Specifically, we can see the following:

- Cluster 0 contains 41 SKUs.
- Clusters 1 and 2 contain one SKU each, both with low weekly sales and the highest prices.
- Cluster 3 contains one SKU with the highest weekly sales and a low price.

It seems relevant to compare the above model to the Centralized-OLS method. On the one hand, the Centralized-OLS method includes all the 44 SKUs and yields an OOS R^2 of 0.114. On the other hand, the above model with Cluster 0 includes 41 SKUs and the global model yields an OOS R^2 of 0.544, which is much higher. This finding highlights once again that the relevance of the aggregation is often more important than the granularity of the aggregation.

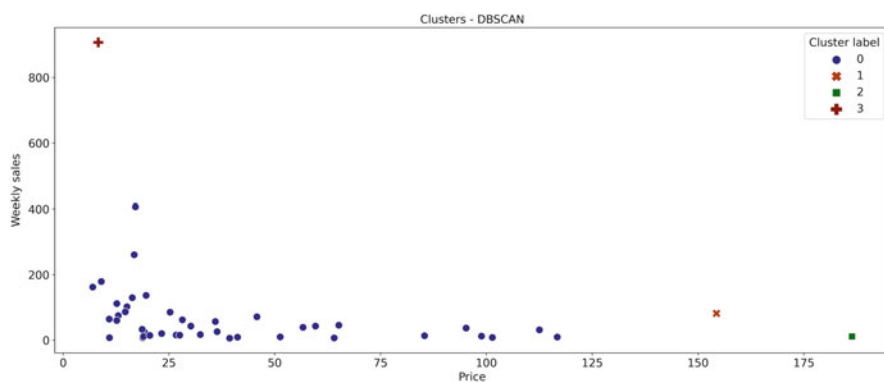


Fig. 5.5 Illustration of DBSCAN clusters with optimal parameters ($\text{eps} = 0.2$ and $\text{min_samples} = 3$)

5.2.3 Adding the Standard Deviation of the Clustering Features

As before, we consider adding the standard deviation of the features. We omit the details to avoid repeating the same process. The summary of the results is presented in Table 5.5.

In this case, we find that the DBSCAN method does not improve the prediction accuracy (compared to k-means). As mentioned before, the process of modeling is an art, and the same approach is not guaranteed to work in different scenarios. In this vein, this book aims to present a wide range of methods and a set of good practices to consider when approaching the problem of demand prediction in retail.

Based on both the OOS R^2 and the computing time, it seems that the best performing setup for DBSCAN is based on using both the average values and standard deviations of the price and weekly sales.

In conclusion, clustering techniques can be leveraged in the context of demand prediction. By grouping several SKUs together, we can augment the number of observations for similar products and ultimately obtain a higher prediction accuracy. At the same time, depending on the clustering method and the set of features used, we can find dramatically different results. One needs to carefully test different alternatives and compare them both in terms of prediction accuracy and interpretability (e.g., which SKUs are clustered together).

In this section, we considered applying an OLS method to predict the demand for each cluster. More generally, one can use the same procedure by combining the clustering step with alternative prediction methods (e.g., Lasso, Decision Tree, Random Forest).

Remark: Clustering being an unsupervised learning method, one may wonder how we can compare different clustering methods. Since our ultimate goal is to predict demand (and clustering is only an intermediate step), we directly compare the demand prediction accuracy, captured by the OOS R^2 , as opposed to comparing the clustering outcomes. Thus, we can compare different clustering techniques and hyperparameter combinations.

Table 5.5 Summary of the results for the DBSCAN clustering method

Features	Best model	OOS R^2	Computing time (sec)
Avg. price and weekly sales	eps=0.2,min_samples=3	0.544	0.036
Avg. and std. dev.	eps=0.3,min_samples=2	0.545	0.048

References

- Ester, M., Kriegel, H. P., Sander, J., Xu, X. 1996. A density-based algorithm for discovering clusters in large spatial databases with noise. In KDD (Vol. 96, No. 34, pp. 226–231).
- Hastie, T., Tibshirani, R. and Friedman, J., 2009. The elements of statistical learning: data mining, inference, and prediction. Springer Science & Business Media.

Chapter 6

Evaluation and Visualization



In this section, we summarize the results, compare the different methods, and present several simple ways to visualize and communicate the prediction results with managers.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 6/Evaluation and Visualization.ipynb
- results.csv

6.1 Summary of Results

We create a bar plot that includes all the OOS R^2 for the 15 different methods we have covered in this book. Below is the code that can be used to generate the bar plot in Fig. 6.1.

```
import pandas as pd
res = pd.read_csv('results.csv')
res

results=pd.DataFrame()
results['model']=res.columns
results['OOS R2']=res.values.tolist()[0]
results['method-type']=['Traditional','Traditional','Traditional',
                        'Traditional','Traditional','Traditional',
                        'Tree-based','Tree-based','Tree-based',
                        'Tree-based','Tree-based','Tree-based',
                        'Clustering','Clustering']

results
```

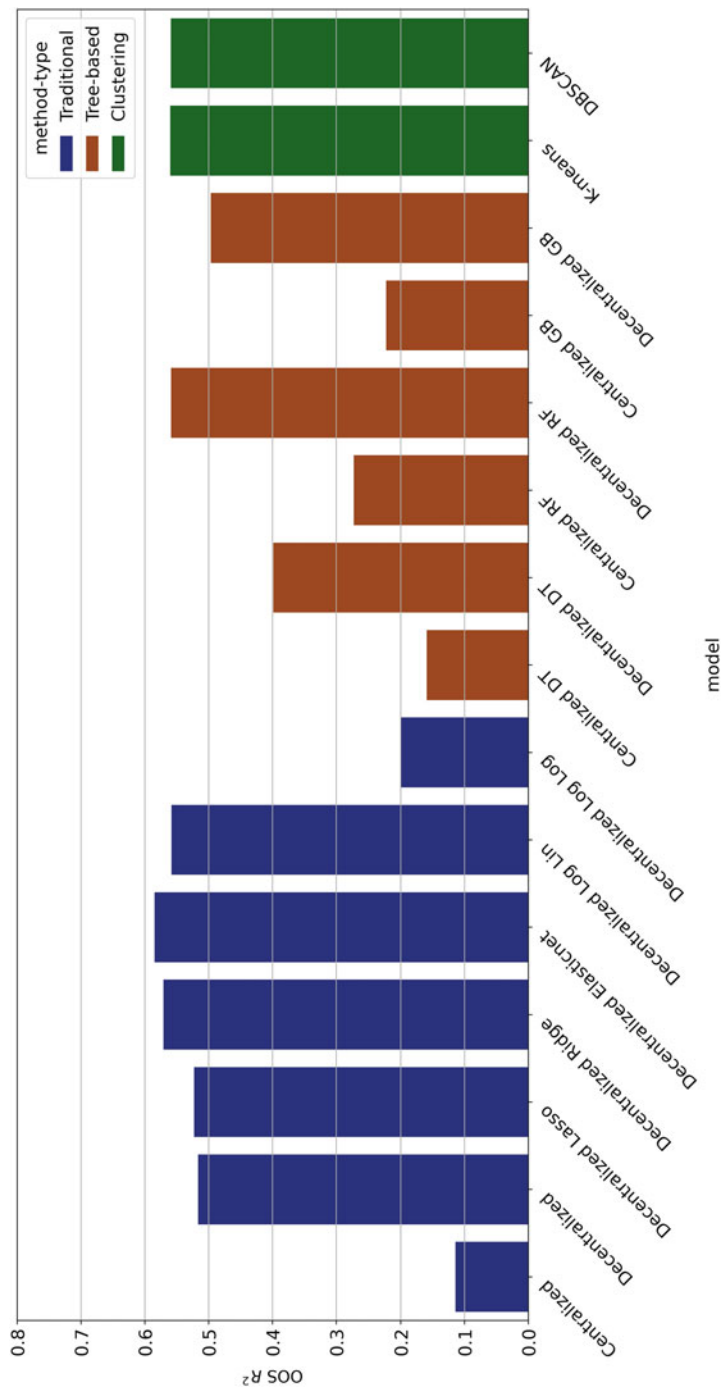


Fig. 6.1 OOS R^2 for all 15 demand prediction methods

We next select the various parameters of the figure (colors, labels, axes, etc.).

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.rcParams.update({'font.size': 15})

fig, ax = plt.subplots(figsize=(20, 8))

g = sns.barpplot(data=results, x='model', y='OOS R2', ax=ax,
hue='method-type', palette='dark', dodge=False)
ax.set_ylabel('OOS  $R^2$ ', size=14)
ax.set_xticklabels(list(res.columns),
                    rotation=45, ha='right')
ax.set_ylim([0, 0.8])
ax.yaxis.grid(True)
plt.xticks(size=15)
plt.yticks(size=15)
plt.savefig('results_plot.png', dpi=400, bbox_inches='tight')
plt.show()
```

As we can see from Fig. 6.1, the R^2 varies greatly across the different methods. The first seven bars (in blue) correspond to OLS-based methods, the next six bars (in orange) correspond to tree-based methods, and the last two bars (in green) correspond to clustering-based approaches. Several methods can reach an OOS R^2 higher than 0.55, which is a reasonable performance. Interestingly, in each type of approaches, we can find at least one method with a good performance. It is important to remember that the above R^2 values are the average across all SKUs and all weeks. It may thus be desirable to look at the prediction performance for a subset of SKUs and for a more specific time period. It is definitely possible that the prediction accuracy is much higher for a subset of specific SKUs (e.g., fast-moving items) in specific seasons. After estimating a prediction model, it can be valuable to identify the subset of SKUs for which the model provides the highest accuracy. Then, one may potentially decide to use the model only for those SKUs. Another idea that is worth mentioning is to tailor the model to the different SKUs. For example, a decentralized OLS may be working well for SKUs in one category, whereas a clustering-based Random Forest may be more appropriate for SKUs in another category.

In the next section, we compare the predicted and actual sales over time.

6.2 Prediction vs. Actual

A more convincing analysis can be to test the prediction performance by plotting the predicted values against the actual realized values over time. We start by investigating the performance of our different models for a single SKU (SKU 11). For conciseness, we only consider the OLS method.

We present below the code to plot (on the same figure) the training data (i.e., the first 68 weeks of our dataset), the testing data (i.e., the 30 remaining weeks), and the predicted values.

```
plt.rcParams.update({'font.size': 10})

plt.title('Weekly sales SKU 11')
plt.ylabel('Sales')

plt.plot(data.iloc[:68]['week'],
         y_train_primer,
         label='training',
         color=sns.color_palette(palette='colorblind')[3])
plt.plot(data.iloc[68:]['week'],
         y_test_primer,
         label='test',
         color=sns.color_palette(palette='colorblind')[2],
         linestyle='dotted')

plt.plot(data.iloc[68:]['week'],
         y_pred_primer,
         color=sns.color_palette(palette='colorblind')[1],
         label='prediction',
         linestyle='dashdot')

plt.legend(loc='upper right', fontsize='small')

plt.ylim([0, 50])
locs, labels=plt.xticks()
x_ticks = []
plt.xticks(locs[2:10], data.week[2:10], rotation=30)

plt.show()
```

The output is presented in Fig. 6.2. On the left side (the plain line), the curve corresponds to the actual sales over the training period. On the right side, the two curves represent the actual sales over the test period (dashed green line) and the predicted sales over the test period (orange dashed line). As we can see, the predicted sales somewhat capture the patterns and trends of the actual sales, albeit imperfectly. Specifically, the predicted demand captures the increases and decreases but it sometimes misses the right magnitude. Note that we obtained an OOS R^2 of 0.31 for SKU 11.

We next extend the visual comparison presented in Fig. 6.2 to the total sales across all 44 SKUs. Specifically, we select the best-performing model for each type

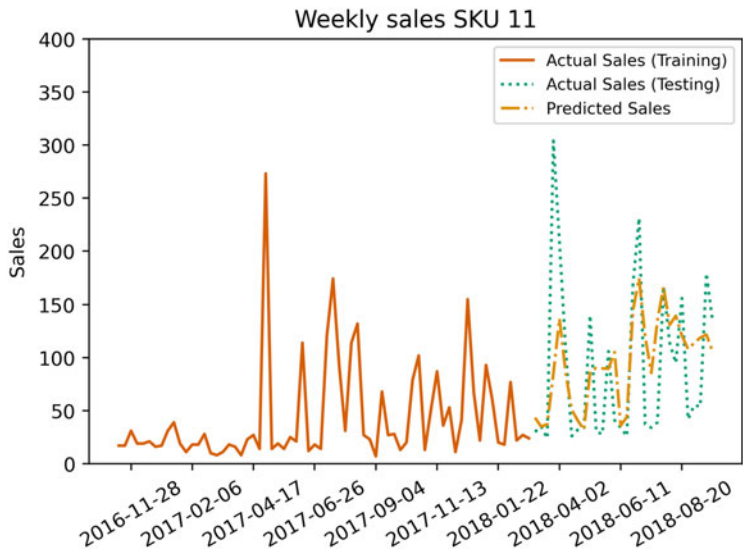


Fig. 6.2 Predicted versus actual weekly sales for SKU 11

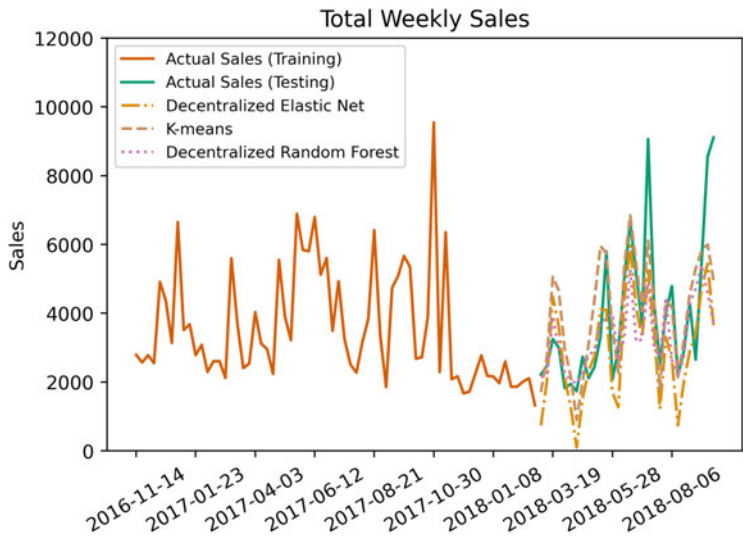


Fig. 6.3 Total weekly sales comparison

of method: Decentralized Elastic Net, k-means,¹ and Random Forest. We then sum up the prediction over all the SKUs and plot the predicted and actual weekly sales over time. The results are presented in Fig. 6.3.

¹The clustering procedure is based on the average values and standard deviations of the price and weekly sales.

```

df_test = pd.DataFrame()
df_test['actual'] = y_test
df_test['week'] = list(data.iloc[68:].week) * len(skuSet)

# run decentralized elasticnet
...
df_test['decentralized_elasticnet'] = y_pred
# run K-means
...
df_test['K-means'] = y_clus_pred

# run Decentralized Random Forest
...
df_test['decentralized-RF'] = y_pred

# sum up prediction over all SKUs for each model
sum_pred = df_test.groupby('week')['decentralized_elasticnet',
                                     'K-means',
                                     'decentralized-RF',
                                     'Actual'].sum().reset_index()

df_train = pd.DataFrame()
df_train['train'] = y_train
df_train['week'] = list(data.iloc[:68].week) * len(skuSet)

# sum up historical sales
sum_train = df_train.groupby('week')['train'].sum().reset_index()

plt.rcParams.update({'font.size': 10})

plt.title('Total Weekly Sales')
plt.ylabel('Sales')

plt.plot(sum_train.iloc[:68]['week'],
         sum_train.train,
         label='Actual Sales (Training)',
         color=sns.color_palette(palette='colorblind')[3])

plt.plot(sum_pred['week'],
         sum_pred.actual,
         label='Actual Sales (Testing)',
         color=sns.color_palette(palette='colorblind')[2])

plt.plot(sum_pred['week'],
         sum_pred.decentralized_elasticnet,
         color=sns.color_palette(palette='colorblind')[1],
         label='Decentralized Elastic Net',
         linestyle='dashdot')

```

(continued)

```

plt.plot(sum_pred['week'],
         sum_pred['K-means'],
         color=sns.color_palette(palette='colorblind')[5],
         label='K-means',
         linestyle='dashed')

plt.plot(sum_pred['week'],
         sum_pred['decentralized-RF'],
         color=sns.color_palette(palette='colorblind')[4],
         label='Decentralized Random Forest',
         linestyle=':')

plt.legend(loc='best', fontsize='small')
plt.ylim([0, 12000])
locs, labels = plt.xticks()
x_ticks = []
plt.xticks(locs[0::10], data.week[0::10], rotation=30)

plt.savefig('total_predictions_comparison_full.png', dpi=400,
          bbox_inches='tight')
plt.show()

plt.rcParams.update({'font.size': 10})

plt.title('Total Weekly Sales')
plt.ylabel('Sales')

plt.plot(sum_pred['week'],
         sum_pred.actual,
         label='Actual Sales (Testing)',
         color=sns.color_palette(palette='colorblind')[2])

plt.plot(sum_pred['week'],
         sum_pred.decentralized_elasticnet,
         color=sns.color_palette(palette='colorblind')[1],
         label='Decentralized Elastic Net',
         linestyle='dashdot')

plt.plot(sum_pred['week'],
         sum_pred['K-means'],
         color=sns.color_palette(palette='colorblind')[5],
         label='K-means',
         linestyle='dashed')

plt.plot(sum_pred['week'],
         sum_pred['decentralized-RF'],
         color=sns.color_palette(palette='colorblind')[4],

```

(continued)

```

label='Decentralized Random Forest',
linestyle=':')

plt.legend(loc='best', fontsize='small')
locs, labels=plt.xticks()
x_ticks = []
plt.xticks(locs[0::8],data.week.iloc[68:][0::8], rotation=30)

plt.show()

```

This section concludes the evaluation and visualization of the prediction results we obtained in the previous sections. As we can see, the performance varies substantially depending on the method and on the SKU under consideration. While prediction accuracy metrics (e.g., R^2) are informative, the plots displayed above are much more instrumental for communication and managerial purposes. After conducting such an exhaustive comparison, one needs to select the appropriate prediction method (it is possible to select a different method for the different SKUs, albeit it increases the implementation complexity). If the results are not satisfactory, one may decide to wait and collect additional data before retesting the various prediction methods. However, it is important to remember that a consistent perfect prediction does not exist, so that it is totally acceptable to use a model that yields an imperfect prediction accuracy (Fig. 6.4).

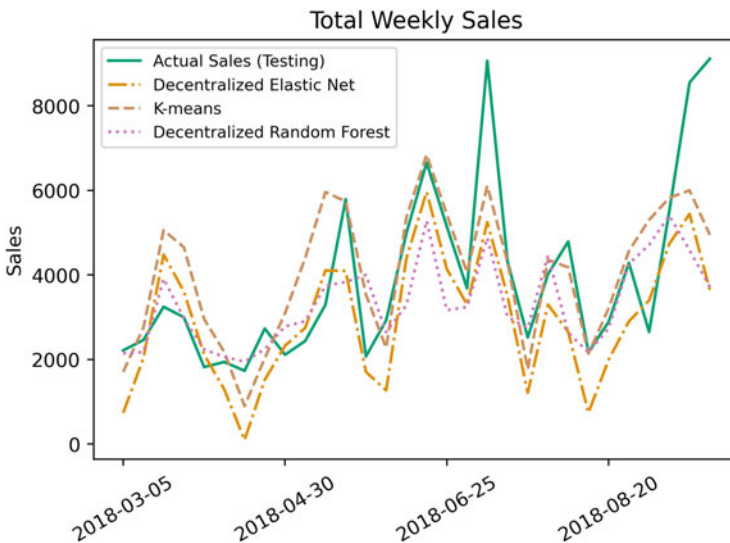


Fig. 6.4 Weekly sales comparison on the test set

6.3 Varying the Split Ratio

The objective of this section is to assess the robustness of our results with respect to the value of the split ratio. As mentioned in Section 1.3.1, the main advantage of performing a time-based training-test split is to preserve the temporal structure of the data. The major drawback is that one cannot perform a cross-validation procedure. One possible way to assess the robustness of the results is to vary the value of the split ratio. Specifically, we consider three split ratios: 65–35%, 70–30%, and 75–25%. (Remark: for the sub-split to identify the best parameter values, we use the same ratio as the train-test split.)

For conciseness, we directly import the table with all the results (see Table 6.1). The reader can replicate this table by running the scripts with the different split ratios.

```
res = pd.read_csv('robustness_test.csv')
```

We next plot the results in order to better interpret them. The first step is to transform the above wide table into a long table (i.e., we want to unpivot the table). More precisely, here is what we mean:

Table 6.1 OOS R^2 for different methods using various split ratios

Model	Method type	65–35%	70–30%	75–25%
Centralized	Traditional	0.11	0.11	0.12
Decentralized	Traditional	0.42	0.52	0.55
Decentralized Lasso	Traditional	0.44	0.52	0.56
Decentralized Ridge	Traditional	0.54	0.57	0.59
Decentralized Elastic Net	Traditional	0.56	0.58	0.61
Decentralized Log Lin	Traditional	0.47	0.56	0.57
Decentralized Log Log	Traditional	0.20	0.20	0.17
Centralized DT	Tree-based	0.37	0.16	0.21
Decentralized DT	Tree-based	0.48	0.40	0.45
Centralized RF	Tree-based	0.34	0.27	0.37
Decentralized RF	Tree-based	0.56	0.56	0.57
Centralized GB	Tree-based	0.38	0.22	0.28
Decentralized GB	Tree-based	0.54	0.50	0.46
K-means ^a	Clustering	0.56	0.56	0.15
DBSCAN ^b	Clustering	0.54	0.54	0.57

^aK-means clustering relies on using the average of the predictors and weekly sales
^bDBSCAN clustering relies on using the average of the predictors (and excluding weekly sales)

Table 6.2 Illustration of the long table format

Model	Method type	Split (%)	R^2
Centralized	Traditional	65–35	0.11
Decentralized	Traditional	65–35	0.42
Decentralized Lasso	Traditional	65–35	0.44
Decentralized Ridge	Traditional	65–35	0.54
Decentralized Elastic Net	Traditional	65–35	0.56

- In a wide table, the different variables are presented in separate columns. This format is easier to understand and interpret.
- In a long table, the different variables are presented only in one column, called the “value” column. There is also another column that contains the corresponding variable in the wide format, called the “variable” column. This results in a smaller number of columns and a larger number of rows (thus the name long table). This format is more convenient to perform operations (as one can apply the operation to only one column). For example, the `seaborn` library² (that we use for visualization) handles this format better.

To transform the data, we use the `melt` function of the `pandas` library.³ The arguments of this function that we use are as follows:

- `DataFrame` to transform (in our case, `res`).
- `id_vars`: The list of columns to use as identifier variables. These columns are not to unpivot (model and method_type).
- `value_vars`: The list of columns to unpivot (65–35%, 70–30%, and 75–25%).
- `var_name`: The name of the “variable” column (split).
- `value_name`: The name of the “value” column (OOS R^2).

```
results = pd.melt(res,
                  id_vars=['model', 'method_type'],
                  value_vars=['65-35%', '70-30%', '75-25%'],
                  var_name='split',
                  value_name='OOS R2')
```

The first five rows of the `results` `DataFrame` are presented in Table 6.2 for an illustration.

We plot the results in Fig. 6.5. The code used to create this figure is presented below:

²<https://seaborn.pydata.org/introduction.html>.

³<https://pandas.pydata.org/docs/reference/api/pandas.melt.html>.

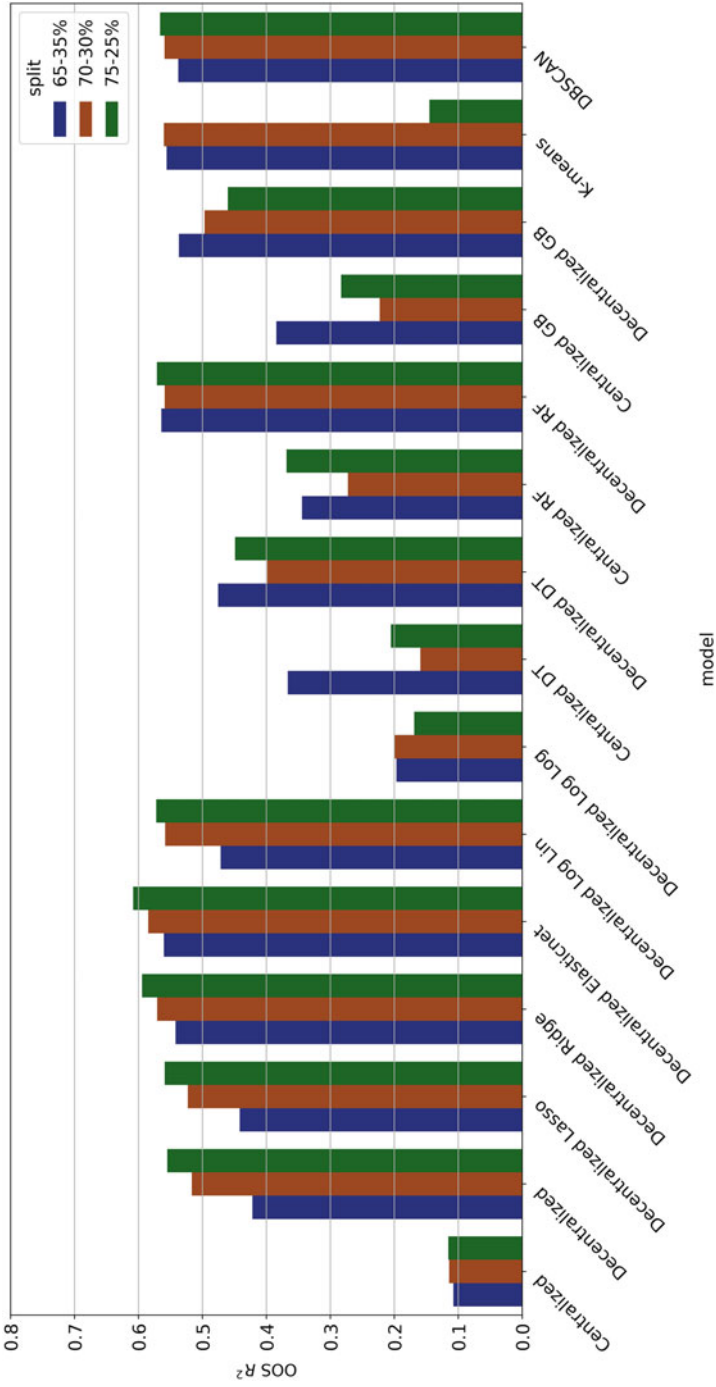


Fig. 6.5 Visualization of the robustness with respect to the split ratio

```
plt.rcParams.update({'font.size': 15})
fig, ax = plt.subplots(figsize=(20, 8))
g = sns.barplot(data=results,
                x='model',
                y='OOS R2',
                hue='split',
                palette='dark',
                ax=ax)
ax.set_ylabel('OOS  $R^2$ ', size=14)
ax.set_xticklabels(list(res.model), rotation=45, ha='right')
ax.set_ylim([0, 0.8])
ax.yaxis.grid(True)
plt.xticks(size=15)
plt.yticks(size=15)
plt.show()
```

The above test assesses the robustness of our findings in terms of the prediction accuracy and the methods in comparison. In particular, it increases our confidence on the following findings:

- It is pertinent to predict the demand at the SKU level given that the OOS R^2 of the centralized approach is lower relative to its decentralized counterpart.
- The log-transformation boosts the predictive power when applied to the price variable (but not to the target variable).
- Performing an aggregation by clustering several SKUs together is a useful approach in our context (i.e., the clustering approaches perform well for all split ratios).
- The Decision Tree does not seem to be a good approach for our problem and our dataset.
- The Gradient Boosted Tree does not yield good results (we do not have enough data to fully leverage the power of this method).

In addition, the predictive power of our models with a lower split ratio can be seen as a good indicator of what happens if one wants to predict further in the future. Indeed, an increase of the test period is equivalent to a decrease in the split ratio (assuming that the same amount of data available). For example, the decentralized OLS method yields a lower OOS R^2 when using a 65–35% ratio relative to 75–25% (0.42 versus 0.55, namely, a 23% decrease). Conversely, the Elastic Net decentralized method seems to produce more stable results. Consequently, if we are interested in predicting future demand for a longer time horizon than the 30-week period used in this book, one suggestion would be to use the Elastic Net decentralized method over the Decentralized-OLS method (at least for the focal dataset).

One pattern that draws our attention is the significant variation of the OOS R^2 with respect to the split ratio for the k -means clustering method. As we explain below, this is due to the low R^2 value on the validation set. In Table 6.3, we report the validation and OOS R^2 for the eight methods that require hyperparameters tuning.

Table 6.3 Validation and OOS R^2 using a 70–30% split

Model	Validation R^2	OOS R^2
Centralized - Decision Tree	0.570	0.159
Decentralized - Decision Tree	0.685	0.399
Centralized - Random Forest	0.457	0.272
Decentralized - Random Forest	0.573	0.559
Centralized - Gradient Boosted Tree	0.475	0.223
Decentralized - Gradient Boosted Tree	0.607	0.497
k-means clustering	0.264	0.560
DBSCAN clustering	0.263	0.545

It appears that for tree-based methods, the Validation R^2 is higher than the OOS R^2 , as expected. Indeed, the validation R^2 is the highest among all the tested combinations of parameters, whereas the OOS R^2 is the performance of the model tested on fresh data. However, for both clustering methods, we observe the opposite pattern (i.e., the OOS R^2 is significantly higher than the validation R^2). In such cases, one should be particularly cautious when drawing conclusions on the model performance.

Chapter 7

More Advanced Methods



In this section, we discuss two more advanced methods. These two methods are recent advancements in demand prediction. Of course, a very large number of other advanced methods can also be found in the academic literature and are beyond the scope of this book. We first present the Prophet method, which is a time-series demand prediction method that often works well on large-scale problems. We then discuss a method that can strike a good balance between data aggregation and demand prediction.

The files associated with this section can be found in the following website:

<https://demandpredictionbook.com>

- 7/More Advanced Methods.ipynb

7.1 The Prophet Method

7.1.1 *What is the Prophet Method?*

7.1.1.1 How it Works

Prophet is an open-sourced library available in either R or Python released by Facebook researchers in 2017.¹ It aims to help data scientists analyze and forecast time-series values. Thus, a natural application relates to demand prediction in retail.

At a high level, this method decomposes the time series into three main model components: trend, seasonality, and holidays. These components are combined in the following equation:

¹Taylor and Letham (2018).

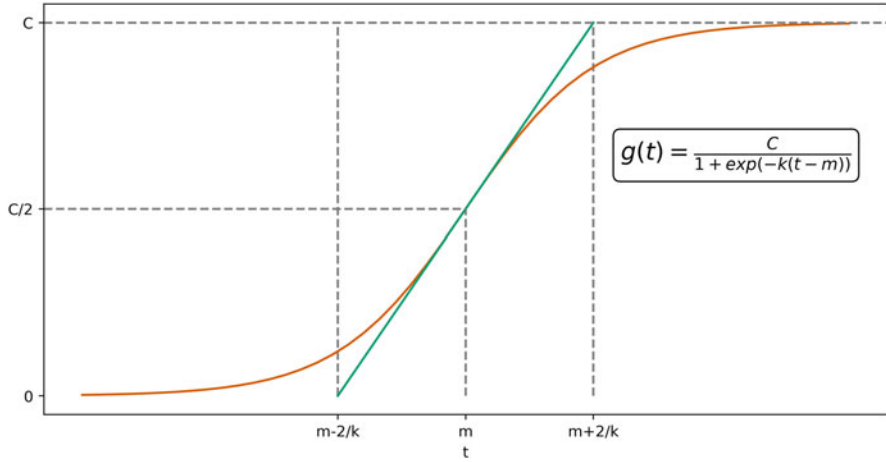


Fig. 7.1 Illustration of a sigmoid function

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t,$$

where:

- $g(t)$ is the growth (or trend) function which captures non-periodic changes,
- $s(t)$ represents periodic changes (e.g., weekly and yearly seasonality),
- $h(t)$ represents the effects of holidays which occur on potentially irregular schedules over a period of 1 or more days, and
- ε_t is the error term.

The growth is modeled using the logistic growth function, also called sigmoid. This model is widely used.² The formula is given by:

$$g(t) = \frac{C}{1 + \exp(-k(t - m))}.$$

The parameters C , m , and k are illustrated in Fig. 7.1. Specifically, C is the maximal asymptotic value of the growth function (i.e., attained when t approaches infinity), m can be interpreted as a pivot point (i.e., the value of t that yields a growth value of $C/2$), and k characterizes the growth rate around the pivot point.

We can see three zones of growth: exponential growth (for $t < m$), linear growth (for $t \approx m$), and saturated growth (for $t > m$).

The seasonality is modeled using a Fourier series, which is a common model for periodic functions. The core principle relies on modeling any given periodic (i.e., seasonal) function with a sum of sines and cosines. A simple illustration is presented in Fig. 7.2. This principle can be further extended to any periodic function, as explained by Eric W. Weisstein in a MathWorld article.³

²Hutchinson (1978).

³Weisstein, Eric W. Fourier Series. From MathWorld—A Wolfram Web Resource. <https://mathworld.wolfram.com/FourierSeries.html>.

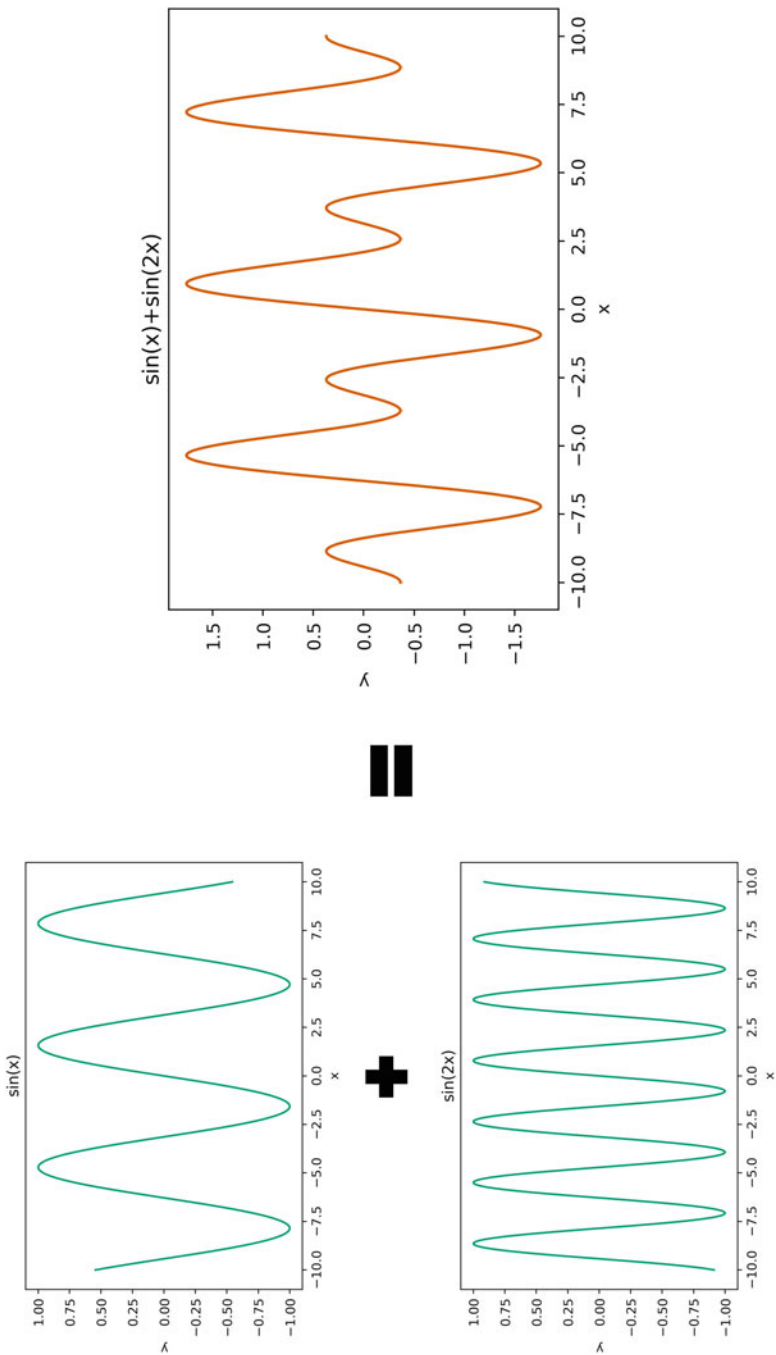


Fig. 7.2 Illustration of the function $\sin(x) + \sin(2x)$

Additional modeling details on the Prophet method can be found in the Facebook Prophet documentation.⁴

7.1.1.2 Illustration for One SKU

We next discuss how the above modeling behind the Prophet method translates to our dataset. As in Chap. 2, we focus on a specific SKU (SKU 11). The idea of this section is to illustrate how the model works and draws further insight on the data of SKU 11. The forecasting part will be presented in the following section. The implementation code is presented below.

First, we need to create a frame that includes the data of SKU 11. We can rely on the publicly available Prophet library, which is easy to use. We just need to format the input data in order to use the Prophet library. Specifically, we create a data frame with two columns:

- `ds`: The Monday of the week—we identify a week by its Monday,
- `y`: The sales of the week, which are the values we aim to predict.

```
df_11=sales[sales.sku==11]
df_train = pd.DataFrame()
df_train['ds']=list(df_11['week'])[:68]
df_train['y']=list(df_11['weekly_sales'])[:68]
```

Second, we need to specify the model parameters. In our case, we will use a simple approach in which we only model the yearly seasonality (one can also potentially model monthly or weekly seasonality). We also include the U.S. holidays (for example, Thanksgiving and Christmas are marked as holidays, and their impact on sales can be monitored). We set the `yearly_seasonality` parameter as 12, meaning that there are 12 cycles in the year (e.g., it can correspond to the 12 calendar months). Of course, one can decide to use a different value, depending on the setting. A high value for this parameter will lead to better fitting the training data (but can increase overfitting). In general, the parameters should be tuned according to both business knowledge and predictive power. We then fit our model using the training data.

```
m = Prophet(yearly_seasonality=12)
m.add_country_holidays(country_name='US')
m.fit(df_train)
```

⁴<https://facebook.github.io/prophet/>.

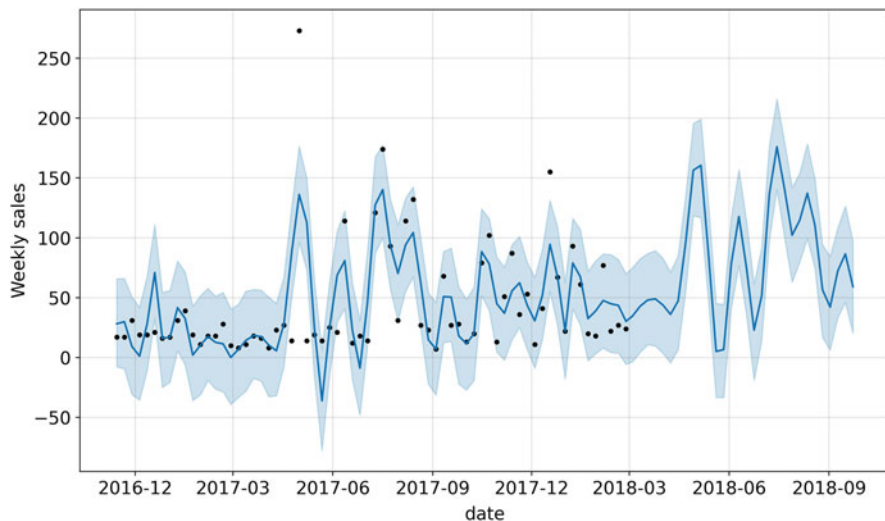


Fig. 7.3 Historical data and predictions for SKU 11 using the Prophet method

The next step is to specify the desired frequency of prediction (in our case, at the week level) and the number of periods we aim to predict (we want to predict for a period of 30 weeks). Finally, we plot and save the resulting figure. The results are presented in Fig. 7.3.

```
future = m.make_future_dataframe(periods=30, freq = 'W')
forecast = m.predict(future)
plt.rcParams.update({'font.size': 14})
fig1 = m.plot(forecast)
plt.xlabel('date')
plt.ylabel('Weekly sales')
fig1 = m.plot(forecast)
```

In Fig. 7.3, each dot represents a historical weekly sales value, the plain curve corresponds to the predicted values, and the shaded region represents the 80% confidence interval of the weekly sales prediction.

The Prophet library also includes a function to plot the different components of the decomposition ($g(t)$, $s(t)$, $h(t)$). This can potentially help us better understand the model and draw additional insights on the data. The decomposition into the three components using the data for SKU 11 is presented in Fig. 7.4.

In Fig. 7.4, we can observe a linear trend. This shows the ability of the sigmoid function to capture different types of growth (e.g., exponential, linear, saturated, flat). When analyzing the holidays, we can see that SKU 11 is not purchased during the Christmas period. When looking at the yearly seasonality, we can infer that SKU

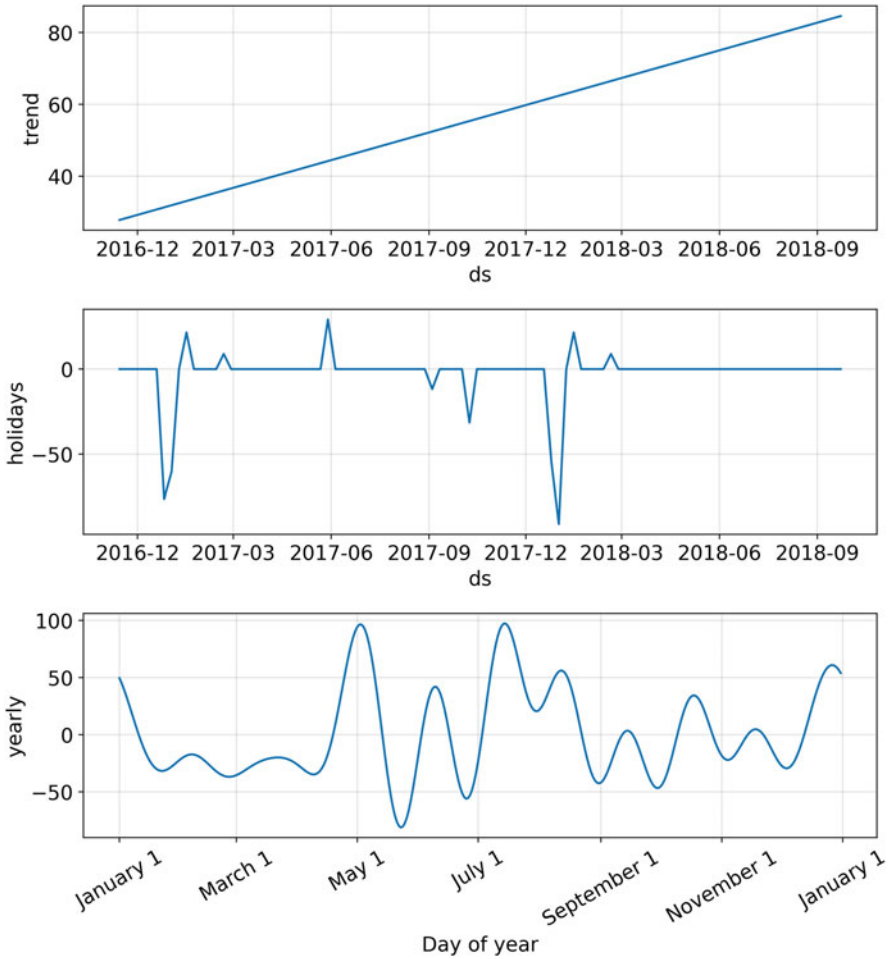


Fig. 7.4 Trend, holidays, and yearly seasonality for SKU 11

11 is more often bought in May and August and less often purchased in June and October.

The code to plot Fig. 7.4 is reported:

```
plt.rcParams.update({'font.size': 14})
fig2 = m.plot_components(forecast)
plt.xticks(rotation=30)
```

We next plot the predicted weekly demand versus the actual values in Fig. 7.5. This gives us an idea of the data variation. In this case, the predictions do not seem to

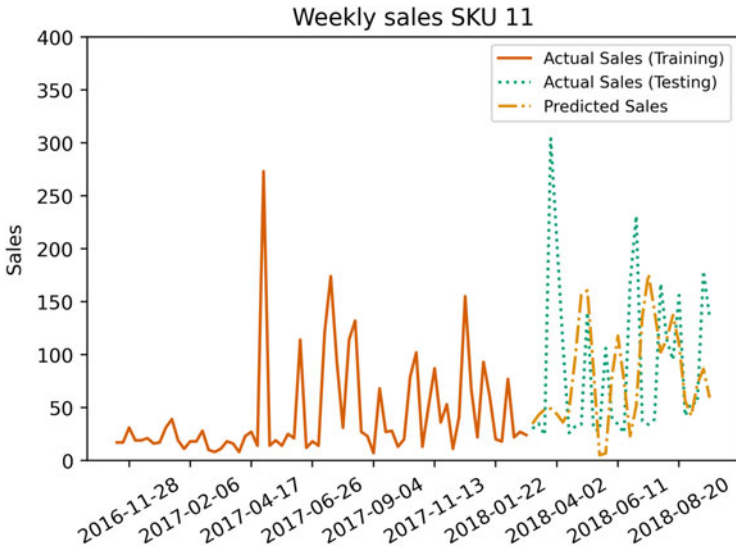


Fig. 7.5 Predicted weekly sales using Prophet versus actual values

be accurate. One potential explanation is the fact that we do not use any of the available features (e.g., price) as inputs to the predictive model. We will investigate a possible way to add features to the Prophet method in the next section.

7.1.2 Forecasting with Prophet

7.1.2.1 Univariate Time-Series

We first extend the previous approach to all 44 SKUs in our dataset. We start by considering the same model as before (i.e., without including features). We call this model univariate time-series forecasting. For each SKU, our goal is to forecast the weekly sales using the Prophet method. The code is presented below and accompanied by the notebooks available in our website.⁵

The first step is to appropriately structure the data.

⁵<https://demandpredictionbook.com>.

```

df_prophet_univariate=sales[['sku','week','weekly_sales']]

skuSet = sales.sku.unique()
skuData = {}
for i in skuSet:
    df_i = df_prophet_univariate[df_prophet_univariate.sku == i]
    skuData[i] = {'X': df_i.week.values,
                  'y': df_i.weekly_sales.values}

X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:

    X_train_i,X_test_i = np.split(skuData[i]['X'], [68])
    y_train_i,y_test_i = np.split(skuData[i]['y'], [68])

    X_dict[i] = {'train': X_train_i, 'test': X_test_i}
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}

    y_test += list(y_test_i)
    y_train += list(y_train_i)

y_train = np.array(y_train)
y_test = np.array(y_test)

```

We then run the Prophet method for each SKU.

Remark: The selection of the yearly seasonality is further detailed below.

```

#Initialization
y_pred = []
y_prophet = []

count=1

for i in skuSet:
    print('item:',count)
    count+=1
    df_train = pd.DataFrame()
    df_train['ds']=X_dict[i]['train']
    df_train['y']=y_dict[i]['train']

```

(continued)

```

size_pred=y_dict[i]['test'].shape[0]

m = Prophet(yearly_seasonality=## Input yearly seasonality ##)
m.add_country_holidays(country_name='US')
m.fit(df_train)
future = m.make_future_dataframe(periods=size_pred, freq='W')
forecast = m.predict(future)

y_pred_i=np.array(forecast['yhat'][-size_pred:])
y_pred += list(y_pred_i)

y_prophet_i=np.array(forecast['yhat'])
y_prophet += list(y_prophet_i)

```

As discussed, we would like to fine tune the parameter that captures the yearly seasonality. To do so, we rely on the predictive performance (i.e., OOS R^2). Specifically, we perform a grid search with seasonality values ranging from 0 (i.e., a flat seasonality throughout the year) to 19 (given that we only have 60 weeks of training data, we do not want to use an excessively high seasonality value to avoid overfitting). We note that we choose the yearly seasonality to be the same across all SKUs. An extension could be to tune the seasonality parameter at the SKU level (i.e., for each SKU separately).

The loop over the grid search can be implemented as follows:

```

res_r2=[]

for yearly_seas in range(20):
    print('\n Seasonality:',seas)
    #Initialization
    ...
    #Loop
    for i in skuSet:
        ...
        m = Prophet(yearly_seasonality=yearly_seas)
        ...
    #Export results
    print('R2:',round(r2_score(y_test, np.array(y_pred)),3))
    res_r2.append(r2_score(y_test, np.array(y_pred)))

```

The results are reported in Table 7.1.

As we can see, the optimal yearly seasonality value is found to be 1. This means that there is one single yearly cycle, which translates into having a single high season and a single low season. Given the modest value of the OOS R^2 , this also means that time-series forecasting is not very powerful in our setting (and our data), as the 1-yearly-cycle seasonality is not the typical pattern we saw earlier in this book (high

Table 7.1 Selection of the optimal yearly seasonality parameter

Yearly seasonality	OOS R^2
0	0.116
1	0.265
2	0.231
3	0.105
4	0.104
5	0.024
6	0.021
7	0.018
8	0.039
9	0.009
10	-0.028
11	-0.032
12	-0.123
13	-0.070
14	-0.094
15	-0.163
16	-0.158
17	-0.231
18	-0.228
19	-0.270

volatility and substantial variation throughout the year). As mentioned, it seems that higher seasonality would typically lead to overfitting.

7.1.2.2 Adding Features

To boost the prediction performance, a potential approach is to estimate some of the models presented earlier in this book, while including a Prophet-generated feature as one of the predictors. Specifically, we will replace the time-related columns (i.e., trend and month variables) by a column that contains the Prophet predicted values for each SKU and each week. The idea is to capture temporal patterns using the Prophet method combined with the other features (price, functionality, etc.).

The implementation code is presented below. We focus on the decentralized OLS model. However, one can easily apply the same approach by using other models (as discussed below).

First, we build the dataset:

```
df_prophet_multivariate=sales.copy()
df_prophet_multivariate['prophet']=y_prophet
#the order is the same (ranked by week and sku)
df_prophet_multivariate
    =df_prophet_multivariate.drop(columns={'trend',
                                           'month_2', 'month_3',
                                           'month_4', 'month_5',
                                           'month_6', 'month_7',
                                           'month_8', 'month_9',
                                           'month_10', 'month_11',
                                           'month_12'})
                                           #we remove other
                                           time-related features

df_prophet_multivariate.head()
```

Second, we structure the dataset as outlined in Sect. 3.2 in Chap. 3.

```
skuSet = list(df_prophet_multivariate.sku.unique())
skuData = {}
colnames = [i for i in df_prophet_multivariate.columns if i not in
['week', 'weekly_sales', 'sku']]
for i in skuSet:
    df_i = df_prophet_multivariate[df_prophet_multivariate.sku == i]
    skuData[i] = {'X': df_i[colnames].values,
                  'y': df_i.weekly_sales.values}

## Decentralized

X_dict = {}
y_dict = {}

y_test = []
y_train = []

for i in skuSet:
    X_train_i, X_test_i = np.split(skuData[i]['X'], [68]) #split for X
    y_train_i, y_test_i = np.split(skuData[i]['y'], [68]) #split for y
    X_dict[i] = {'train': X_train_i, 'test': X_test_i} #filling dictionary
    y_dict[i] = {'train': y_train_i, 'test': y_test_i}
    y_test += list(y_test_i)
    y_train += list(y_train_i)

## Centralized
```

(continued)

```

X_cen_train = X_dict[skuSet[0]]['train'] #initialization with item 0
X_cen_test = X_dict[skuSet[0]]['test']

for i in skuSet[1:]:
    X_cen_train = np.concatenate((X_cen_train, X_dict[i]
                                   ['train']), axis = 0)
    X_cen_test = np.concatenate((X_cen_test, X_dict[i]['test']),
                                axis = 0)

model_cen = LinearRegression(fit_intercept=False).fit
              (X_cen_train, y_train)
print('OOS R2:', r2_score(y_test, model_cen.predict(X_cen_test)))

```

Third, we perform the prediction:

```

y_pred = []
skuModels = {}

for i in skuSet:
    model_i = OLS(y_dict[i]['train'], X_dict[i]['train'],
                  hasconst = False)
    skuModels[i] = model_i.fit()
    y_pred += list(skuModels[i].predict(X_dict[i]['test']))

print('OOS R2:', round(r2_score(y_test, np.array(y_pred)), 3))

```

The result is given by:

OOS R2: 0.566

As we can see, including the features as additional predictors significantly improve the prediction accuracy relative to the univariate Prophet method. In addition, using the Prophet predicted values instead of the trend and monthly seasonality variables seems to improve the prediction accuracy (0.565 versus 0.52).

We next apply this process to the models presented earlier in this book (all the detailed scripts are provided in the notebooks).

The summary results are presented in Fig. 7.6.⁶

Overall, we see that using a Prophet-generated column instead of the traditional trend and seasonality variables yields a better performance, which also happens to be more stable across the different predictive methods. In particular, all the decentralized methods yield an OOS R^2 between 0.45 and 0.6.

⁶K-means and DBSCAN clustering rely on using the average and standard deviation of the price and weekly sales. Typically, it can be relevant to include the ‘Prophet’ column as one of the clustering features.

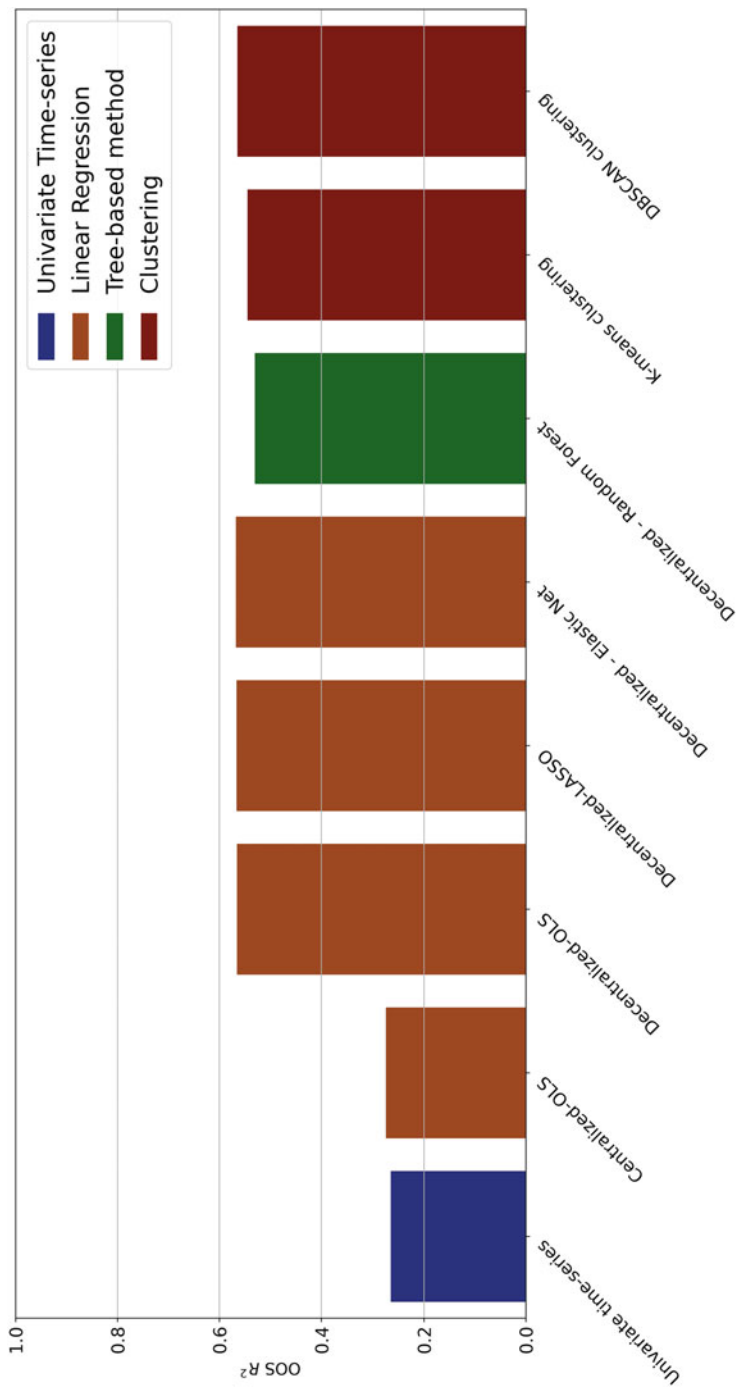


Fig. 7.6 Performance using a Prophet-generated column instead of trend and seasonality

In summary, the combination of the Prophet method (to generate a temporal feature) with our previous demand prediction models yields satisfactory results. In addition, the Prophet method helps gain a better understanding of the temporal variation of the data. Finally, this method can often lead to a better performance for datasets with a strong temporal structure, which is often the case in many retail settings. That being said, despite its simplicity, easy-to-use available package, and accurate results in specific settings and datasets, the prophet method does not always yield good results. A detailed discussion on this topic along with several concrete examples can be found in a recent blog.⁷

We conclude this section by discussing the fact that a large number of time-series methods are often used for demand prediction in several applications. In this book, we did not delve into the topic of time-series forecasting. Several comprehensive references on this topic can be found in the literature.⁸ At a high level, the methods we cover in this book and the traditional time-series methods differ in the way that the temporal dimension is accounted for in the model. In our methods, we account for temporal dimensions by using seasonality and trend effects and by potentially incorporating price-lag variables. In time-series methods, previous demand realizations (e.g., the demand value in the previous week) are used as predictive features. Time-series methods used in the context of demand prediction in retail include the following two classes of methods (as well as many other more sophisticated approaches):

- Exponential methods, such as exponential smoothing (i.e., predicting future values based on a weighted average of past values, where the weight decreases exponentially over time), as opposed to a simple moving average (where the weight remains constant). An additional common method is Holt Winters (also called triple exponential smoothing), which aims to forecast seasonal time series by incorporating both a trend and a seasonality.
- Autoregressive moving average (ARMA) models, such as pure autoregressive models (predicts the current value by using the past or lagged values), pure moving average models (predicts the current value by using the errors or residuals of the previous forecasts), and mixed ARMA models.

The detailed implementation of these methods is beyond the scope of this book (note that several implementations and libraries can be found in open-source packages). We note, however, that the above methods often use the demand values observed in recent periods (e.g., in the past week), so that it may not be applied for situations where one aims to predict the demand in several periods ahead.

⁷<https://www.microprediction.com/blog/prophet>.

⁸See, e.g., Granger and Newbold (2014), Montgomery et al. (2015).

7.2 Data Aggregation and Demand Prediction

The content presented in this section is more advanced and describes a method that can adaptively balance data aggregation and separation in a data-driven fashion. Note that this section requires a more advanced level of coding relative to other sections as well as additional knowledge in statistics. As discussed before, each demand prediction model can be applied using different ways of aggregating the data. At the two extremes, we have the centralized approach (i.e., estimating a single joint model for all SKUs) and the decentralized approach (i.e., estimating a different model for each SKU). At the end of Chap. 4, we presented several intermediate methods that evaluate certain features at the aggregate level and other features at the SKU level. By nature, certain features may have the same impact on the sales of all the SKUs, whereas other features may have a different impact on each SKU. It is also possible that some features affect a group (or cluster) of SKUs in the same way. In this section, we introduce a method that allows us to systematically identify the right aggregation level for each feature: aggregate level, cluster level, or SKU level. This method, called data aggregation with clustering (DAC), works as an additional layer on top of a demand prediction model. To illustrate this method, we will consider a simple linear regression, but it can also be applied in conjunction with other models.

The main question is which estimated coefficients of the demand model should be estimated at the aggregate level (i.e., jointly for all SKUs), at the SKU level, or at the cluster level. Furthermore, for the coefficients that need to be estimated at the cluster level, what is the right clustering structure (i.e., which SKUs should be grouped together). The DAC method aims to provide a data-driven answer to these two questions. This method was recently developed by some of the authors of this book. For more details on the specifics of the DAC method and its theoretical foundations, we refer the reader to the article.⁹

7.2.1 Presentation of the DAC Method

Intuitively, the DAC method relies on studying the results of the decentralized approach and comparing the estimated coefficients across the different SKUs. If the estimated coefficients for a specific feature (e.g., price) for all SKUs have a similar value, then one can assume that this feature has the same effect on all SKUs, and hence this feature should be estimated at the aggregate level. In this case, we will only need to estimate one coefficient for this feature. We call this type of feature aggregate-level features. If, however, the estimated coefficients for a specific feature are substantially different across SKUs, then we can conclude that this specific feature should be estimated at the decentralized level. In this case, we will need to estimate one coefficient for each SKU. We call this type of feature SKU-level

⁹Cohen et al. (2019).

Algorithm 1 DAC

Input: Estimated coefficient $\hat{b}_{i,l}$ and standard error ($SE_{i,l}$) for each item i and feature l .

For each feature $l \in D$,

- 1: Fix an item 1. For all other items $i \neq 1$, compute the p -value based on the null hypothesis $H_{1,i}^0$ that $b_{1,l} = b_{i,l}$ (i.e., the coefficients of feature l are the same for item 1 and item i).
- 2: If $H_{1,i}^0$ is not rejected for all items, then feature l should be estimated at the aggregated level.
- 3: If $H_{1,i}^0$ is rejected for some items and validated for others, then feature l should be estimated at the cluster level.
We then run a one-dimensional k -means on $\{\hat{b}_{i,l} : 1 \leq i \leq n\}$ and obtain the resulting clusters $\hat{C}_1, \hat{C}_2, \dots, \hat{C}_k$.
- 4: If $H_{1,i}^0$ is rejected for all items, then feature l should be estimated at the SKU level.
- 5: Obtain the aggregation level for each feature: $\hat{D}_n$, $\hat{D}_s$, and $\hat{D}_c$.
- 6: Fit an aggregated model to obtain the coefficients $\hat{\beta}$.

Output: (a) Aggregation levels: $(\hat{D}_n, \hat{D}_s, \hat{D}_c)$, (b) Cluster structure: $(\hat{C}_1, \hat{C}_2, \dots, \hat{C}_k)$, (c) Feature coefficients: $\hat{\beta}$.

Fig. 7.7 Pseudocode of the DAC method (borrowed from the article mentioned in footnote 9)

features. Finally, one could identify groups of SKUs for which the estimated coefficients of a specific feature are similar; these features are called cluster-level features. To make the comparison of the estimated coefficients, one can rely on conducting a statistical t -test and inspecting the resulting p -value.

The pseudocode of the DAC method can be found in Fig. 7.7, which is borrowed from the article mentioned above.

We provided in one of the notebooks (7/Extensions.ipynb) a complete implementation of the DAC method under the function `DAC`. After running the decentralized approach on the dataset, we obtain one linear regression for each SKU along with its estimated coefficients for each feature. Then, the DAC function will perform the following three tasks:

- For each feature, the function performs statistical tests on the estimated coefficients of the decentralized linear regressions to ultimately compute a similarity ratio (i.e., the proportion of SKUs that have a statistically close estimated coefficient value).
- Based on the similarity ratio, the function splits the features into three categories: aggregate, cluster, and SKU.
- Then, the function creates SKU- and cluster-fixed effects in a similar way to what we did at the end of Chap. 4.

Ultimately, the above function will return a modified version of the original dataset, with one column for each aggregate-level feature, z columns for each cluster-level feature (assuming that z is the number of clusters), and 44 columns for each SKU-level feature. One can then simply apply a linear regression (or any other model) using this dataset.

In Fig. 7.8, we illustrate the structure of the dataset used in three different methods: centralized approach with price fixed effects, decentralized approach with aggregated seasonality, and the DAC method.

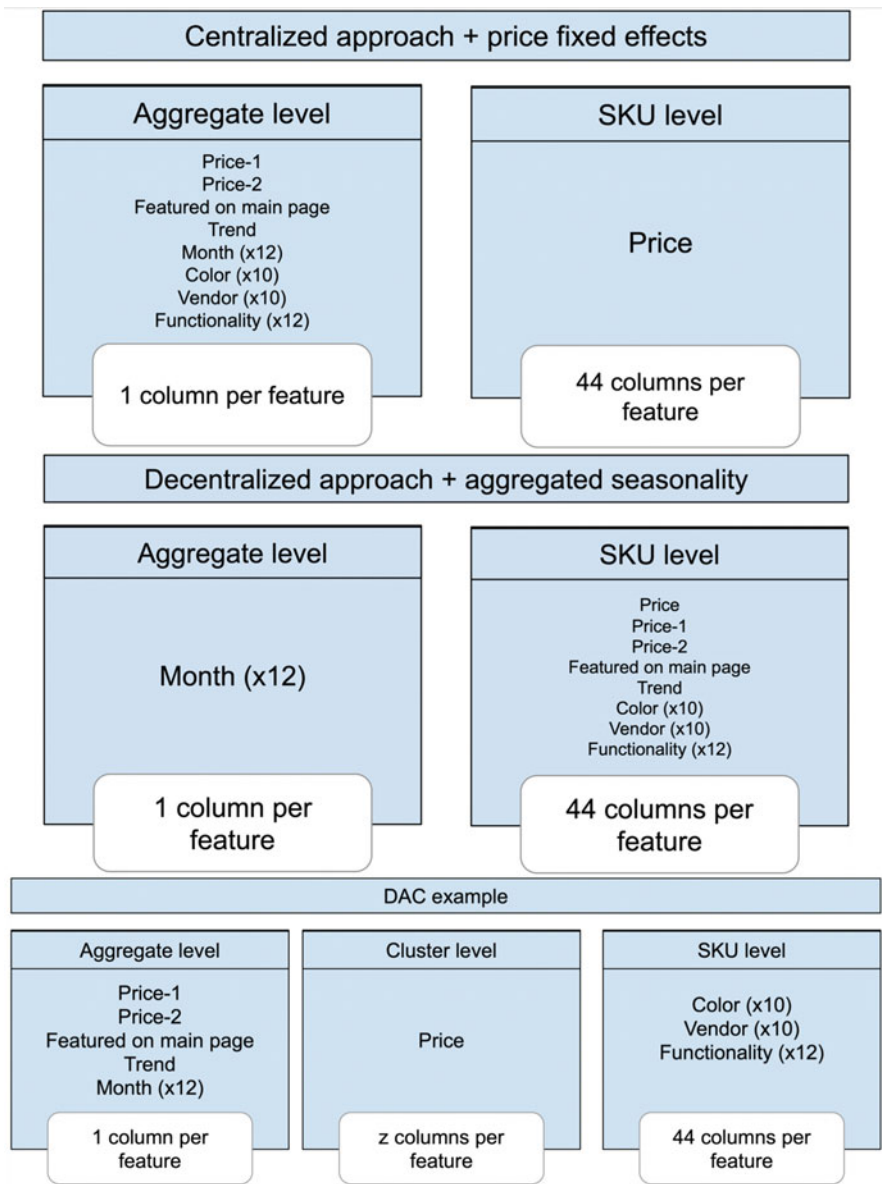


Fig. 7.8 Structure comparison of datasets for the centralized approach with price fixed effects, decentralized approach with aggregated seasonality, and the DAC method

7.2.2 *Fine-Tuning the Hyperparameters*

```
DAC(theta = 0.01, RU = 0.9, RL = 0.1, num_clusters = 9,  
     print_structure = False)
```

As we can see, the function DAC admits the four following design parameters:

- `theta`: It corresponds to the p-value cut-off for statistical significance when comparing the values of the estimated coefficients (it is common to use 0.01, 0.05, or 0.1).
- `num_clusters`: For each feature identified at the cluster level, we perform a k-means clustering by default with $K = \text{num_clusters}$.
- `RU` and `RL`: These parameters represent the thresholds for the ratio of non-rejected hypotheses. In other words, the parameters `RU` and `RL` help us decide whether each feature should be estimated at the aggregate level, SKU level, or cluster level. Specifically, for each feature, we let R be the ratio of SKUs that have a statistically close estimated coefficient value. We then do the following:
 - If $R > RU$, it means that a significant portion of the SKUs have a similar estimated coefficient, and hence we decide to estimate this feature at the aggregate level.
 - If $R < RL$, it means that only a small portion of the SKUs has a similar estimated coefficient, and hence we decide to estimate this feature at the SKU level.
 - If $RL < R < RU$, we will label this feature as a cluster-level feature.

Once a feature is identified to be at the cluster level, one can use the k-means method (or any other clustering technique) to identify the clusters that will each have a different estimated coefficient in the final model. As discussed, the number of clusters is one of the hyperparameters of the model. Note that the k-means clustering method is embedded in the DAC function.

One possible way to fine-tune the four hyperparameters mentioned above is by using simple nested loops as follows. For each combination of hyperparameters, we generate a dataset using the DAC function, fit a linear regression, evaluate its performance, and compare it with the current best value. If we obtain a performance improvement, we update the selected hyperparameters (we continue this process until we have covered the entire grid of parameter values). We highlight that the code below may take up several minutes to run (depending on the computing environment). One can potentially consider a reduced range of possible hyperparameters in order to reduce the running time.

```

i=0
params=[]
maximum1=0

for upp in [0.7,0.8,0.9]:
    for low in [0.1,0.2,0.3]:
        for z in [6,7,8]:
            for theta in [0.01,0.05]:
                i+=1
                X_alg_train, X_alg_test, colnames_alg = DAC(upp = upp, low = low,
num_clusters = z, theta=theta)
                print('Fitting model',i, '/54')
                model_0 = LinearRegression().fit(X_alg_train, y_train)
                print(' Parameters:', upp, low, z, theta)
                score=r2_score(y_test, model_0.predict(X_alg_test))
                print(' R2 score:', str(score)[:5])
                if score>maximum1:
                    params = [upp,low,z,theta]
                    maximum1 = score

print('\nBest Model:')
print(' Parameters:',params)
print(' OOS R2:',str(maximum1)[:5])

```

An alternative way to fine-tune the values of the hyperparameters is by using a cross-validation procedure at the expense of potentially increasing the running time (omitted for conciseness).

We next output the best values of the hyperparameters from the above procedure, along with the performance of the associated model:

```

Best Model:
Parameters: [0.7, 0.2, 8, 0.01]
OOS R2: 0.577

```

As we can see, the DAC method not only provides a good out-of-sample predictive performance (e.g., relative to the decentralized approach, which led to a 0.52 OOS R^2) but it also allows us to reach a deeper understanding on the structure of the features. This knowledge can ultimately be leveraged for operational and strategic decisions as we will discuss next.

7.2.3 *Interpreting the DAC Results*

From the above model, we also obtain for each feature the similarity ratio R and the resulting aggregation level based on the values of RU and RL . We report these findings in Table 7.2 for five of our features.

Table 7.2 Similarity ratio for price, lagged prices, featured on main page, and mobile phone accessories functionality

	Price	Price-1	Price-2	Featured on main page	06. Mobile phone accessories functionality
R	0.56	0.93	0.91	0.95	0.09
Comparison with RU and RL	$RL < R < RU$	$R > RU = 0.8$	$R > RU = 0.8$	$R > RU = 0.8$	$R < RL = 0.2$
Level	Cluster	Aggregate	Aggregate	Aggregate	SKU

Table 7.3 List of SKUs per cluster for the price feature

Price	Cluster 1	Cluster 2	Cluster 3	Cluster 4	Cluster 5	Cluster 6	Cluster 7	Cluster 8
SKUs	15	24	32	33	14, 29	34	35	All other SKUs

As we can see, the DAC method identifies the level of aggregation for each feature, which helps us build intuition on the relationship between the different SKUs in terms of demand prediction dynamics. In our dataset, we find that the price feature should be estimated at the cluster level (eight clusters), whereas the lag prices (for both 1 and 2 weeks) seem to have a uniform impact across all SKUs. This suggests that the impact of past promotions is homogeneous and not SKU dependent.

To further examine the output of the DAC method, we next consider the price feature and its resulting clusters (Table 7.3):

As discussed, the price feature is identified to be estimated at the cluster level. More precisely, the algorithm identifies eight different clusters for this feature. While SKUs 15, 24, 32, 33, 34, and 35 have their own price coefficient, the other SKUs share their price coefficients with different SKUs. For example, we find that SKUs 14 and 29 have the same estimated coefficient for the price feature.

The power of the DAC method partially comes from its ability to identify a group of SKUs with similar dynamics with respect to a specific feature (e.g., price) and estimate the coefficient jointly for the group of SKUs. Consider, for example, Cluster 5 for the price feature, which contains two SKUs (14 and 29). In this case, the DAC method will estimate the price coefficient for Cluster 5 by using a training set that includes the data from two SKUs (as opposed to training two linear regressions, each with a smaller amount of data).

In our case, the DAC method seems to outperform several other methods we considered. In addition, one can apply this machinery to non-linear models (e.g., Random Forest). Most of the time, one does not know upfront the correct aggregation level for each feature in the demand model. Instead of testing and comparing all possible combinations, the DAC method provides a systematic way to identify the

best aggregation level for each feature. It can thus save time, while also providing useful knowledge on the relationships of the different SKUs.

References

- Cohen, Maxime C, Zhang, Renyu, Jiao, Kevin, 2019 Data Aggregation and Demand Prediction. Available at SSRN 3411653.
- Granger, C.W.J., Newbold, P., 2014. Forecasting economic time series. Academic Press.
- Hutchinson, G. E. (1978), An introduction to population ecology.
- Montgomery, D.C., Jennings, C.L., Kulahci, M., 2015. Introduction to time series analysis and forecasting. John Wiley & Sons.
- Taylor, S. J., Letham, B. 2018. Forecasting at scale. *The American Statistician*, 72(1), 37–45.

Chapter 8

Conclusion and Advanced Topics



The intent of this book was to cover the entire demand prediction process for retailers. We discussed all the steps involved, starting from collecting, pre-processing, and understanding the data all the way to evaluating and visualizing the prediction results. In the process, we presented several methods and approaches for demand prediction that are commonly used in retail settings. In each step, we included the relevant code and implementation details to demystify how historical data can be leveraged to predict future demand. We also thoroughly discussed a number of important practical considerations in data-driven environments. We are confident that after reading this book, readers will be prepared to apply this knowledge and predict demand in their retail setting of interest. More precisely, the content of this book can be leveraged by retailers who have access to historical sales data and are interested in predicting the future demand for their products.

The tools and methods covered in this book can be applied to a multitude of retail settings (both online and brick-and-mortar). Specifically, most of the concepts are agnostic to the type of retailer and can be applied to most verticals including fashion, electronics, groceries, and furniture, just to name a few. Of course, depending on the business setting and on the type of data collected, several tweaks will be needed. However, the content presented in this book can serve as a starting point to master the basic ideas and to learn how to implement common methods for demand prediction.

At the same time, the content of this book is not meant to be exhaustive and does not cover many more advanced topics in the context of demand prediction. We next briefly discuss several topics beyond the scope of this book and refer readers to relevant references.

Deep learning methods. Given the recent success of deep learning and neural networks in a myriad of applications, it is unsurprising that such methods have also been applied to demand prediction in retail. While deep-learning methods can yield excellent prediction accuracy in certain settings, they may also perform poorly in other settings. Typically, the performance will depend on the amount of data available. For large retailers with enormous velocity and volumes of transactions,

it may be worth considering deep-learning methods (e.g., artificial neural networks, transformers). For more modest settings (like the dataset accompanying this book), however, such methods will often not perform well due to data scarcity. Finally, these methods often suffer from a lack of interpretability relative to the methods we have covered in this book.¹

Transfer learning. Recent advances in machine learning include the concepts of transfer learning and domain adaptation, where a model developed for a specific task is leveraged as the starting point to a model for a different task. In the context of retail, several use cases where this type of machinery can be used come to mind. The first example is to use transfer learning across several stores. For instance, when opening a new store for which data availability is limited, the data from different stores can be used as a starting point. A second example is to use transfer learning between the online and offline channels. Overall, such techniques, albeit more advanced, can be very useful in retail contexts.²

New types of data. As discussed in this book, an important part of demand prediction is to identify (or construct) the right set of predictive features. We presented several basic features, such as price and seasonality. Of course, a large number of additional features can be leveraged (e.g., promotion information, locations of SKUs on the shelves). In addition to internal data features, one can also exploit external data sources. Examples include Google Trends, social media data, and key words included in news articles. For instance, grocery retailers can monitor social media activities and search trends for keywords related to recipes. Then, these features can be used for demand prediction of the products related to the trending recipes. Several data providers offer assistance in collecting this type of external data sources. One concrete example is to use social media data (e.g., Twitter).³ Another example is to use consumer reviews or recommendations.⁴ A last example is to use competitor data (e.g., by monitoring competitors' websites or by purchasing aggregated indices from data providers). Of course, depending on the setting, incorporating external data features does not necessarily help enhance the demand prediction accuracy. In this context, it is important to highlight the issue of data leakage, that is, the situation when information from outside the training dataset is used to estimate a predictive model.⁵ When using the price as a predictive feature, one needs to ensure that the price values will be available in advance for predicting future demand. Similarly, when using external features, it is crucial to check that these features will be known at the time that the model is trained. For example, one cannot use the real-time traffic (in-store or on the website) to predict the current demand.

¹For a general introduction to deep learning, see Goodfellow et al. (2016). For applying deep learning to demand prediction in retail, see, e.g., Husna et al. (2021).

²See Pan and Yang (2009).

³See, e.g., Gaikar and Marakarkandy (2015).

⁴Chen et al. (2004).

⁵<https://machinelearningmastery.com/data-leakage-machine-learning/>.

Data censoring. Another advanced topic in the context of demand prediction is data censoring. For settings with limited stock levels, the observed sales are different from demand. Assume for example that the retailer has a stock of 50 available t-shirts for sale. If we observe that the sales are equal to 50, it is naturally possible that the demand was higher than 50 (i.e., some customers were interested in buying the item but could not be served due to the limited availability). In such a case, using blindly the sales data to estimate demand will introduce a bias in the predicted values. Several methods have been developed to overcome this issue. In many settings, however, stock-out events are not occurring frequently, so that data censoring is not a critical issue. For more details on this topic, we refer the reader to the recent academic literature.⁶ This stream of work is related to discrete choice models, which aim to model how customers are choosing among several alternatives of products. At a high level, a discrete choice model predicts the likelihood of customers purchasing a specific product (from a category of related products) based on the various products' features.⁷

New products. A legitimate question is how one can predict demand for new products. In the absence of data, all the methods and approaches discussed in this book become futile. Unfortunately, there is no magic answer to this question. Instead, several approaches were developed, and their performance highly depends on the context. This problem is of greater importance for retailers who are constantly refreshing their assortment (e.g., fast-fashion retailers).⁸ We next discuss two potential simple approaches to this problem (several more sophisticated methods exist). A first approach can be to identify other products, which are similar to the new product, and use their data as a starting point (e.g., the same product in a different color, the previous generation of a smartphone). A second approach, somewhat related, is to rely on clustering methods. One can cluster the different products based on their attributes (e.g., color, vendor, size, price). Then, new products will be assigned to a particular cluster and the demand prediction model of the cluster can be used as a starting point. Of course, as more and more data are collected for the new product, these data can be strategically incorporated (e.g., via transfer learning).

Prediction intervals. All the methods presented in this book consider that the prediction values are point estimates (e.g., we predict an average demand value for each SKU and week). In practice, there is naturally uncertainty around these point estimates. Thus, it is often desirable to characterize this uncertainty by computing prediction intervals instead of relying on a single-point estimate. More precisely, a prediction interval is an estimate of the interval in which a future observation will fall, with a certain probability. For specific methods, such as OLS, one can formally characterize the resulting prediction intervals.⁹

⁶See, e.g., Kök and Fisher (2007), Vulcano et al. (2012), and Subramanian and Harsha (2020).

⁷For more details, see the seminar work Ben-Akiva and Lerman (2018).

⁸See, e.g., Khan (2002), Hu et al. (2019), and Baardman et al. (2017).

⁹http://web.vu.lt/mif/a.buteikis/wp-content/uploads/PE_Book/3-7-UnivarPredict.html.

Endogeneity. The last topic worth mentioning is endogeneity. Although this problem is more related to econometrics (and causal inference) than prediction, it is often discussed in the context of demand modeling. Endogeneity refers to situations where an explanatory variable is correlated with the (unobserved) error term. In this case, when using an OLS regression, the estimate of the coefficient will be biased. For a good illustrative example as well as more details on this topic, we refer the reader to this post.¹⁰ An extensive body of work in econometrics has focused on developing solutions to overcome this problem. A common strategy is to identify and use instrumental variables along with a two-stage least squares regression. Again, this issue is more important for casual interpretations than for prediction, so that most predictive demand models often ignore endogeneity concerns.¹¹

Now that we know how to predict future demand using historical data, what is the next step? The answer is simple: shifting from predictive analytics to prescriptive analytics. Ultimately, retailers are interested in making the best possible operational decisions informed by historical data. Being able to accurately predict demand bears several practical implications. First, retailers can use the predicted demand values to decide inventory replenishment strategies. For example, if they anticipate a large demand boost due to a promotional event, they can strategically adapt their inventory levels. Similarly, if a retailer has multiple stores, demand prediction for each store can help allocate inventory levels (from a centralized warehouse) to the different stores. Second, demand prediction models can be used as an input to optimize future prices and promotions.¹² Third, retailers can also use demand prediction to guide assortment decisions (i.e., which products to offer in their stores) and planogram arrangements (i.e., where to position products on the shelves). Fourth, demand prediction models can sometimes be used to simulate the effect of different potential “what-if” strategies (e.g., upcoming promotional campaigns). Regardless of the specific application, being able to accurately predict demand can provide a significant competitive advantage to retailers and, ultimately, increase their bottom line. Finally, while the focus of this book was on prediction, sometimes the parameter estimates themselves are also useful from a business perspective. For example, some demand prediction models can be used to compute the price elasticity of demand, which measures how sensitive the demand level is with respect to the price. Such knowledge is instrumental to better understand customers’ behavior and ultimately design the right pricing and promotion strategies. All in all, demand prediction can be seen as one of the first building blocks of a data-driven business culture and opens several avenues for improving operational and tactical decisions.

¹⁰<https://towardsdatascience.com/endogeneity-the-reason-why-we-should-know-about-data-part-i-80ec33df66ae>.

¹¹ See, e.g., Angrist and Pischke (2008), Angrist et al. (2000).

¹² See, e.g., Cohen et al. (2017), Cohen et al. (2021), and Ferreira et al. (2016).

References

- Angrist, J. D., Graddy, K., & Imbens, G. W. 2000. The interpretation of instrumental variables estimators in simultaneous equations models with an application to the demand for fish, *The Review of Economic Studies*, 67(3), 499–527.
- Angrist, J. D., Pischke, J. S. 2008. *Mostly harmless econometrics: An empiricist's companion*. Princeton University Press.
- Baardman, L., Levin, I., Perakis, G. and Singhvi, D., 2017. Leveraging comparables for new product sales forecasting. Available at SSRN 3086237.
- Ben-Akiva, M. and Lerman, S.R., 2018. *Discrete choice analysis: theory and application to travel demand*. Transportation Studies.
- Chen, P.Y., Wu, S.Y., Yoon, J., 2004. The impact of online recommendations and consumer feedback on sales. *ICIS 2004 Proceedings*, p.58.
- Cohen, M.C., Kalas, J.J. and Perakis, G., 2021. Promotion Optimization for Multiple Items in Supermarkets. *Management Science*, 67(4): 2340–2364.
- Cohen, M.C., Leung, N.H.Z., Panchamgam, K., Perakis, G. and Smith, A., 2017. The impact of linear optimization on promotion planning. *Operations Research*, 65(2): 446–468.
- Ferreira, K.J., Lee, B.H.A. and Simchi-Levi, D., 2016. Analytics for an online retailer: Demand forecasting and price optimization. *Manufacturing & Service Operations Management*, 18(1), pp.69–88.
- Gaikar, D. and Marakarkandy, B., 2015. Product sales prediction based on sentiment analysis using twitter data. *International Journal of Computer Science and Information Technologies*, 6(3), pp.2303–2313.
- Goodfellow, I., Bengio, Y., Courville, A., Bengio, Y. 2016. *Deep learning* (Vol. 1, No. 2). Cambridge: MIT press, Cambridge: MIT press.
- Hu, K., Acimovic, J., Erize, F., Thomas, D.J. and Van Mieghem, J.A., 2019. Forecasting new product life cycle curves: Practical approach and empirical analysis, *Manufacturing & Service Operations Management*, 21(1), pp.66–85.
- Husna, A., Amin, S.H., Shah, B., 2021. Demand Forecasting in Supply Chain Management Using Different Deep Learning Methods. In *Demand Forecasting and Order Planning in Supply Chains and Humanitarian Logistics* (pp. 140–170). IGI Global.
- Khan, K.B., 2002. An exploratory investigation of new product forecasting practices. *Journal of Product Innovation Management*, 19(2), pp.133–143.
- Kök, A.G. and Fisher, M.L., 2007. Demand estimation and assortment optimization under substitution: Methodology and application. *Operations Research*, 55(6), pp.1001–1021.
- Pan, S. J., & Yang, Q. 2009. A survey on transfer learning. *IEEE Transactions on knowledge and data engineering*, 22(10), 1345–1359.
- Subramanian, S., Harsha, P., 2020. Demand modeling in the presence of unobserved lost sales, *Management Science*.
- Vulcano, G., Van Ryzin, G., Ratliff, R., 2012. Estimating primary demand for substitutable products from sales transaction data. *Operations Research*, 60(2), pp.313–334